

Edge AI on Constrained Devices: A Survey of Deployment, Optimization, and Lifecycle Management

MIRIANA PASSAROTTO, RISE Research Institutes of Sweden AB, Stockholm, Sweden and Uppsala University, Uppsala, Sweden

FEHMI BEN ABDESSLEM, RISE Research Institutes of Sweden AB, Stockholm, Sweden

JOAKIM ERIKSSON, RISE Research Institutes of Sweden AB, Stockholm, Sweden

THIEMO VOIGT, Uppsala University, Uppsala, Sweden and RISE Research Institutes of Sweden AB, Stockholm, Sweden

Edge Artificial Intelligence (AI) allows machine learning models to run directly on resource-constrained IoT and edge devices instead of relying only on cloud resources. This shift reduces latency and improves privacy but introduces strict constraints on memory, energy, and connectivity across heterogeneous sensor nodes. Deploying AI on constrained devices also requires addressing fragmented toolchains and inconsistent evaluation practices. This survey provides a structured review of deployment methods for edge AI, including model compilation, intermediate representations, hardware-aware optimization, scheduling strategies, and connectivity-aware execution. Post-deployment aspects are also examined, including lifecycle management, secure over-the-air updates, and benchmarking frameworks that combine latency, accuracy, and energy. Use cases in healthcare, smart cities, autonomous systems, manufacturing, and agriculture illustrate practical applications. Across these areas, common challenges in portability, reproducibility, adaptivity, and security are identified, along with emerging directions such as dynamic orchestration across heterogeneous devices and modular update mechanisms. By combining perspectives from compiler design, system optimization, and lifecycle management, this survey highlights how deployment pipelines are evolving into key enablers for reliable and scalable intelligence at the network edge.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Computing methodologies** → **Artificial intelligence**; • **Software and its engineering** → **Embedded software**; • **Hardware** → *Sensor devices and platforms*.

Additional Key Words and Phrases: Edge AI, TinyML, On-device Inference, Constrained Devices, Toolchains, Lifecycle Management

1 Introduction

Artificial Intelligence (AI) is no longer confined to centralized cloud infrastructures. Increasingly, AI capabilities are shifting toward the edge of the network, the physical locations where data is generated. This transition, commonly referred to as edge AI, is transforming the way intelligent systems operate by enabling low-latency processing, enhancing data privacy, and reducing bandwidth requirements. Edge AI is being adopted across various domains, including industrial automation, healthcare, smart cities, and autonomous systems [39, 226, 286].

At the hardware level, this transformation covers a diverse spectrum of IoT platforms. These range from ultra-constrained microcontrollers (MCUs) to system-on-chip (SoC) devices and single-board computers (SBCs)

Authors' Contact Information: Miriana Passarotto, RISE Research Institutes of Sweden AB, Stockholm, Sweden and Uppsala University, Uppsala, Sweden; e-mail: miriana.passarotto@ri.se; Fehmi Ben Abdesslem, RISE Research Institutes of Sweden AB, Stockholm, Sweden; e-mail: fehmi.ben.abdesslem@ri.se; Joakim Eriksson, RISE Research Institutes of Sweden AB, Stockholm, Sweden; e-mail: joakim.eriksson@ri.se; Thiemo Voigt, Uppsala University, Uppsala, Sweden and RISE Research Institutes of Sweden AB, Stockholm, Sweden; e-mail: thiem.voigt@angstrom.uu.se.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).

ACM 1558-3465/2026/6-ART

<https://doi.org/10.1145/3827611>

Table 1. Comparison of edge AI Surveys by Lifecycle Phase and Benchmark Coverage (✓ = explicit coverage, ✗ = absent coverage, - : partial coverage)

Survey	Year	Pre-Deployment	Deployment	Post-Deployment	Benchmarks	Key Contribution
Dutta et al. [63]	2021	✓	✗	✗	✗	TinyML on edge IoT devices
Surianarayanan et al. [232]	2023	✓	✓	✗	✗	Edge AI optimization survey
Pazmiño Ortiz et al. [188]	2025	✓	✓	✗	✗	System-level TinyML
Wang et al. [257]	2025	✓	✓	-	✗	On-Device AI Models
This Survey	2026	✓	✓	✓	✓	Deployment, Lifecycle, Toolchains

equipped with specialized AI accelerators. Despite their differences in scale and capability, all these platforms share a common goal: to sense, process, and interpret data locally, allowing AI-driven perception and decision-making at the extreme edge of the network [209, 221]. Bringing AI to the edge introduces significant challenges. AI architectures originally designed for cloud or data center environments often exceed the compute resources, memory, and energy budgets of embedded devices. For example, OpenAI’s Whisper large speech-recognition model comprises approximately 1.55 billion parameters and, according to official documentation, requires around 10 GB of GPU memory for inference, which far exceeds the capabilities of typical edge platforms [184].

Enabling AI deployment on constrained platforms requires navigating a complex set of trade-offs. In particular, developers must balance model size, memory organization, latency, and hardware-specific acceleration in order to fit AI workloads within limited resource budgets [86]. In addition, the diversity of hardware platforms, including differences in instruction sets, memory hierarchies, and available accelerators, further complicates cross-platform deployment and increases the need for specialized optimization workflows [137].

Recent surveys have addressed different facets of this emerging field. Singh et al. [226] and Zhou et al. [286] provide system-level perspectives on edge AI architectures and computing paradigms. Wang et al. [258] concentrate on trustworthiness aspects, including security and privacy, while Surianarayanan et al. [232] survey optimization techniques for TinyML and embedded learning. The integration of AI with IoT ecosystems has also been investigated, for example, by Siam et al. [225]. While these works offer valuable contributions, they primarily emphasize system design, specific algorithmic techniques, or trust/security challenges, rather than covering the complete end-to-end deployment lifecycle.

Table 1 summarizes recent surveys on edge AI for constrained devices, limited to works from the past five years. While these studies cover important aspects such as optimization techniques, TinyML pipelines, and deployment toolchains, they typically focus on the design and execution of models up to the inference stage. In contrast, post-deployment operation and long-term maintenance, such as Over-The-Air (OTA) updates, runtime monitoring, and lifecycle adaptation, remain largely unaddressed. To make this distinction explicit, we categorize contributions along three phases of the deployment lifecycle: pre-deployment (data preparation, architecture design, model training, and optimization), deployment (compilation toolchains, hardware mapping, and scheduling), and post-deployment (monitoring, OTA updates, runtime adaptation, performance evaluation, and benchmarking).

Despite growing interest in edge AI, the post-deployment operation of models in the field remains underrepresented. Beyond optimizing models for inference, production systems must remain maintainable, updatable, and observable. Prior work highlights that many edge deployments rely on static, pre-trained models that seldom adapt after deployment [188], and that on-device deployments face substantially more complex update and maintenance burdens than cloud-managed systems [257]. Moreover, operational requirements such as model version management, runtime monitoring/telemetry, and safe update workflows are central to real-world edge deployments [137]. These needs are particularly acute in safety- or mission-critical domains, such as healthcare and industrial automation, where reliability lapses or outdated models can pose significant risk [232, 258].

In recent years, the ecosystem of deployment tools has expanded rapidly. Modern pipelines now encompass model-level optimization, graph transformations at the compiler level, hardware-aware code generation, and runtime management tools for deployment and monitoring. These solutions aim to bridge the gap between AI research and real-world embedded systems. Yet the landscape remains fragmented: developers and practitioners must often navigate inconsistent hardware support, opaque performance trade-offs, and a lack of unified, scalable deployment workflows [86, 137].

While edge AI emphasizes local inference and autonomy, many real-world applications require a careful balance between edge and cloud processing. In domains such as wearable health monitoring [232, 258] or predictive maintenance in industrial IoT [188, 232], strictly on-device operation may be essential due to latency, connectivity, or privacy constraints. In contrast, other scenarios benefit from cloud assistance for tasks such as model updates, coordination, or compute-intensive inference [286].

At the same time, the demand for reliable and maintainable edge AI deployment continues to grow. According to IDC, worldwide spending on edge computing is projected to rise from about \$261 billion in 2025 to nearly \$380 billion by 2028, driven by applications that require real-time intelligence, data privacy, and autonomy at the edge [109]. Developing robust, scalable deployment tooling is therefore a key enabler for the next generation of AI applications across sectors.

Scope of the Survey. Figure 1 illustrates the three principal phases of the edge AI deployment lifecycle considered in this survey: pre-deployment covering data acquisition and preparation, model design and optimization; deployment covering compiler toolchains, hardware integration, and connectivity-aware workflows; and post-deployment covering in-field monitoring, over-the-air updates, and lifecycle management. The figure also maps representative tools and frameworks used in each phase, highlighting how the current ecosystem covers the complete edge AI workflow from data processing to long-term maintenance.

This survey, centered on literature from 2020-2025 and supported by earlier foundational work for context, aims to address the gap in deployment-focused literature. The specific contributions are as follows:

- (1) **Scope and Taxonomy:** We define the scope of edge AI deployment for IoT platforms, covering heterogeneous devices from ultra-low-power MCUs to high-performance SBCs. A taxonomy of deployment tools is presented, spanning the AI lifecycle from model design to long-term maintenance.
- (2) **Pre-deployment and Deployment Workflows:** We provide a comprehensive review of both academic frameworks and commercial toolchains, analyzing model optimization, compiler pipelines, hardware integration, and connectivity-aware workflows.
- (3) **Post-deployment Lifecycle:** We survey approaches for runtime monitoring, OTA updates, and lifecycle management, highlighting their importance for reliability and maintainability in constrained environments.
- (4) **Evaluation and Benchmarking:** Where available, we compare tool capabilities and discuss trade-offs in latency, memory footprint, energy efficiency, and deployment portability.
- (5) **Applications:** We illustrate practical relevance through application domains such as industrial monitoring, medical devices, and autonomous systems, showing how deployment complexity affects real-world use.
- (6) **Research Challenges and Open Research Questions:** We identify challenges and open research questions in modularity, usability, lifecycle management, deployment adaptivity, and security, and outline future directions for scalable, maintainable, and trustworthy edge AI deployment.

While Fig. 1 summarizes the broad landscape of tools and frameworks across the edge AI lifecycle, practitioners often approach deployment from a concrete bottleneck rather than from a lifecycle phase. To support this use case, Fig. 2 introduces a constraint-driven decision-support scheme that maps common deployment constraints to relevant lifecycle phases and methodology categories. The scheme is intended as an indicative guide rather than a prescriptive tool: concrete choices depend on hardware class, operator support, vendor SDKs, connectivity conditions, application requirements, and lifecycle constraints. Together with Fig. 1, it helps readers navigate the

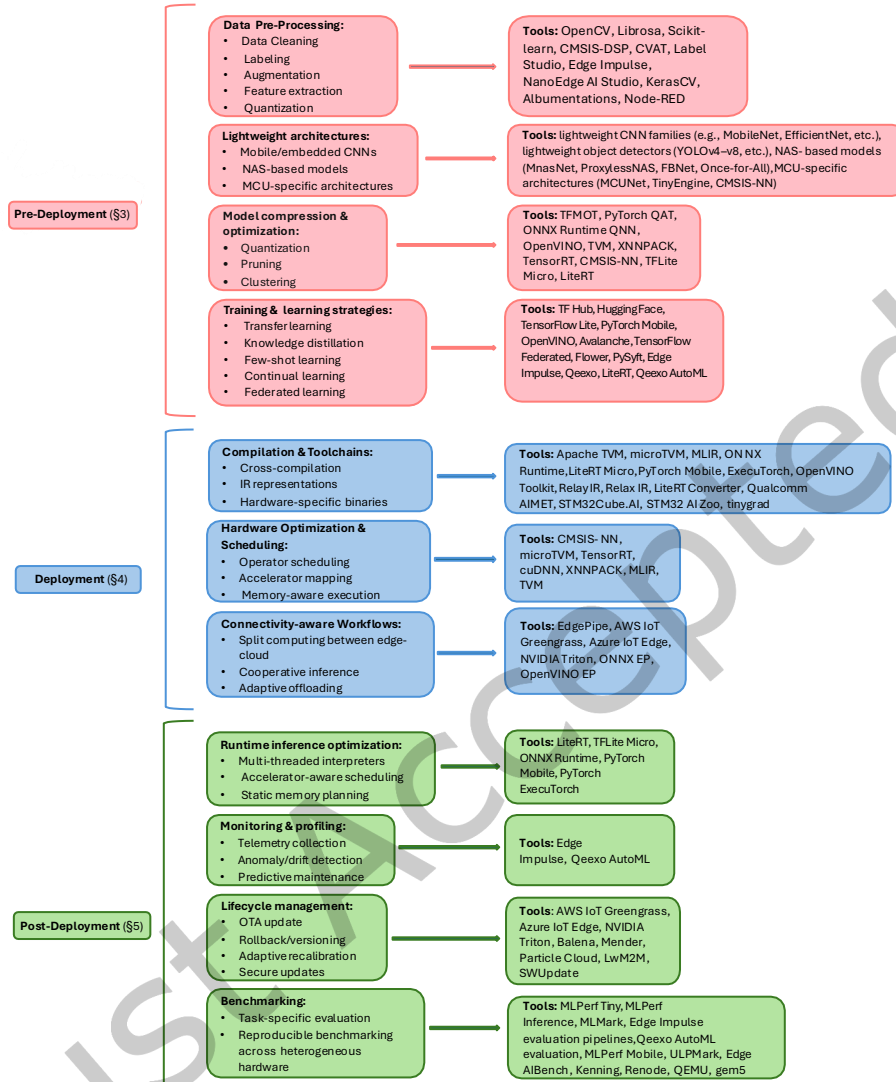


Fig. 1. Taxonomy of edge AI tools and frameworks across the pre-deployment, deployment, and post-deployment phases.

fragmented edge AI deployment landscape and identify the survey sections most relevant to their design problem. The techniques, hardware classes, and design concerns summarized in the figure are discussed in greater detail in the following sections.

To improve readability, Table 2 summarizes the acronyms frequently used in this survey.

The remainder of this paper is organized as follows. Section 2 reviews the evolution of edge AI along with hardware platforms and their performance constraints. Section 3 examines the pre-deployment phase, focusing on data preparation, model design, and optimization. Section 4 surveys deployment toolchains and connectivity-aware workflows, while Section 5 covers the post-deployment phase, including lifecycle management, OTA updates,

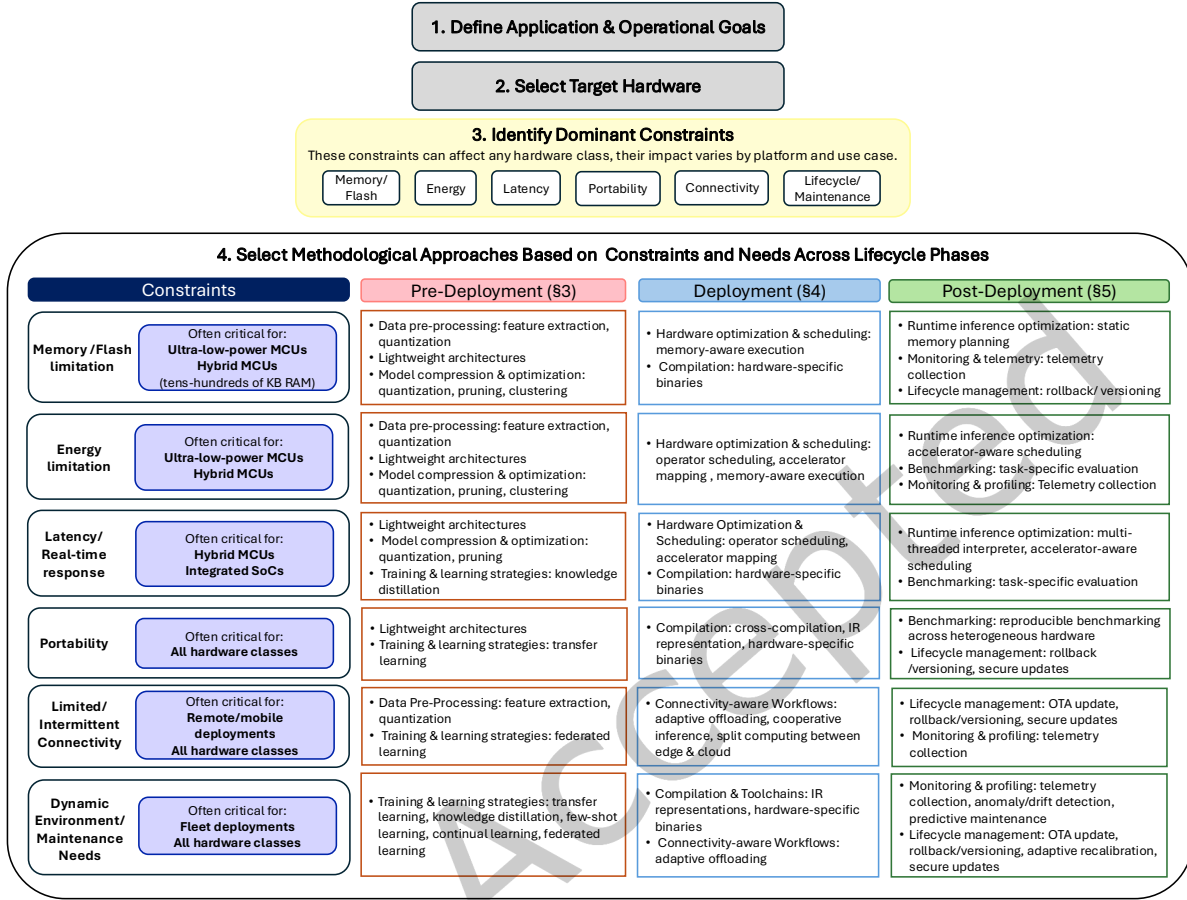


Fig. 2. The matrix maps dominant deployment constraints to relevant lifecycle phases and methodology categories from Fig. 1. Entries are indicative rather than prescriptive, as concrete choices depend on hardware, operator support, vendor SDKs, connectivity, and lifecycle requirements. Representative tools are listed in Fig. 1 and discussed in the survey tables.

and evaluation through performance metrics and benchmarking. Section 6 discusses cross-layer interactions across the edge AI lifecycle. Application domains are highlighted in Section 7, and open research challenges and future directions are outlined in Section 8. Finally, Section 9 concludes the paper.

2 The Evolution and Landscape of Edge AI for Constrained Devices

2.1 Historical Perspective

The evolution of edge AI spans from cloud-connected edge servers and fog nodes to intelligent devices at the extreme network edge. This survey focuses on IoT edge devices: sensing-capable platforms that perform AI inference locally. These range from low-power microcontrollers and system-on-chip designs to more capable single-board computers, all operating under constraints in compute, memory, energy, and latency. Unlike edge servers or gateways, such devices process data directly on board, enabling real-time and privacy-preserving

Table 2. List of Common Acronyms

Acronym	Definition
AI	Artificial Intelligence
CPU	Central Processing Unit
DSP	Digital Signal Processor
DNN	Deep Neural Network
FL	Federated Learning
GPU	Graphics Processing Unit
IoT	Internet of Things
IR	Intermediate Representation
MCU	Microcontroller Unit
MLIR	Multi-Level Intermediate Representation
MLPerf	Machine Learning Performance Benchmark Suite
NPU	Neural Processing Unit
ONNX	Open Neural Network Exchange
OTA	Over-the-Air update
SBC	Single-Board Computer
SDK	Software Development Kit
SoC	System-on-Chip
TPU/VPU	Tensor/Visual Processing Unit
TVM	Tensor Virtual Machine

intelligence. This section reviews the historical developments in hardware, software frameworks, and AI models that have enabled practical edge AI deployment on IoT edge devices. Key milestones are summarized in Table 3.

Conceptual Foundations: 1990s and Early 2000s. In the 1990s, the concept of moving computation closer to the physical world began to emerge, as researchers explored how intelligent systems could interact more directly with their environments. One important contribution was Mark Weiser’s vision of ubiquitous computing, which proposed that computation would become seamlessly integrated into everyday objects and environments [261]. A few years later, Schilit et al. introduced the concept of context-aware computing, where systems adapt their behavior based on contextual information such as location, nearby devices, and user activity. The term “Internet of Things” is attributed to Kevin Ashton, who introduced it in 1999 during a presentation at Procter & Gamble [169]. The concept was subsequently popularized by Gershenfeld, Krikorian, and Cohen in their Scientific American article “The Internet of Things” [80]. Cisco later quantified the practical emergence of IoT as the point when the number of connected devices surpassed the number of people, estimating that IoT was “born” between 2008 and 2009 [72]. These early developments, while not directly involving AI, established the foundational idea of pushing intelligence closer to the physical world, an idea that would later be enabled by advances in machine learning and efficient edge computing.

Early Edge Intelligence and Local Processing (2011-2014). Building on earlier visions of ubiquitous and context-aware computing, the early 2010s saw research and industry begin exploring architectures to bring computation closer to data sources, aiming to complement cloud computing with localized capabilities at the edge. Fog computing, introduced by Flavio Bonomi at VANET 2011 and formalized in 2012 [36], proposed a distributed layer between cloud and IoT devices, enabling geo-distributed, low-latency processing for real-time applications such as connected vehicles, smart grids, and sensor networks.

Industry initiatives like the Embedded Vision Alliance (2011) promoted the adoption of computer vision in embedded and resource-constrained products [68]. Meanwhile, ETSI launched the Mobile-edge Computing (MEC) industry initiative in 2014, later renamed Multi-access Edge Computing [71], to standardize application deployment at the mobile network edge, with use cases spanning IoT, video analytics, augmented reality, and local content distribution [70].

At the same time, conceptual frameworks such as the Wisdom Web of Things (2012) [87] envisioned augmenting IoT with embedded intelligence. Together, these developments marked the shift from connected IoT systems toward localized, intelligent processing at the network edge.

Enabling AI at the Edge (2015-2018). Between 2015 and 2018, advances in hardware, software, and algorithms transformed edge AI from architectural vision to practical deployment on IoT devices. Dedicated platforms such as NVIDIA’s Jetson TX1 (2015) [75] pioneered GPU-accelerated inference for computer vision in constrained environments, while the Raspberry Pi 3 (2016) [1] democratized prototyping by providing an accessible, low-cost platform.

In parallel, lightweight frameworks bridged cloud-trained models with edge deployment. TensorFlow Lite (2017) [3], now renamed as Lite Runtime (LiteRT), introduced model quantization and optimization for mobile and edge devices, while the Open Neural Network Exchange (ONNX, 2018) [2] enabled portability across heterogeneous hardware. On ultra-constrained platforms, ARM’s CMSIS-NN (2018) [134] brought efficient neural kernels to Cortex-M microcontrollers, extending AI to battery-powered IoT sensors.

The period also saw the emergence of specialized accelerators. ARM’s Project Trillium (2018) [4] and Google’s Edge TPU (2018) [53] delivered NPUs optimized for embedded inference. Meanwhile, efficient ML techniques such as the “Hello Edge” keyword spotting system [277] demonstrated that deep learning could run within 100 KB of memory, laying the foundation for the TinyML movement. Together, these developments established the technical feasibility of practical, localized AI across the edge spectrum.

Ecosystem Growth and First Mature Deployments (2019-2020). By 2019, edge AI had moved from feasibility to broader ecosystem adoption. NVIDIA’s Jetson Nano [5] provided affordable GPU-enabled inference for makers and researchers, lowering the barrier to prototyping computer vision and robotics applications at the edge. At the same time, Google’s TensorFlow Lite for Microcontrollers [207] extended deep learning to bare-metal microcontrollers with only kilobytes of memory, enabling on-device inference for ultra-constrained IoT sensors and marking a milestone for the emerging TinyML community.

In parallel, practical deployments began to validate the edge AI vision. Google’s Gboard integrated federated learning to train language models directly on smartphones, demonstrating privacy-preserving distributed training at a global scale [35]. Meanwhile, the emergence of 5G networks provided high-bandwidth, low-latency infrastructure to complement local inference [235]. Together, these developments marked the transition from isolated prototypes to deployable edge AI systems across diverse IoT devices.

Industrialization and Full-Stack Maturity (2021-2025). From 2021 onward, edge AI entered a phase of industrial consolidation, defined by the integration of hardware, toolchains, and lifecycle support into production-grade workflows. Unlike earlier stages centered on individual components, this period emphasized full-stack systems spanning model optimization, deployment, and long-term maintenance.

On the hardware side, system-on-chips increasingly embedded neural accelerators, exemplified by Google’s Tensor SoC (2021) for on-device speech, vision, and translation [83]. At the ultra-constrained end, ARM Cortex-M microcontrollers and low-power SoCs enabled practical inference in health monitoring, voice control, and predictive maintenance [24], extending applicability across both high-end and deeply embedded devices.

Table 3. Key milestones in the evolution of edge AI for IoT devices.

Period	Focus	Representative Milestones	Refs
1990s-2000s	Conceptual foundations	Ubiquitous computing; Context-aware systems; Internet of Things vision	[72, 80, 218, 261]
2011-2014	Early edge intelligence and local processing	Fog computing; MEC standardization; Wisdom Web of Things	[36, 68, 70, 87]
2015-2018	Enabling AI at the edge	First practical edge devices and accelerators; Lightweight frameworks (LiteRT, ONNX, CMSIS-NN); Google Edge TPU	[1–3, 53, 134, 181]
2019-2020	Ecosystem growth and first mature deployments	TinyML Summit; TensorFlow Lite Micro; Federated Learning in GBoard; Initial 5G rollouts	[5, 35, 153, 207, 235]
2021-2025	Industrialization and full-stack maturity	Google Tensor SoC; Edge Impulse, Qeexo AutoML; PyTorch Mobile, ExecuTorch; MLPerf Tiny; MLIR/TVM Unity	[17, 24, 65, 83, 136, 193–195]

Full-stack toolchains such as Edge Impulse [65] and Qeexo AutoML [195] matured into unified platforms covering data ingestion, training, quantization, deployment, and telemetry for targets ranging from microcontrollers to Linux-based SBCs. Complementary runtimes, including LiteRT, TFLite Micro, microTVM [49], PyTorch Mobile [193], ONNX Runtime [2], and more recently PyTorch ExecuTorch [194], focus on portability and efficient execution, without broader lifecycle coverage.

During this period, system-level expectations grew around resilience, scalability, and maintainability [46, 64, 242], making features such as remote updates, telemetry-aware deployment, and continuous lifecycle management standard. Security and privacy were also prioritized, with techniques including model watermarking [142], secure boot with encrypted models [19], and federated learning for privacy-preserving training [35, 138], particularly in sensitive domains such as healthcare and industrial control.

Standardization and benchmarking have progressed via MLPerf Tiny, which provides open, reproducible TinyML benchmarks designed to compare systems fairly across diverse hardware/software stacks [24]. In parallel, compiler and runtime standardization for portability across heterogeneous targets has advanced through MLIR and the TVM Unity unification effort [17, 136]. At the systems level, hybrid edge-cloud designs combine local inference with cloud/edge coordination for federated model updates and service orchestration, alongside online methods for resource management [59]. Emerging patterns include multi-node/pipelined inference across heterogeneous edge devices, runtime-aware resource allocation, and online learning-style coordination [59, 101].

Between 2021 and 2025, edge AI has moved from demonstrating feasibility to achieving ecosystem-level maturity. It is now established as a mainstream design approach, supported by standardized workflows, scalable development workflows, and integration across hardware platforms and application domains.

2.2 Why Tools Matter in End-to-End Edge AI Deployment

The deployment of machine learning on edge devices spans a complex, multi-stage workflow that introduces unique challenges at each step. Unlike conventional cloud-based machine learning pipelines, edge AI workflows must address more stringent constraints on memory, energy consumption, computational resources, and update mechanisms [59]. Structured, end-to-end toolchains offer considerable advantages in streamlining the deployment of edge AI systems, particularly in managing the inherent complexity of multi-stage workflows. Without such tools, developers must contend with highly fragmented processes that demand manual intervention, platform-specific customization, and low-level optimization, significantly increasing development time and reducing portability [24, 86].

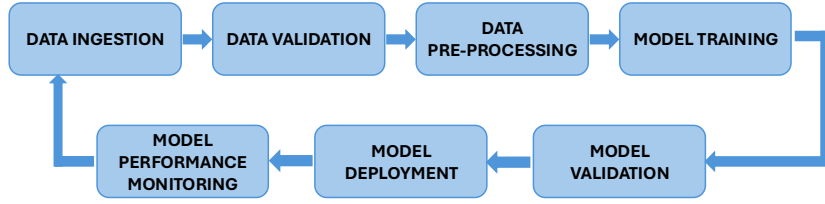


Fig. 3. Simplified representation of a typical edge AI workflow, showing the sequential stages from data collection and preprocessing to model deployment and post-deployment monitoring.

A typical end-to-end edge AI workflow follows a structured sequence of stages, from data acquisition to model deployment and monitoring. Figure 3 presents a simplified example of such a pipeline, highlighting the essential stages of the machine learning lifecycle: from data collection to model performance monitoring after deployment. While real-world deployments often involve additional optimization and feedback mechanisms, this simplified view illustrates the fundamental progression that most edge AI toolchains are designed to support.

This survey organizes the workflow into three interconnected stages: (i) pre-deployment, encompassing data preparation, model design, and training; (ii) deployment, covering optimization and compilation; and (iii) post-deployment, focused on inference, monitoring, and lifecycle management. The following sections expand on each stage in detail.

Pre-Deployment: Data Preparation, Model Design, and Training. The pre-deployment phase establishes the foundation for successful edge AI deployment by aligning both the data and the model with the constraints of the target environment. These constraints, such as memory footprint, latency, and power consumption, require careful choices before deployment [256]. Without dedicated tools, this phase often becomes fragmented, relying on manual preprocessing steps, ad-hoc model adaptation, and hardware-specific trial-and-error [136, 219]. End-to-end tool support provides standardized workflows that reduce measurement variability, improve reproducibility, and shorten development cycles [24].

On the data side, representative datasets must be acquired to capture the operational variability of the deployment context [246]. In many cases, this requires direct collection from the intended edge environment, where sensing hardware may have limited preprocessing capabilities. Tools for data cleaning, labeling, and augmentation, whether manual, semi-automated, or fully automated, are designed to preserve signal quality, mitigate noise, and produce balanced datasets that reduce bias while reflecting real-world conditions [246, 256]. Feature extraction and quantization further reduce dimensionality and resource usage [257].

On the model side, design is guided by the need to achieve competitive accuracy under strict resource budgets. At the architectural level, this can involve selecting lightweight families, adapting existing models, or applying hardware-aware automated search methods tailored for low-resource devices [42, 98, 237]. At the optimization level, model compression techniques, such as pruning, quantization, and weight clustering, reduce memory footprint and computational load while retaining accuracy [92]. Toolkits that automate these steps enable deployment even on devices with tight energy and storage constraints [143]. Training-time and pre-deployment optimizations, including quantization-aware training, transfer learning, knowledge distillation, and federated learning, are increasingly supported by integrated frameworks [84, 121], which adapt models to specific hardware profiles and operational conditions while helping preserve accuracy under edge limitations.

Deployment: Model Optimization, Compilation, and Runtime Execution. The deployment phase adapts trained models into efficient, executable artifacts for resource-constrained edge devices. Beyond algorithmic innovation, this stage depends heavily on toolchains that automate hardware-specific adaptation and streamline the transition

from abstract model design to device-ready binaries. Without such tools, deployment often involves error-prone manual optimization, fragmented compilation workflows, and repeated hardware-specific trial-and-error [136, 219]. Integrated deployment frameworks instead provide reproducibility, portability, and scalability across heterogeneous hardware classes [49, 136, 205].

On the optimization side, frameworks apply transformations such as pruning, quantization, compression, kernel fusion, and operator reordering to reduce memory footprint, energy use, and latency while maintaining acceptable accuracy [49, 232]. These steps are often guided by profiling and hardware feedback loops that expose performance bottlenecks, enabling the deployment pipeline to adapt to device-specific resource limits.

On the compilation side, toolchains translate optimized models into hardware-compatible binaries or intermediate representations, frequently relying on cross-compilation and backend-specific code generation. By abstracting away device-level details, such compilers allow developers to deploy the same model family across ultra-low-power MCUs, NPUs, and high-performance SBCs with minimal re-engineering [49, 134]. This hardware abstraction is critical for ensuring both portability and efficiency, enabling unified deployment workflows even in diverse, multi-vendor environments.

Beyond static optimization and compilation, deployment also involves runtime-aware resource management [234, 278]. In heterogeneous edge systems, execution efficiency depends on how deployed workloads are mapped, scheduled, and adapted under device-specific constraints such as memory, latency, energy, and connectivity [101, 286]. These concerns link deployment toolchains with the runtime and system-level strategies discussed in the following subsections.

The following subsections analyze these deployment dimensions in detail, with emphasis on how tools support: (i) optimization techniques such as quantization, pruning, compression, and kernel fusion; (ii) compilation workflows that bridge high-level model definitions with low-level device execution across heterogeneous platforms; and (iii) hardware-, runtime-, and connectivity-aware execution strategies for efficient deployment.

Post-Deployment: Inference, Monitoring, and Lifecycle Management. The post-deployment phase ensures that deployed models remain efficient, reliable, and secure throughout their operational lifetime. Unlike optimization and compilation, which end once a model artifact is produced, this phase requires continuous oversight and adaptation. Without dedicated tools, sustaining performance often relies on ad-hoc monitoring, manual updates, and fragmented maintenance processes, which limit scalability and reproducibility [64, 137]. Tool-supported runtime and lifecycle management instead provide standardized infrastructures that automate inference scheduling, monitoring, and model updating across heterogeneous devices.

At the runtime level, inference frameworks configure execution according to application-specific goals such as ultra-low latency, energy efficiency, or constrained memory usage [134, 207]. By exploiting available hardware accelerators, supporting multi-threaded interpreters where available, and applying static memory planning with efficient kernel execution, these environments maximize performance under strict device limitations.

For monitoring and profiling, dedicated tools collect telemetry data to detect performance drift, resource bottlenecks, and anomalous behavior during operation [178, 232]. Continuous visibility into deployed systems not only validates compliance with design requirements but also supports predictive maintenance and adaptive tuning. STM32Cube.AI, together with STM32CubeMonitor-AI, provides runtime performance measurement and energy-profiling capabilities for STM32-based devices, enabling developers to evaluate inference efficiency and optimize firmware-level performance under real operating conditions. Together with other runtime environments such as Edge Impulse and Qeexo AutoML, these frameworks illustrate how deployment toolchains are evolving to include post-deployment visibility and optimization capabilities.

Lifecycle management frameworks extend this feedback loop by enabling over-the-air (OTA) updates, integration of newly collected training data, and iterative model improvements [64, 137]. By automating update delivery and retraining pipelines, these tools allow systems to adapt to hardware heterogeneity, shifting workloads, and

Table 4. Classification of Hardware Platforms for Constrained Edge AI

Class	Example Platforms	Characteristics	Typical Use Cases
High-End Devices	NVIDIA Jetson Nano NVIDIA Jetson Xavier NX NVIDIA Jetson Orin Nano Raspberry Pi 4 Google Coral Dev Board	• Multi-core CPUs, GPUs, or TPUs • >1 GB RAM • Minimal model compression • Real-time, multi-modal edge AI	Smart surveillance Industrial robotics Autonomous navigation AI gateways for lower-tier devices
Integrated SoCs	Google Tensor Kendryte K510 NXP i.MX 8M Plus	• CPUs with on-chip NPUs/DSPs • 512 MB-several GB RAM • Hardware-accelerated inference • Mobile-class edge intelligence	Mobile vision Smart cameras AR/VR inference Consumer wearables
Hybrid MCUs	STM32U5, STM32 N6 ESP32-S3 MAX78000 NXP i.MX RT1170	• MCU cores with DSP, graphics, vector, or AI acceleration support • 256 KB-10 MB RAM • Low-latency inference • Energy-efficient design	Voice assistants Predictive maintenance AI-enabled IoT nodes Embedded vision
Ultra-Low-Power MCUs	STM32L4 ARM Cortex-M0+ nRF52840 Ambiq Apollo4 Plus	• Bare-metal or lightweight MCUs • Tens-hundreds of KB RAM • Always-on sensing, simple inference • Energy-constrained environments	Keyword spotting Gesture recognition Environmental sensing Wearable health trackers

evolving performance requirements, ensuring that edge AI deployments remain robust and trustworthy over time [46].

Finally, post-deployment evaluation should account not only for isolated inference performance, but also for end-to-end pipeline costs, security overheads, update granularity, and multi-objective trade-offs among accuracy, latency, energy, memory, and maintainability [24, 137, 201].

2.3 Hardware Platforms for Constrained Edge AI

This survey focuses on resource-constrained edge devices that perform local sensing and on-device inference at the edge of the network. These platforms span from microcontrollers to single-board computers (SBCs) with integrated AI accelerators while explicitly excluding higher-tier edge platforms such as cloudlets, fog nodes, and edge servers, which are beyond the scope of this review.

Additionally, highly specialized hardware platforms such as FPGA-based systems and ASIC-edge accelerators are acknowledged for their emerging relevance in ultra-low-latency and ultra-low-power edge AI deployments. However, these platforms typically require custom hardware design, vendor-specific toolchains, or bespoke synthesis workflows that fall outside the general-purpose, software-centric deployment pipelines emphasized in this survey. As such, they are excluded from the hardware classification presented here.

General-Purpose Hardware Classes for Edge AI

Edge AI deployments across this constrained spectrum rely on a diverse range of hardware platforms, each offering distinct trade-offs in compute capability, memory capacity, energy efficiency, and deployment complexity. These hardware classes also align with specific application domains, where constraints and performance priorities vary widely. Based on these characteristics and practical deployment trends, we classify edge AI platforms into four primary categories that reflect both current use cases and the evolution of modern edge systems.

The classification in Table 4 reflects practical deployment categories used in edge AI, focusing on platforms that balance on-device intelligence with resource constraints. These categories, ranging from high-end edge devices to ultra-low-power MCUs, highlight distinct trade-offs in compute power, memory capacity, energy consumption,

and system complexity. It should be noted, however, that this taxonomy is not exhaustive nor mutually exclusive: many platforms exhibit hybrid characteristics that span across categories. The table summarizes major deployment trends, rather than a rigid partition of all possible hardware platforms.

High-End Devices. Platforms such as the NVIDIA Jetson Nano, Jetson Xavier NX, Google Coral Dev Board, and Raspberry Pi 4 represent high-end edge systems capable of running relatively complex models with minimal compression. These devices typically feature multi-core CPUs, integrated GPUs or TPUs, and at least 1 GB of RAM. They support real-time, multi-modal inference and are commonly used in robotics, autonomous navigation, and smart surveillance. In many deployments, they also act as AI gateways, aggregating data, performing inference, and coordinating multiple lower-tier edge nodes.

Integrated SoCs. This category includes AI-capable System-on-Chip platforms like Google Tensor, NXP i.MX 8M Plus, and Kendryte K510. These devices combine general-purpose CPUs with on-chip AI accelerators such as NPUs or DSPs. With memory capacities ranging from hundreds of MBs to several GBs, they support hardware-accelerated inference for tasks like mobile vision, Augmented Reality/Virtual Reality, and intelligent cameras. Their power-performance balance makes them well-suited for consumer-facing applications that require local processing under moderate energy constraints.

Hybrid MCUs. Hybrid MCU platforms combine low-power microcontroller cores with heterogeneous acceleration support, including DSP, vector, graphics, or dedicated neural processing hardware. Platforms such as the ESP32-S3 and STM32U5 primarily rely on vector or DSP acceleration, whereas devices such as the STM32 N6 and MAX78000 integrate dedicated NPUs or CNN accelerators for neural inference. These systems typically offer 256 KB to 10 MB of RAM and target low-latency, energy-efficient edge AI workloads such as voice interfaces, industrial monitoring, embedded vision, and battery-powered AI sensor nodes.

Recent benchmarking studies have highlighted the advantages of MCU-NPU hybrid architectures for TinyML inference workloads. By combining low-power MCU operation for sensing and control tasks with specialized neural inference hardware, these systems achieve substantially improved latency and energy efficiency compared to general-purpose MCUs [166]. Experimental results further show that platforms without neural accelerators exhibit significantly lower inference efficiency than devices equipped with dedicated NPUs or CNN accelerators [166]. Examples of such hybrid devices include the STM32 N6 and MAX78000, which integrate neural accelerators alongside MCU cores for sensing and peripheral management.

Ultra-Low-Power MCUs. Examples such as the STM32L4, Ambiq Apollo4 Plus, ARM Cortex-M0+, and nRF52840 represent the most constrained class of edge devices. With RAM capacities typically limited to tens of kilobytes, these platforms are designed for always-on sensing and extreme energy efficiency. They can only accommodate compact, heavily quantized models, making them suitable for tasks such as keyword spotting, gesture recognition, and environmental monitoring. Deployment at this scale relies on lightweight inference frameworks such as TensorFlow Lite Micro and CMSIS-NN, which produce small-footprint binaries and are optimized for efficient execution on microcontrollers.

Across these classes, memory fragmentation, power constraints, and hardware heterogeneity remain persistent challenges. Deployment toolchains play a critical role in bridging this diversity by offering cross-platform support, static memory planning, and hardware abstraction. In this way, the classification in Table 4 not only highlights the trade-offs among hardware platforms but also sets the stage for understanding how software ecosystems adapt to, and mitigate, the constraints imposed by edge hardware.

Specialized and Emerging AI Hardware Platforms. Beyond general-purpose MCUs and SoCs, a number of specialized hardware architectures have emerged that target ultra-low-latency or ultra-low-power AI inference. These include

neuromorphic processors and custom ASICs, which offer promising capabilities but remain peripheral to this survey due to limited integration with mainstream deployment toolchains.

Neuromorphic Processors. Architectures like Intel’s Loihi 2 [56] and BrainChip’s Akida [110] implement spiking neural networks (SNNs) that process information using asynchronous, event-based computation. These chips are designed for sparse, temporal data and can achieve significant energy savings for workloads like gesture recognition, olfactory sensing, or auditory processing. Loihi 2 supports programmable neuron models and a scalable mesh architecture, while Akida targets edge applications through its on-chip learning capabilities and ultra-low power envelope.

FPGAs and Custom ASICs. While not the focus of this survey, Field-Programmable Gate Arrays (FPGAs) and application-specific integrated circuits (ASICs) also play a role in edge inference, particularly when strict latency or power budgets must be met. Examples include low-power inference accelerators synthesized from RTL descriptions, or optimized ML workloads on platforms like Xilinx Zynq or Intel Cyclone V [190]. However, these solutions generally require vendor-specific design flows, such as Vivado or Vitis AI [111], and are less accessible to non-specialist developers.

Despite their potential, these specialized platforms lack broad support in mainstream ML frameworks (e.g., LiteRT, ONNX Runtime, TVM). Most rely on proprietary APIs, non-standard model representations, or hardware-specific compilation flows, which limit portability and integration with general-purpose deployment pipelines. As a result, they are currently more common in research prototypes.

2.4 Edge-Cloud Trade-offs and Connectivity Considerations

The growing proliferation of IoT devices and the increasing demand for real-time AI applications have catalyzed a shift toward distributed intelligence at the network edge. While cloud computing offers virtually unlimited computational power and centralized data management, relying solely on cloud resources introduces significant limitations in scenarios that require low-latency responses, strong privacy guarantees, or operate under bandwidth constraints [286].

Pushing AI workloads closer to data sources, a concept often referred to as edge AI (or Edge Intelligence, EI), seeks to address these constraints by executing inference and, in some cases, parts of the training process at edge nodes [232, 286]. On-device inference provides latency benefits and improves data privacy by reducing external transmissions. However, the constrained compute and memory resources of edge devices impose strict limits on model size and complexity, often necessitating compression techniques such as quantization and pruning [59, 222].

Conversely, cloud-based inference enables more sophisticated models but suffers from network-induced latency and reliability issues. Transmitting large volumes of raw sensor data to cloud datacenters is often infeasible in terms of bandwidth cost and response time, particularly in applications like autonomous vehicles, smart surveillance, and industrial monitoring [59, 286].

To bridge this trade-off, hybrid edge-cloud architectures have emerged as a prevalent solution, where workloads are dynamically partitioned based on resource availability, latency requirements, and network conditions [232]. Hierarchical computing models further distribute computation across devices, edge servers, and cloud datacenters to balance workload, data locality, and energy efficiency [216]. The taxonomy of edge intelligence levels introduced by Zhou et al. [286] explicitly categorizes systems based on the degree of computation performed at the edge versus the cloud, highlighting the absence of a single solution.

Connectivity variability further complicates deployment strategies. Fluctuating bandwidth and intermittent links in remote deployments mandate adaptive offloading strategies [222]. Meanwhile, the need for privacy-preserving computation (e.g., in healthcare or defense applications) reinforces the importance of context-aware

edge-cloud partitioning methods [59]. As a result, model architectures, partitioning methods, and deployment pipelines must be designed with edge-cloud collaboration in mind from the earliest development stages. This interdependence between system architecture and deployment environment directly impacts pre-deployment decisions such as model design, resource allocation, and lifecycle management (see Section 5).

2.5 Performance Constraints, Trade-offs, and Metrics

Deploying AI models on edge devices requires careful consideration of performance constraints and trade-offs that are often more restrictive than those encountered in cloud or desktop environments, as highlighted both by benchmarking efforts such as MLPerf Tiny [24] and by recent survey studies [226].

Key performance metrics for edge AI include inference latency, typically measured in milliseconds or frames per second, which is critical for real-time responsiveness [24, 232]. Energy consumption is another central metric, often quantified as millijoules per inference, particularly in battery-powered devices such as wearables and remote sensors [86]. Model accuracy remains essential, although some loss may be acceptable when optimizing for latency or energy [232]. Memory usage, encompassing both RAM and on-device storage, directly constrains deployability on microcontrollers and other low-resource devices. Trade-offs are inevitable: improvements in accuracy typically incur higher latency and energy consumption on resource-constrained devices [86]. Improving throughput on edge often relies on model compression and hardware-aware tuning, which boost speed but can reduce model size and lock deployments to specific accelerators, limiting portability [232].

Different applications emphasize different metrics: wearables prioritize energy efficiency and minimal memory footprint [137], drones and robotics demand ultra-low-latency inference for real-time control loops [226], while industrial systems often balance accuracy with continuous uptime under noisy conditions [86]. These trade-offs are revisited in Section 5.3, where latency, energy, accuracy, memory footprint, and model size are discussed as interdependent constraints rather than isolated metrics.

One persistent challenge is the lack of standardized benchmarks that fully capture the operational constraints of edge and embedded devices [24, 137]. Inference performance cannot be fully described by model size alone; it must account for latency, memory access patterns, energy efficiency, and real-time responsiveness [232]. Benchmarking methodologies are discussed in detail in Section 5.4.

3 Pre-Deployment Phase: Data, Design, and Optimization

This section reviews the key stages of preparing machine learning models for deployment on resource-constrained edge devices, covering data acquisition and preprocessing, model design, and training under device constraints, with emphasis on efficiency, compatibility, and reliability.

3.1 Data Preprocessing

Pre-deployment pipelines in edge AI involve preprocessing raw sensor and media streams and optimizing models to enable lightweight on-device inference [232, 257]. The computational budget of the target hardware strongly constrains the scope of on-device data analytics. While high-end edge SoCs can sustain real-time feature extraction [73], hybrid microcontrollers are often limited to lightweight signal processing [117]. For instance, Ulkar et al. report that extracting spectral audio features on a Cortex-M4F MCU consumes $826\mu\text{J}$, over three times the $251\mu\text{J}$ required for complete inference on the MAX78000 CNN accelerator [250]. This has led many systems to either offload preprocessing to upstream gateways [62] or to train models directly on raw input [167].

Unless noted otherwise, we focus on methods that can be executed entirely on the target device within its compute, memory, and energy budgets.

Data Cleaning and Normalization. Data cleaning, which removes noise, errors, and inconsistencies from collected datasets, is essential for maintaining inference accuracy under constrained compute and memory

budgets [29, 173]. Data consistency and quality have likewise been identified as fundamental requirements in edge intelligence systems [267]. Common operations include missing-value imputation, outlier filtering, and denoising [14, 76], followed by normalization to improve numerical stability and model convergence. On Python-capable platforms, these tasks are often performed with domain-oriented libraries such as OpenCV for vision, Librosa for audio, or Scikit-learn for general ML preprocessing [54, 55, 163]. On MCU-class devices without Python support, optimized libraries such as CMSIS-DSP are employed [20, 131].

Data Labeling and Annotation. Labeling assigns semantic meaning to raw data and remains essential for supervised learning. Manual annotation is among the most costly stages of this process [69], a challenge further amplified in edge AI domains where data is often diverse and task-specific. Open-source tools such as CVAT [220] and Label Studio [243] support annotation for images, video, and sensor data, and can be integrated into edge-oriented pipelines such as Edge Impulse [107]. These tools generally operate off-device, providing annotation environments that interface with edge ML workflows rather than running directly on constrained hardware. Recent advances include hardware-aware annotation pipelines that run directly on AI cameras, significantly reducing bandwidth and storage requirements. For example, Mori et al. [168] achieved over 93% transmission reduction and 92% throughput improvement compared to manual bounding-box labeling. Methods like Co-active [97] selectively relabel high-impact samples, boosting accuracy by 18.67% while reviewing only 10% of data. Semi-supervised federated learning approaches such as SemiPFL [239] alleviate labeling costs by combining small labeled datasets with abundant unlabeled data, a paradigm that is gaining attention in edge AI research.

Data Augmentation, Balancing, and Aggregation. Data augmentation, balancing, and aggregation are key techniques for addressing limited and skewed datasets in edge AI deployments. Augmentation generates synthetic variability from existing samples [102, 272], balancing mitigates class skew through oversampling, undersampling, or distributed resampling [102, 247], and aggregation combines information from multiple sources to enhance coherence and reduce redundancy [273].

In resource-constrained settings, augmentation must be computationally lightweight, as demonstrated in on-chip learning studies where simple transformations are applied directly on edge hardware [273]. Speech applications benefit from environment-aware augmentation, such as reverberated playback or injected background noise, which can reduce false rejection rates by up to 47% and improve MCU-level accuracy by 14% [52, 197]. Image augmentations can leverage OpenCV [54] and Albumentations [41], while audio transformations use Librosa [163]; MCU-class devices rely on CMSIS-DSP [20] or integrated blocks in platforms such as Edge Impulse [107]. While augmentation improves generalization, excessive or unrealistic transformations risk introducing bias or degrading accuracy [213].

Unlike augmentation, which introduces synthetic variation, aggregation addresses the challenge of massive and distributed IoT data by consolidating measurements across devices, sensors, or sources. This process reduces redundancy, improves spatial and temporal coverage, and yields more representative datasets, while also enhancing efficiency in bandwidth and energy usage [272]. Common frameworks include Flower [30] for federated model aggregation, and Node-RED [182] for integrating and managing IoT data from multiple sources. MCU-class devices may employ CMSIS-DSP statistical functions or the Edge Impulse Data Forwarder for lightweight pre-aggregation prior to cloud transmission. Federated learning extends this concept by enabling aggregation without raw data centralization, preserving privacy while leveraging distributed information [177]. Overly complex or centralized methods, however, can add latency and communication overhead, underscoring the need for lightweight and distributed aggregation tailored to real-time, energy-constrained deployments.

Feature Extraction. Feature extraction reduces data dimensionality to highlight the most informative aspects of the input, a crucial step for edge devices where direct processing of high-dimensional raw data is infeasible [48].

Table 5. Representative tools and methods for data preprocessing in edge AI workflows.

Task	Methods	Frameworks	Libraries / Tools
Data Cleaning and Normalization	Missing-value imputation; filtering; denoising; feature scaling	Edge Impulse; NanoEdge AI Studio	CMSIS-DSP; OpenCV; Librosa; Scikit-learn
Data Labeling and Annotation	Manual or semi-automatic labeling; assisted annotation	Edge Impulse; PT-Mobile	CVAT; Label Studio
Data Augmentation and Balancing	Cropping; flipping; noise injection; pitch shifting; resampling	Edge Impulse; KerasCV	Albumentations; CMSIS-DSP; Librosa; OpenCV
Feature Extraction	Spectrograms; MFCC; texture metrics; geometric descriptors; statistical measures	Edge Impulse; MediaPipe	CMSIS-DSP; Librosa; OpenCV; Scikit-image
Data Quantization	Uniform / non-uniform discretization; adaptive bit-depth reduction	Edge Impulse	CMSIS-DSP

In vision tasks, methods range from YOLOv8-based multi-scale feature extraction with multi-object tracking [285], to attention-based spatial and multi-scale frequency pooling for facial expression recognition [264], and texture metrics for plant disease detection [249]. For signal and time-series data, approaches include waveform statistics, spectral energy bands, and ECG morphology for arrhythmia detection [196], while semantic decomposition has been applied to improve communication efficiency in federated forecasting [279].

Implementations vary by hardware: CMSIS-DSP provides optimized routines for MCU-class platforms, and modular feature-extraction blocks are available in toolchains such as Edge Impulse. While effective, feature extraction pipelines carry the risk of discarding critical information if not carefully designed, underscoring the importance of domain knowledge and hardware-aware toolchains [211].

Data Quantization. Data quantization reduces the numerical precision of sensor readings, features, or intermediate representations, lowering bandwidth, storage, and computation costs in resource-constrained deployments [93, 100].

Typical approaches include uniform quantization, which divides the input range into equal intervals, and non-uniform schemes that allocate higher precision to frequently occurring values [100]. Adaptive quantization further adjusts bit-depth dynamically based on signal variability, making it well-suited for applications such as environmental sensing where activity levels fluctuate [100, 148].

Support for data quantization is commonly found in embedded signal-processing libraries and edge toolchains. For example, CMSIS-DSP provides fixed-point conversion routines for MCU-class platforms, while Edge Impulse integrates quantization into its DSP preprocessing blocks to reduce memory and communication overhead.

While data quantization can provide substantial gains in efficiency, overly aggressive precision reduction risks introducing noise, obscuring fine-grained patterns, and degrading model accuracy [93].

The five stages outlined above form the foundation of pre-deployment workflows for edge AI. Table 5 summarizes representative tools, platforms, and methods, with downstream implications for model design, compression, and deployment efficiency.

3.2 Lightweight Architectures

Designing AI models for edge deployment demands strategies that address hardware limitations from the architectural stage [98, 237, 257]. Instead of retrofitting generic models, edge workflows employ efficiency-oriented

architectures, constraint-aware optimization, and data-efficient training techniques. Lightweight architectures form the foundation of edge AI because they offer favorable accuracy-efficiency trade-offs under strict compute, memory, and energy budgets. They can be grouped into four broad categories. Many of these models are supported by deployment toolchains such as LiteRT, PyTorch Mobile, or vendor SDKs, which provide optimized kernels and reference implementations for constrained devices.

General-Purpose Lightweight CNN Families. These models provide the backbone of mobile and embedded vision systems and serve as widely adopted baselines. The MobileNet series [98, 99, 212] exemplifies this trajectory: V1 reduced multiply-adds nearly tenfold with depthwise separable convolutions, V2 introduced inverted residuals with linear bottlenecks, and V3 combined Neural Architecture Search (NAS) with attention for higher accuracy at lower latency. Other notable families include SqueezeNet [108], ShuffleNet [157, 276], EfficientNet [237], and GhostNet [90], the latter explicitly exploiting redundancy in feature maps for improved efficiency. While these networks provide strong baselines for mobile and embedded deployment, scaling them down to MCU-class hardware typically requires additional compression or quantization, and their reduced capacity may limit performance on tasks that demand fine-grained feature discrimination.

Object Detection Lightweight Models. Lightweight object detectors adapt high-accuracy frameworks to the strict memory, latency, and energy budgets of edge devices. The YOLO family has become a leading baseline: YOLOv4 introduced a backbone with cross-stage partial connections for efficient real-time detection; YOLOv5 added streamlined variants (nano, small) that reduce model size to a few megabytes while retaining practical accuracy for edge deployment; and YOLOv8 moved to an anchor-free head with refined feature fusion, improving small-object performance while sustaining real-time operation on constrained platforms [34, 106].

As an alternative, EfficientDet [238] builds on EfficientNet backbones for compact feature extraction and introduces a bi-directional feature pyramid network that reuses multi-scale features with lightweight top-down and bottom-up connections. These design choices allow EfficientDet to balance accuracy and latency more effectively than prior detectors, making it competitive on mobile and embedded platforms.

More recent designs, such as YOLOv10n [252] and YOLOv11n [124], further refine detection heads and scaling, though their adoption in edge toolchains is still limited, with most practical deployments relying on YOLOv4-v8 and EfficientDet-Lite as established baselines.

NAS-Based Models. Neural architecture search automates the co-design of architectures and deployment constraints. FBNet [262] pioneered differentiable NAS with a latency-aware loss based on real device measurements. ProxylessNAS [43] reduced the memory footprint of NAS itself through path-level binarization, enabling direct optimization of accuracy and device latency in large search spaces. MnasNet [236] introduced a reinforcement-learning controller with a multi-objective reward that explicitly balances accuracy and real-world latency on mobile devices. Together, these approaches illustrate how NAS methods have evolved from abstract efficiency proxies to direct optimization against on-device performance. Once-for-All networks [42] extend this line of work by training a single super-network that supports elastic kernel sizes, depths, and widths, from which specialized subnetworks can be extracted on demand for heterogeneous edge devices. This decouples model training from architecture search, reducing the cost of adapting models to new latency or hardware constraints. However, training the super-network itself is computationally intensive, which can limit rapid adoption in practice.

Specialized MCU/IoT Models. Extreme resource constraints in sub-megabyte RAM microcontrollers demand co-designed architectures and runtimes. MCUNet [144] combines TinyNAS, which searches architectures under Flash and SRAM limits, with TinyEngine, an inference engine optimized for SRAM-only execution. This design enables ImageNet-scale models to run within 1 MB Flash and 320 KB SRAM, illustrating the feasibility of deep learning on highly constrained devices. Processor-level innovations such as Micronets [22] further explore asynchronous, distributed scheduling for ultra-low-power AI. More recently, Neuro-C [204] presents a compact

neural architecture designed around MCU hardware limits. It removes multiply-accumulate operations and uses a simplified network structure where each neuron applies its own scaling factor, cutting inference time and memory use by up to 90% on Cortex-M0 devices. These approaches achieve impressive efficiency, but their device-specific nature means that porting across different MCU families often requires new searches or re-implementation, limiting generality.

Emerging Transformer-Based Lightweight Models. Although CNNs dominate current edge AI deployments, hybrid convolution-transformer architectures are beginning to appear. MobileViT [164] embeds lightweight transformers within convolutional backbones, while EfficientFormer [140] co-designs transformer blocks with neural architecture search to meet mobile latency and memory constraints. Both demonstrate competitive accuracy-efficiency trade-offs on benchmarks such as ImageNet, indicating the potential of attention mechanisms in edge vision. However, compared to mature CNN families, deployment support in edge toolchains remains less developed, and adoption is still emerging.

3.3 Model Compression and Deployment-Oriented Optimization

Model compression techniques and deployment-oriented optimization methods focus on adapting a trained model to the strict resource constraints of edge devices. Rather than changing the learning task or data, these techniques reshape the final artifact to reduce latency, memory, and energy consumption, while striving to preserve accuracy for efficient on-device inference. A key enabler is the integration of such methods into widely adopted deployment toolchains, which makes them practical for real-world embedded AI.

Post-Training Quantization (PTQ). Post-training quantization (PTQ) reduces weights and activations to low precision, typically 8-bit integers, thereby decreasing model size and improving inference latency on constrained hardware [115]. Because PTQ requires no retraining, it is supported across nearly all major toolchains, LiteRT, ONNX Runtime, OpenVINO, and vendor SDKs, making it one of the most accessible techniques for practitioners.

However, standard PTQ can cause accuracy degradation, especially in compact models or sensitive tasks. Recent refinements mitigate this effect. For example, BRECQ reconstructs intermediate outputs during calibration [141], SmoothQuant rebalances activations and weights for large models [266], and PTQ4SAM adapts the method to segmentation transformers [156]. Comprehensive studies confirm these trends: Gholami et al. survey quantization methods across architectures [81], while Yao et al. show that fine-grained PTQ enables efficient compression even for billion-parameter LLMs [271]. In practice, PTQ offers a favorable trade-off between efficiency and accuracy, and its broad tool support ensures straightforward deployment on edge accelerators.

Quantization-Aware Training (QAT). Unlike PTQ, which applies quantization post hoc, QAT integrates low-precision arithmetic directly into the forward and backward passes so that parameters adapt during training [115]. QAT typically achieves higher accuracy than PTQ, staying within 0.2-0.5% of full-precision baselines at 8-bit, while also delivering measurable latency gains on embedded GPUs such as the Jetson Nano [233]. Support for QAT is now provided in toolchains such as TensorFlow Model Optimization Toolkit (TFMOT), PyTorch QAT, and ONNX Runtime QNN, which embed hardware-aware quantization schemes directly into training workflows.

The main drawbacks are higher training cost and reduced portability when the simulated quantization scheme differs from backend support [50, 171]. Zhao et al. introduce the notion of recoverable accuracy to quantify when QAT is necessary, showing that it is most beneficial at very low precision (e.g., INT4, INT2) or in layer-sensitive settings [280]. These insights have motivated selective-QAT approaches, increasingly integrated into frameworks like TVM Unity and OpenVINO, which allow hybrid precision deployment with reduced retraining burden.

Sparse Training and Pruning. Pruning improves efficiency by eliminating redundant parameters from trained or partially trained networks. Sparse training typically employs sparsity-inducing regularizers such as weight decay

or ℓ_1 penalties to promote near-zero weights that can later be removed [74, 92]. Structured pruning removes entire filters or channels, which maps well to NPU, VPU, and DSP accelerators supported in deployment toolchains like CMSIS-NN and TensorRT. Unstructured pruning, by contrast, maximizes compression but requires specialized sparse kernels; tool support here is still uneven, though libraries such as XNNPACK and cuSPARSE have begun to offer partial support [77]. Empirical studies show that pruning can shrink parameter counts by $\sim 75\%$ with little accuracy loss, and when combined with PTQ achieves storage reductions approaching 90% on MCU-class devices [74].

Recent methods extend pruning into dynamic sparse training. Liu et al. show that regrowing connections (neuroregeneration) improves plasticity during sparse-to-sparse training [149]. Even random pruning at initialization can suffice: with carefully chosen sparsity ratios, subnetworks can match or surpass dense baselines while improving robustness [150]. Dynamic Sparse Training (DST) iteratively prunes and regrows connections, and has been implemented in PyTorch extensions for ImageNet-scale models, achieving $\sim 76.6\%$ top-1 accuracy at 80% sparsity [96, 180]. While pruning has matured from static magnitude-based rules to dynamic regimes, tool integration remains less standardized than quantization, limiting portability across heterogeneous edge hardware.

Knowledge Distillation (KD). Knowledge distillation compresses a larger teacher into a smaller student by aligning logits or intermediate features [185]. Using KD, Paparounas et al. reduce model size by $\sim 30\%$ and, when combined with INT8 quantization using TensorFlow Lite, achieved over 50% total reduction with competitive accuracy and latency [185]. KD is increasingly integrated into deployment pipelines: TensorFlow Lite Micro and Edge Impulse support distilled models for MCUs, while ONNX Runtime allows student models to be exported with quantization-aware optimizations.

Practical deployments confirm its value. For instance, in poultry health monitoring, a distilled lightweight detector ran at 27 fps on a Sony IMX500 edge-AI sensor with only 8 MB memory, maintaining a mean Average Precision of 94.3% when detections are considered correct if the predicted bounding box overlaps the ground truth by at least 50% after 8-bit quantization [244]. KD has also been extended to privacy-sensitive distributed settings: TinyFL_HKD integrates hierarchical KD into federated learning frameworks, enabling compact and personalized student models on Raspberry Pi and MCU-class platforms [105]. More recently, XAI-guided KD methods such as DiXtill transfer not only predictive ability but also interpretability from teacher to student [45].

Despite progress, KD inherits practical downsides: dependence on teacher quality, additional training cost from evaluating the teacher, sensitivity to hyperparameters such as temperature, and lack of standardized tool support compared to quantization. Overall, KD provides a flexible complement to quantization methods, and its gradual adoption in deployment toolchains highlights its relevance for deployment-oriented optimization.

3.4 Training and Learning Strategies

Beyond model compression, another line of research adapts the training process to the constraints of edge deployment [286]. These strategies reduce data and computation requirements during learning while sustaining model utility in dynamic environments where input distributions, devices, and connectivity shift over time [177]. Rather than optimizing a fixed model solely for efficient inference, training- and learning-oriented approaches emphasize generalization, adaptability, and distributed learning, enabling models to remain accurate and relevant throughout the edge AI lifecycle [186, 286]. Representative methods include transfer, few-shot, continual, and federated learning, each balancing efficiency and robustness in different ways [177, 186].

Table 6 summarizes representative approaches, including transfer, few-shot, continual, and federated learning, highlighting their benefits, challenges, and current tool support.

Because continual and federated learning also support post-deployment adaptation and model updating, their lifecycle role is revisited in Section 5.2.

Transfer Learning (TL). TL exploits pretrained models by freezing early layers and fine-tuning task-specific output layers, reducing both data and computation cost. In mobile traffic prediction, deep TL between base stations improved accuracy in 85% of experiments while cutting training energy by up to 60% for CNNs and 90% for RNNs, since only weights rather than raw data were exchanged [189]. Tool support is growing, with TensorFlow Hub and Hugging Face providing pretrained models and APIs for selective fine-tuning, which can be integrated into edge deployment toolchains such as LiteRT or ONNX Runtime. On-device evaluation confirms that TL can sustain real-time inference on constrained hardware: MobileNet, fine-tuned for face-mask recognition, achieved approximately 92% accuracy while sustaining 20-30 FPS on a Raspberry Pi with an Intel NCS2 accelerator, whereas InceptionV3 achieved approximately 95% accuracy but only 7-10 FPS, both deployed via TensorFlow Lite and OpenVINO [132]. TL also supports local recalibration: in-field retraining of soil conductivity models on a Raspberry Pi reduced mean absolute percentage error from about 7.8% to 1.7%, thereby avoiding reliance on cloud-based retraining [146]. A remaining limitation, however, is the domain gap: performance can degrade significantly when pretrained models are applied to edge data of different quality or characteristics [51].

Few-Shot Learning (FSL). Few-shot learning enables models to generalize from only a few labeled samples per class through metric learning, which compares similarity between examples, and meta-learning, which learns how to adapt model parameters efficiently to new tasks. This capability is particularly valuable at the edge, where collecting and annotating large datasets is often impractical [12]. In medical tasks, Siamese convolutional networks for ECG classification achieved 92.25% accuracy on the MIT-BIH arrhythmia dataset in a 5-shot setting, while requiring only limited data and computation suitable for IoT and wearable devices [88]. Hardware-focused studies show that quantization can preserve few-shot learning accuracy, with performance remaining comparable to floating-point even at 5-6 bit precision [123]. In addition, bit-width-aware FPGA pipelines sustain real-time inference throughput while reducing hardware cost [122]. Tool support is maturing, with PyTorch and TensorFlow meta-learning libraries complemented by lightweight deployment via LiteRT, PyTorch Mobile, and commercial platforms such as Edge Impulse and Qeexo AutoML. A key limitation, however, is sensitivity to distribution shift: performance degrades when target tasks diverge strongly from the meta-training set, while extreme quantization (<5 bits) or reliance on FPGA-specific optimizations can limit portability to general-purpose edge platforms [12, 122, 123].

Continual Learning (CL). CL incrementally extends model knowledge without retraining from scratch, addressing catastrophic forgetting when new tasks or classes emerge [186, 287]. This capability is particularly relevant at the edge, where data distributions shift over time due to user behavior, sensor drift, or environmental variability. In embedded keyword spotting, Michieli et al. demonstrated that CL methods support rapid personalization to new user-defined words on low-resource devices, achieving up to 11.3% accuracy gains over baselines while remaining lightweight [165]. Similarly, in edge-based license plate detection, Ramjattan et al. showed that CL enables adaptation to real-world shifts such as blur and low lighting, with performance comparable to full retraining but at lower resource cost [198]. Tool support is gradually maturing: replay-based approaches have been experimentally validated on TensorFlow Lite, enabling on-device adaptation in mobile environments, while Avalanche provides an open-source PyTorch ecosystem with standardized benchmarks (e.g., SplitMNIST, CORe50, OpenLORIS) and implementations of replay, regularization, and architecture-based strategies [152]. Limitations remain, including memory overhead from replay buffers, the high cost of on-device training relative to inference [165], and the tendency of regularization methods to underperform when tasks are only weakly related. Dynamic model expansion increases size and complexity [186], while cloud-assisted CL mitigates device constraints but introduces latency and bandwidth overheads [206]. Although introduced here as a learning strategy, CL also supports post-deployment adaptation by incorporating new data, addressing drift, and reducing full retraining needs.

Table 6. Training and learning strategies in edge AI.

Strategy	Key Idea	Edge Benefits	Challenges	Tool Support
Transfer Learning	Pretrained backbone with task-specific fine-tuning	Low data/training cost; fast adaptation; supports local recalibration	Domain gap; limited pretrained models	TF Hub, Hugging Face, TFLite, PyTorch Mobile, OpenVINO
Few-Shot Learning	Meta-/metric-learning from few samples	Label-efficient; compact models	Sensitive to noise/domain shift	PyTorch/TensorFlow libs, TFLite, Edge Impulse, Qeexo
Continual Learning	Incremental updates without full retraining	Adaptation to evolving data; avoids full retraining	Catastrophic forgetting; replay memory overhead; model expansion adds complexity	Avalanche (PyTorch), TFLite, PyTorch Mobile
Federated Learning	Collaborative training via model/update sharing	Privacy-preserving; avoids raw data transfer	High communication cost; non-IID data; privacy attacks	TF Federated, Flower, PySyft

Federated Learning (FL). FL trains a shared global model by aggregating updates from distributed devices rather than centralizing raw data, thereby preserving privacy and reducing bandwidth usage in domains such as healthcare and IoMT [240]. Widely adopted algorithms include FedAvg, with extensions such as FedProx and SCAFFOLD to address device and data heterogeneity [240]. In practice, orchestration frameworks such as TensorFlow Federated, Flower [30], and PySyft provide support for deployment and management of FL systems. In applied e-health scenarios, a token-based FL architecture coordinating 1,500 IoT devices achieved around 96% accuracy in authenticating and optimizing device operations, although the additional privacy mechanisms introduced measurable latency compared with centralized training [13]. Despite these advances, significant challenges remain: frequent synchronization rounds can impose high communication overhead on wireless links [255], heterogeneous data and hardware often slow convergence, and intermittent participation reduces efficiency. Moreover, although FL avoids raw data sharing, it is still vulnerable to gradient leakage, poisoning, and inference attacks, while stronger defenses such as secure aggregation increase computational cost and latency [13, 240]. Taken together, these trade-offs highlight that FL offers stronger privacy and robustness than TL, but demands careful orchestration to achieve scalable deployment in resource-constrained environments.

4 Deployment Toolchains and Connectivity-Aware Workflows

Deployment toolchains bridge the gap between trained AI models and their execution on embedded and IoT platforms. Unlike cloud-based workflows, where scalability and throughput dominate design objectives [222, 286], edge deployments must be engineered within tight constraints of memory, energy, and heterogeneous hardware [24, 59]. To meet these requirements, modern toolchains combine model compilation, optimization, and hardware-aware scheduling, often via operator fusion and target-specific code generation [49]. Increasingly, these workflows extend beyond static deployment to support hybrid edge-cloud execution and runtime adaptivity [59, 286].

This section reviews the state of practice in deployment toolchains for edge AI. We first examine compilation pipelines and intermediate representations, then turn to hardware-aware optimization and scheduling, followed by

inference runtimes and connectivity-aware execution, and conclude with a comparative analysis of representative toolchains.

4.1 Compilation Pipelines and Intermediate Representations

Compilation pipelines transform trained models from high-level frameworks (e.g., TensorFlow, PyTorch) into deployment-ready binaries tailored to specific hardware targets [49]. Most modern toolchains introduce an intermediate representation (IR) layer, e.g., ONNX (exchange IR), MLIR (multi-dialect compiler IR), or Relay (TVM’s graph/IR), to decouple model definition from backend-specific optimizations, enabling portability across heterogeneous accelerators and easing integration with inference engines [2, 49, 136].

In many deployment stacks, the IR is explicitly exposed alongside the runtime engine, supporting both hardware mapping and lifecycle integration [78]. In accelerator-oriented compilers, the IR also provides a substrate for memory-constrained placement and dataflow mapping under strict on-chip resource limits [118].

Representative toolchains illustrate the diversity of IR-based deployment. TensorFlow Lite Converter transforms TensorFlow models into FlatBuffer-encoded binaries and supports optimizations such as post-training quantization [199]. ONNX exporters and ONNX Runtime enable cross-framework portability through the ONNX exchange format and backend-specific Execution Providers [10, 116]. Apache TVM compiles models to optimized kernels on heterogeneous targets and applies graph-level transformations via Relay; in its newer Unity stack, the Relax IR adds first-class dynamic shapes and heterogeneous execution [17, 49, 203]. MLIR provides a flexible compiler infrastructure for defining domain-specific dialects and progressive multi-level lowering [136]. OpenVINO, finally, is a framework-agnostic inference engine optimized for Intel hardware, with limited support for graph-level optimization [151]. In contrast, Tinygrad is an open-source framework that combines model definition, automatic differentiation, and a compact compiler and runtime supporting CPU, GPU, and OpenCL execution. It provides a simplified example of integrated deployment functions [241].

Beyond open-source toolchains, commercial ecosystems increasingly embed IR workflows into end-to-end deployment pipelines. For example, NVIDIA Triton integrates TensorRT engines into both Jetson-class devices and cloud services, illustrating how IRs can serve as the foundation of unified edge-cloud execution [217]. Qualcomm AIMET extends this idea to mobile platforms by combining IR conversion with compression and quantization for Snapdragon NPUs. Meanwhile, cloud providers such as AWS Greengrass and Microsoft Azure Percept expose IR-based deployment flows, but these remain tightly coupled to their proprietary infrastructures [119]. Taken together, these examples show that intermediate representations are increasingly used as integration points linking device optimization with lifecycle management. In parallel, microcontroller ecosystems have evolved to address the specific constraints of embedded platforms. STMicroelectronics provides a parallel deployment environment for MCU-class devices through STM32Cube.AI, an integrated compiler and profiler that converts trained neural networks into optimized STM32 firmware, supporting quantization, static memory planning, and runtime profiling. Complementing it, the STM32 AI Zoo offers a curated collection of pre-optimized reference models, datasets, and deployment examples targeting STM32 MCUs and MPUs. These tools are examples of vendor-specific ecosystems that connect standard machine-learning frameworks with practical deployment on embedded microcontrollers [229, 230].

Challenges and Emerging Trends in IR-based Workflows. Despite standardization efforts, intermediate-representation (IR) workflows continue to face several obstacles. Operator coverage remains inconsistent across toolchains: in an analysis of 200 ONNX converter issues, the node-conversion stage alone accounted for 75% of reported defects [116]. IR-based approaches must also provide mechanisms to express and schedule partitioning across heterogeneous devices, EdgePipe demonstrates that partition strategies, combined with device and network heterogeneity, can significantly affect inference throughput on edge clusters [101]. Another limitation is the widespread reliance on static input shapes: TVM, for example, leverages knowledge of concrete input dimensions

to specialize optimizations and apply static memory planning at the graph level [49]. By contrast, MLIR retains higher-level semantics and structured control flow through progressive lowering, making it more suitable for heterogeneous targets [136]. These factors underscore a recurring portability-efficiency tension, with surveys documenting how extreme hardware heterogeneity complicates comparability and deployment across stacks [59].

Recent compiler infrastructures directly address these gaps. MLIR’s extensible dialect and transform systems preserve high-level semantics while enabling progressive lowering and cross-layer optimizations [136, 154]. TVM’s Relax IR adds first-class symbolic shape support for dynamic workloads, and the broader TVM Unity initiative extends heterogeneous execution across CPUs, GPUs, and accelerators [17]. BladeDISC introduces symbolic-shape tracking with runtime kernel selection, making it well suited for streaming or batched inference [284]. Beyond handling dynamic inputs, IRs increasingly encode structural properties of workloads: MLIR’s sparse tensor dialect makes sparsity explicit, while the Indexed Streams IR formalizes fused tensor contractions and indexing semantics, enabling optimizations such as tiling and loop fusion to improve locality [33, 130]. Commercial and research prototypes are converging on similar principles: NEST-C integrates profiling-based quantization, dynamic graph partitioning, and multi-level IRs to enhance portability across NPUs and processing-in-memory accelerators [187]. Recent advances indicate that IRs are shifting from static compiler artifacts toward orchestration layers that support partitioning, lifecycle updates, and scheduling across heterogeneous edge environments.

4.2 Hardware Optimization

Graph and Kernel Optimization. Once a model is converted into an intermediate representation, deployment toolchains apply graph-level optimizations to restructure computation for runtime efficiency. Common passes such as operator fusion and constant folding reduce redundant work and improve memory locality, while systematic infrastructures like Apache TVM and MLIR make it straightforward to implement these and related transformations [49, 136].

Beyond generic passes, frameworks increasingly rely on hardware-optimized kernel libraries provided by vendors or open-source communities. Widely used examples include CMSIS-NN for Arm Cortex-M devices, cuDNN for NVIDIA GPUs, and XNNPACK for mobile CPUs. By mapping high-level operators to such kernels, compilers achieve large performance gains with minimal developer effort. Mainstream deployment stacks such as Apache TVM, TensorRT, and OpenVINO integrate these mechanisms, reflecting a broader trend toward pairing generic graph optimizations with hardware-specific acceleration layers [2, 59].

Empirical results confirm that software- and compiler-level optimizations can yield substantial speedups [25, 201]. For example, the MLPerf Tiny benchmark suite defines accuracy targets of at least 90% top-1 accuracy for keyword spotting and 80% for visual wake words, and shows that optimized submissions outperform the TensorFlow Lite Micro baselines while still meeting these thresholds across embedded platforms [24].

Beyond Graph-Level Optimizations. While classical compiler passes such as operator fusion and constant folding provide efficiency gains, recent work has advanced toward automated, search-based scheduling. For example, Anso constructs a large hierarchical search space and uses a learned cost model with evolutionary search to explore it. Experiments show that it discovers schedules outside the reach of template-based methods and achieves up to 3.8x, 2.6x, and 1.7x speedups on Intel CPUs, ARM CPUs, and NVIDIA GPUs, respectively, often rivaling or surpassing hand-tuned baselines [283]. More recently, Felix [281] relaxes schedule variables into a differentiable space and applies gradient descent to optimize tensor programs. In evaluation on six networks across server, desktop, and edge GPUs, Felix reached 90% of peak Anso performance 3.4x faster (batch size 1) and up to 5.8x faster (batch size 16), while ultimately achieving similar peak performance. Complementary to learning-driven methods, Welder [223] targets memory as the primary bottleneck in DNN inference, introducing a tile-graph abstraction and two-step scheduling algorithm to minimize traffic. In evaluations on ten workloads,

Welder reports up to 21.4x speedup over PyTorch and up to 3.0x over TensorRT, while automatically discovering hundreds of fusion patterns beyond manually designed rules.

At the system level, hardware-software co-design has emerged as a key strategy for reducing memory and energy overheads in deployment pipelines. For example, JAQ [253] jointly optimizes network architecture, quantization precision, and accelerator mapping to achieve substantial reductions in memory and energy cost on edge devices. Its channel-wise sparse quantization scheme cuts memory use by up to 5x, while the BatchTile compiler mapping method reduces hardware search overhead from tens of seconds to 0.15s per iteration. Experiments on CIFAR and ImageNet show up to 7% higher accuracy than prior co-design methods, along with flexible trade-offs between energy, latency, and chip area. Complementarily, MLPerf Tiny standardizes latency, accuracy, and energy-per-inference reporting, making such optimizations reproducible across heterogeneous MCU deployments [24]. These advances suggest a shift from static graph transformations toward automated, learning-based, and memory-focused compilation pipelines.

Runtime-Aware Resource Management. Beyond static graph and scheduling optimizations, runtime-aware resource management provides an additional layer for improving edge AI efficiency. These approaches adapt execution parameters such as processing-element assignment, voltage-frequency operating points, memory usage, and edge-cloud partitioning according to workload and system conditions. DVFS is a representative mechanism, as it adjusts operating points to trade inference latency against energy consumption on heterogeneous platforms. DVFO [278] illustrates this direction by jointly optimizing CPU, GPU, and memory frequencies together with the proportion of DNN feature maps offloaded to the cloud, using deep reinforcement learning under varying network conditions. Complementarily, MEDEA [234] provides a design-time manager for heterogeneous ultra-low-power platforms, selecting the processing element, voltage-frequency point, and tiling mode at kernel granularity under timing and memory constraints. These works show that runtime and design-time resource management complement graph rewriting, kernel selection, and automated scheduling in hardware optimization for edge AI.

4.3 Scheduling, Orchestration, and Connectivity-Aware Execution

Deployment efficiency depends on both code generation and the scheduling of inference tasks across heterogeneous resources.

On Device Scheduling. On modern platforms, runtimes must carefully allocate layers across CPUs and NPUs to meet constraints on latency, energy, and accuracy. Lin et al. [145] show that online CPU-NPU co-inference with Lyapunov-based scheduling reduces average inference delay by up to 33% and energy consumption by about 18% compared to static partitioning. On CPU-GPU systems, BlastNet demonstrates that adaptive layer allocation improves throughput for real-time multi-model inference by 1.6× while lowering deadline misses by up to 35% [147]. For GPU-centric workloads, D-STACK [60] leverages cross-model scheduling to balance throughput and latency, while Pantheon applies fine-grained preemption to improve multi-DNN timeliness on mobile GPUs, cutting deadline misses to ~1% and boosting accuracy in IoT workloads [91]. These results highlight how device-level schedulers translate compiler and runtime policies into measurable efficiency gains on constrained hardware.

Multi-Model Scheduling. IoT nodes often execute multiple DNN workloads concurrently, requiring runtimes to manage limited resources while minimizing end-to-end latency. Li et al. [139] formulate multi-model inference as a latency optimization problem on embedded edge devices, and propose both an optimal and a heuristic scheduling strategy to allocate tasks across containerized applications running on ONNX Runtime. Experiments on a Jetson Nano show that collaborative scheduling reduces response time by up to 16.7% compared to independent execution. More recently, Jun et al. [120] introduced a hierarchical dispatcher that partitions DNNs into sub-models and schedules them across heterogeneous processors, improving performance by 51.6% over CPU-only execution.

Such results underscore that multi-model scheduling is critical for realistic IoT deployments where devices must serve diverse applications in parallel.

Adaptive Inference Across Edge and Cloud. While the above methods optimize within-device execution, many IoT deployments must also orchestrate inference across device-cloud boundaries under variable connectivity. This challenge has been addressed through split computing and cooperative inference. CoEdge orchestrates distributed DNN inference across heterogeneous edge devices, dynamically adapting workload partitioning to device capabilities and network conditions, achieving up to 7x latency reduction and 25-67% energy savings [275]. EdgePipe [101] complements this with pipelined execution over volatile wireless devices, introducing a super-neuron-based partitioning and scheduling mechanism that is 2.47x faster than model parallelism and 2.13x more accurate than PipeDream under lossy networks. More recently, PArtNNer and CO-PILOT report strong gains: PArtNNer achieves on average 10.2x lower latency than edge-only and 3.2x than cloud-only (up to 21.1x and 6.7x, respectively), while CO-PILOT reduces end-to-end latency by up to 85% with less than 1% accuracy loss through resource-aware lossy partitioning [82, 126].

Mainstream DNN runtimes provide only limited forms of adaptivity. ONNX Runtime's Execution Providers [10] and OpenVINO's [112] enable hardware-aware operator placement across CPUs, GPUs, and NPUs on a given node, but their public documentation does not describe built-in mechanisms for connectivity-aware switching between local execution and cloud offloading. At a higher level, device-edge-cloud pipelines and adaptive partitioning have been widely investigated in the edge-intelligence literature, yet these capabilities are not integrated into commercial toolchains.

Industrial ecosystems emphasize deployment, integration, and monitoring rather than fine-grained partitioning. For instance, NVIDIA Triton provides inference-side scheduling (dynamic batching and concurrency management), enabling scalable service orchestration [217]. AWS IoT Greengrass [6] and Azure IoT Edge [7] similarly focus on modular container deployment and cloud integration, with support for running ML workloads at the edge, but do not offer first-class support for connectivity-adaptive partitioning.

Overall, Section 4 shows that deployment support is more mature at the device-optimization level than at the system-adaptation level. As summarized in Table 7, compilers, IRs, and hardware-aware runtimes increasingly improve execution on heterogeneous devices, while connectivity-aware partitioning and lifecycle integration remain less consolidated in mainstream toolchains. The main challenge is therefore to move from isolated optimization tools toward deployment workflows that jointly support portability, specialization, and adaptation.

5 Post-Deployment Phase, Evaluation Criteria and Benchmarking

The post-deployment phase in edge AI covers the operational lifetime of deployed models. In contrast to compilation and deployment, which focus on producing a working artifact, this phase emphasizes long-term reliability, adaptability, and security. Effective post-deployment strategies ensure that models continue to perform robustly under dynamic environmental conditions, shifting data distributions, and evolving application requirements. This is particularly important in distributed edge deployments, where fleets may consist of thousands of resource-constrained devices, making manual updates impractical and necessitating automated monitoring and OTA updates [28].

In addition to lifecycle management, rigorous evaluation and benchmarking are central to sustaining reliable operation. Metrics such as latency, accuracy, memory footprint, and energy consumption define the performance envelope of deployed systems, while standardized benchmarks provide the means to compare solutions across heterogeneous hardware and software stacks. Taken together, lifecycle mechanisms and evaluation frameworks form the foundation for trustworthy post-deployment operation of edge AI.

Table 7. Summary of key aspects and open challenges in deployment toolchains and connectivity-aware workflows for edge AI

Aspect	Representative proaches	Ap- Key Capabilities	Open Challenges
Compilation and IRs	TVM, MLIR, ONNX, OpenVINO	Hardware-agnostic IRs, graph optimization, quantization	Limited operator coverage, static-shape constraints
Hardware Optimization	Ansor, Felix, Welder, JAQ	Auto-scheduling, learned cost models, HW-SW co-design	Balancing portability and specialization, memory bottlenecks
Scheduling & Orchestration	BlastNet, Pantheon, D-STACK, EdgePipe	Adaptive layer mapping, multi-model inference, edge-cloud partitioning	Runtime integration in commercial stacks, heterogeneous scheduling
Industrial Toolchains	TFLite, STM32Cube.AI, AIMET	End-to-end deployment flows, quantization, profiling	Vendor lock-in, limited cross-platform standardization
Emerging Trends	Relax IR, MLIR dialects, hybrid edge-cloud pipelines	Symbolic shapes, heterogeneous runtimes, unified IR workflows	Lifecycle integration, connectivity-aware execution

5.1 Lifecycle and Monitoring

Model Versioning and Rollback. Maintaining consistency across large, heterogeneous fleets of edge devices requires disciplined version control, authenticated update mechanisms, and robust rollback support. Versioning schemes combined with cryptographic signatures provide a foundation for integrity and authenticity, while gradual or staged deployment strategies are widely recommended to mitigate the risk of system-wide failures. A frequently adopted technique to realize reliable update deployment is dual-bank (A/B) partitioning. In this approach, the new firmware image is first staged in an inactive partition and activated only after successful verification. If validation fails, the system automatically reverts to the previous stable image, thus minimizing downtime and ensuring recovery from faulty or malicious updates. This rollback-capable design has become a de facto standard in resilient IoT and edge platforms [18]. Surveys note that manual intervention does not scale, making automated rollback a key requirement for IoT and edge maintainability in IoT and edge deployments [67].

Sorensen et al. evaluated differential firmware updates on embedded IoT fleets using an A/B dual-partition setup with rollback support, showing that efficiency gains can be achieved while still allowing the device to automatically fall back to its previous stable firmware if an update fails [227]. Complementary efforts on runtime partial updates for microcontrollers have achieved near-zero downtime, with only millisecond-level interruption, by replacing targeted code segments during execution. The proposed method leverages temporary shadow buffering for verification, allowing updates without reboot while preserving ongoing application functionality [174]. Large-scale empirical analyses of commercial firmware update systems continue to reveal vulnerabilities such as incomplete version checks and missing freshness guarantees. These weaknesses enable downgrade and rollback attacks, demonstrating that flaws in the update logic can undermine device security even when other protective mechanisms are in place [263]. In response, security-by-design OTA frameworks have been proposed, embedding modular cryptographic protections, explicit rollback policies, and lifecycle governance directly into the update pipeline [114].

Telemetry-Based Monitoring and Drift Detection. Telemetry-driven monitoring is necessary to ensure that machine-learning models remain reliable after release. In production, telemetry commonly combines model-performance signals, input- and prediction-data statistics, outlier detection, and drift detection to identify degraded

predictions or distributional changes [127]. This is particularly important for edge- and TinyML-deployed models, where sensor degradation, environmental variation, seasonality, device faults, and usage shifts can induce data or concept drift and silently reduce accuracy [61, 95].

On-device drift detectors differ in label requirements, assumptions, and resource footprints. Error-rate detectors such as DDM and EDDM are lightweight but require labeled feedback or delayed ground truth, which is often unavailable in autonomous IoT deployments. Statistical stream detectors such as ADWIN and Page-Hinkley can monitor available signals such as prediction errors, confidence scores, feature summaries, or sensor statistics. Unsupervised approaches, including anomaly-score monitoring, lightweight clustering, and reconstruction-error tracking, avoid direct label dependence and are attractive for MCU- and sensor-level deployments, although they remain sensitive to thresholds, false alarms, and operating-condition changes [16, 95, 127].

Streaming detectors such as ADWIN and Page-Hinkley are common online change-detection mechanisms, but they rely on different principles: ADWIN compares statistics across adaptive subwindows, whereas Page-Hinkley tracks cumulative deviations from a running mean [228]. Empirical studies show that drift handling is detector-, task-, drift-type-, and evaluation-dependent [210, 228]. For instance, Stevanoski et al. report that drift-aware iSOUP variants often improve over non-adaptive baselines, but ADWIN-based ensembles perform best for gradual and incremental artificial drift, Adaptive-iSOUP-Bag for abrupt drift, and Adaptive-iSOUP overall on real-world streams [228]. Thus, drift detection is best treated as a configurable deployment component rather than a universally optimal add-on.

At the edge, drift monitoring must also be balanced against resource constraints and task management. Moustakas and Kolomvatsos propose DBTM, which monitors data and concept drift while orchestrating task offloading across edge nodes. On synthetic distributed edge scenarios and a real-world stream emulated across edge nodes, DBTM keeps node load within the configured threshold while maintaining accuracy comparable to or higher than First-Come-First-Served and Longest-Job-First scheduling, with only a few seconds of additional decision overhead [170]. Their ablation study indicates that monitoring both data and concept drift gives the highest mean accuracy in the reported cases, although gains vary across scenarios [170].

TinyML case studies further point toward local, label-independent monitoring. Antonini et al. demonstrate an unsupervised anomaly-detection and local-monitoring pipeline on an ESP32-class microcontroller in an extreme industrial environment, where data collection, isolation-forest training, anomaly inference, and gateway alerting are performed locally [16]. Edge Impulse complements this deployment view by supporting TinyML model optimization, code generation, and device-specific profiling of preprocessing latency, inference latency, RAM, and flash usage [107]. These profiling signals are not drift detectors, but help determine whether monitoring, adaptation, or retraining is feasible on a given device.

Recent TinyML-oriented work combines drift detection with limited local adaptation. TML-CD uses a fixed deep feature extractor, a lightweight k-nearest-neighbor classifier, and hybrid passive/active adaptation on microcontroller-class devices [61], while Yamada and Matsutani couple a fully sequential drift detector with on-device neural-network learning on a Raspberry Pi Pico-class platform [269]. These works show that edge drift handling is both a monitoring and deployment problem, involving detection quality, delay, false alarms, post-drift accuracy, memory, execution time, energy, label availability, communication overhead, and adaptation strategy [61, 210, 269]. Since both supervised and fully unsupervised benchmarks evaluate these dimensions separately and expose limitations in individual metrics and proxy measures, edge/TinyML evaluations should report them jointly rather than rely on a single aggregate score [155, 159].

Reliability and Security in Safety-Critical Deployments. In safety-critical domains such as healthcare, industrial automation, and autonomous mobility, post-deployment management should go beyond latency, energy, and accuracy to also consider reliability, security, and update accountability. Deployed models may degrade because of data drift, data-quality issues, changing operating conditions, or delayed availability of ground-truth labels, making

continuous monitoring and model governance important for safe operation [125, 129, 251]. When abnormal behavior is detected, deployments should support conservative responses, such as triggering re-evaluation, retraining, retiring or replacing the model, taking the model offline, or involving human review when feasible. Security is equally important because edge and TinyML devices are often resource constrained and may be physically exposed, making them vulnerable to adversarial inputs, model extraction or tampering, unauthorized updates, rollback attacks, firmware compromise, and attacks on update channels [66, 104]. Secure post-deployment pipelines should therefore combine authenticated model and firmware delivery, integrity verification, secure boot or hardware roots of trust where feasible, anti-rollback checks, staged rollout policies, and version or update records. Recent TinyML and OTA update studies show that secure model-update mechanisms are feasible on constrained devices, but they introduce latency, energy, memory, storage, and communication trade-offs that should be evaluated explicitly [103, 179].

5.2 OTA Updates and Model Adaptation

Secure and Efficient OTA. Over-the-air mechanisms enable remote patching and upgrading of models and firmware without requiring physical device access, which is essential for managing large, geographically distributed fleets. Security is typically enforced through lightweight public-key cryptography. Recent evaluations confirm that elliptic-curve signatures are practical even on microcontrollers, providing equivalent security with much smaller key sizes and reduced computational and energy overhead compared to RSA [192]. Complementary work has demonstrated the feasibility of stateful hash-based schemes, such as XMSS and LMS, on ARM Cortex-M devices, with optimized implementations achieving signing and verification within a few hundred milliseconds and fitting comfortably within tens of kilobytes of RAM, providing forward-secure alternatives for update authentication under resource constraints [44].

Efficiency is achieved through techniques such as delta encoding, block-level compression, and chunk-based streaming. Empirical evaluations show that differential update schemes can reduce transmission overhead by nearly half and installation time by up to 78%, while preserving rollback safety in embedded deployments [227, 260]. Building on this, hybrid distribution and scheduling strategies have been proposed to balance bandwidth and latency in large-scale IoT and automotive environments.

Beyond transmission and installation efficiency, secure OTA workflows should also quantify the resource overhead introduced by security mechanisms. We refer to this as security overhead: the additional latency and energy consumption, as well as memory usage, storage footprint, transmitted bytes, or installation time, caused by mechanisms such as signature verification, integrity checking, authenticated manifests, encrypted payload handling, and rollback safeguards. These overheads depend on the update mechanism, network technology, and device class [66]. Recent work on LoRaWAN firmware updates further shows that latency and energy are central evaluation dimensions for constrained wireless deployments, especially for time-sensitive security patches [37]. RIOT-ML provides partial support for such evaluation by measuring memory, storage, manifest, and network-transfer costs for SUIIT-based secure TinyML OTA updates across different update granularities [103]. However, RIOT-ML does not define security overhead as a unified criterion and excludes SUIIT payload encryption from its measurements. Future OTA evaluations should therefore report security overhead explicitly, particularly for encrypted model updates on constrained devices.

At the system level, frameworks like LwM2M and SWUpdate have been standardized or widely adopted in industrial and academic testbeds, while research prototypes continue to extend them with energy-aware scheduling and secure bootstrapping for heterogeneous edge devices [224]. Together, these results highlight that secure and efficient OTA updates are not only feasible on MCU-class devices but also critical to sustaining maintainability across large-scale, heterogeneous edge deployments.

Federated, Continual, and Decentralized Updating. In scenarios where connectivity or privacy constraints limit direct model replacement, federated learning provides a complementary strategy. Devices collaboratively update a shared model by exchanging parameter deltas or gradients instead of raw data. Such approaches reduce bandwidth usage and enhance privacy, though they introduce new challenges in heterogeneity, energy consumption, and catastrophic forgetting. Over-the-air federated learning (OTA-FL) leverages the superposition property of wireless multi-access channels to perform low-latency model aggregation, thereby alleviating spectrum bottlenecks in large-scale edge deployments. Recent surveys highlight both its promise in reducing communication overhead and the unique challenges of synchronization, fading, and privacy that distinguish OTA-FL from conventional federated learning [265].

Continual learning provides a related mechanism for post-deployment adaptation, allowing models to incorporate progressively arriving data, respond to distribution drift, and reduce the need for full retraining. In distributed edge settings, federated continual learning (FCL) combines privacy-preserving federated learning with continual adaptation, making it suitable for dynamic environments where data distributions evolve and clients may join or leave unpredictably [259].

Beyond model aggregation, decentralized principles are also being applied to the update dissemination process itself. Oliveira et al. present an OTA framework for software-defined vehicles that combines mobile edge computing with the InterPlanetary File System (IPFS) to deliver resilient updates under intermittent V2X connectivity. Real-world trials in a smart-city living lab confirm that this design enables agile update distribution while mitigating service disruption and reducing operational costs [183]. Complementary work by Malik et al. proposes a fog-federated dissemination scheme in which pivot vehicles relay updates from fog nodes using V2V links, coordinated through SDN controllers. Their evaluation reports approximately 40% improvement in convergence time and 95% trustworthiness against malicious-node scenarios, compared to prior fog-based schemes [162]. These results indicate that decentralized update dissemination, whether for model retraining or binary delivery, remains a promising yet underexplored strategy for large-scale, heterogeneous edge environments.

Challenges and Trends. Commercial IoT OTA frameworks, such as Balena [8], Mender [9], Particle Cloud [11], provide secure phased rollouts and delta updates for firmware and application binaries. However, empirical analyses of vendor practices confirm that such solutions remain predominantly artifact-centric, treating models as opaque assets for replacement rather than exposing model-aware primitives [32, 38, 231]. Consequently, fine-grained adaptation mechanisms, such as subgraph-level updates, shadow model deployment, or selective operator replacement, are not yet present in mainstream OTA workflows [38]. Research prototypes such as RIOT-ML demonstrate that partial OTA updates, like updating only specific layers, can dramatically shrink payloads. In their MCU case study, transmitting only the model instead of the full firmware reduced transfer costs by more than 60%, while updating only the final layer eliminated over 99% of the network load [103]. These mechanisms remain early-stage: although RIOT-ML integrates uTVM to enable compilation and optimization, broader adoption in production-grade OTA workflows is still limited. Similarly, adaptive recalibration, where models are periodically fine-tuned with locally collected data to address concept drift, is highlighted in the literature as a promising strategy for preserving accuracy [95]. These gaps suggest the need for modular OTA pipelines that integrate with intermediate representations and deployment runtimes, allowing selective and adaptive updates while balancing reliability, bandwidth, and energy.

5.3 From Metrics to Deployment Evaluation

As outlined in Section 2, edge AI deployments are constrained by metrics such as latency, energy consumption, accuracy, memory footprint, and model size, with application-specific trade-offs shaping their prioritization. While these metrics define the performance envelope, a crucial question for both research and practice is how they are measured, benchmarked, and compared across heterogeneous hardware platforms and deployment

contexts. This section, therefore, focuses not on reintroducing the metrics themselves but on their practical evaluation, methodological gaps, and emerging benchmarking frameworks.

While conventional ML evaluation metrics such as accuracy, F1-score, or mean Average Precision remain necessary, they are insufficient for edge deployments, where feasibility is determined by a combination of accuracy, latency, energy consumption, and resource constraints [24, 232]. Several aspects of deployment evaluation are particularly important. Because these dimensions are interdependent, they should ultimately be interpreted through a multi-objective lens rather than as isolated metrics.

Latency and Responsiveness. While many deployment studies report average inference time and throughput, averages alone can be misleading in real-time and interactive contexts. In safety-critical control (e.g., UAV navigation), even bounded bursts of deadline misses can destabilize the closed loop, as shown in modern weakly-hard real-time control analyses [158]. More broadly, systems research has shown that rare but extreme latency outliers dominate user-perceived responsiveness in large-scale services, motivating the use of tail-latency metrics (e.g., 95th or 99th percentiles) rather than means alone [58]. Taken together, these perspectives emphasize that deployment evaluations must consider worst-case and tail behaviors, not just averages, to capture the true risks to stability and responsiveness. Reflecting these insights, standardized ML benchmarks now emphasize latency distributions rather than averages: MLPerf Inference requires percentile-based reporting for server- and edge-class devices [201], while MLPerf Tiny extends standardized latency, energy, and accuracy reporting to microcontrollers and embedded platforms [24].

Energy and Power Efficiency. Energy per inference (mJ/inference) provides a more deployment-relevant metric than instantaneous power draw, as it directly reflects device lifetime under battery constraints. However, measurement methodologies remain heterogeneous, ranging from benchtop probes to on-chip counters, which hinders comparability across studies. Tu et al. [248] address this gap by introducing kernel-level energy predictors and standardized scoring metrics across diverse edge devices, offering one of the first systematic baselines for energy benchmarking. Complementary work on embedded AI platforms demonstrates empirical energy models for CNN inference on NVIDIA Jetson boards [133], while application-driven deployments, such as UAV TinyML pipelines, highlight real-world energy-accuracy trade-offs under strict power budgets [200].

Deployment-Aware Footprint. Model size and memory usage must be evaluated in the context of deployment, not in isolation. OTA update pipelines are tightly constrained by binary size and the feasibility of lightweight update mechanisms, a challenge emphasized in TinyMLOps operational studies [137]. Real-world IoT deployments reveal that the severe Flash/RAM constraints of microcontroller-class devices hinder the adoption of conventional reliability and security mechanisms, creating potential failure points in update and maintenance workflows [67]. Surveys of secure firmware update frameworks further highlight that even tiny metadata such as manifests or cryptographic signatures can exceed RAM budgets on microcontroller-class systems, underscoring that Flash and RAM requirements are truly deployment-limiting constraints [274]. Security overhead, discussed in the OTA update subsection, complements deployment-aware footprint evaluation by quantifying the added latency, energy consumption, memory usage, transmitted bytes, and installation time introduced by secure model-update mechanisms.

Beyond Inference: End-to-End Pipeline Costs. Deployment evaluation should consider the complete sensing-to-decision pipeline, rather than model execution alone. On constrained platforms, accelerated inference can shift the main latency or energy cost to sensor processing, feature extraction, data movement, or CPU-side post-processing. For example, Ulkar et al. [250] show that MFCC feature extraction on a Cortex-M4F MCU can consume more energy than CNN inference on the MAX78000 accelerator. Recent TinyML benchmarking work similarly separates pre-inference, inference, and post-inference phases to expose such non-inference costs [27]. Stage-level benchmarking of ultra-low-power NPUs further shows that initialization, memory I/O, inference,

and CPU post-processing affect platforms differently, with memory movement becoming dominant in some cases [166]. These findings highlight the importance of reporting both isolated inference measurements and end-to-end pipeline costs when evaluating edge AI deployments.

Evaluation under Dynamic Conditions. A further challenge is that edge workloads are often non-stationary: sensor inputs, environmental conditions, user behavior, and connectivity can change after deployment. Such data drift necessitates lifecycle adaptability [31], while robustness to distribution shifts remains a central concern in real-world ML deployments [128]. Few current benchmarks capture these dynamics, underscoring the gap between static evaluation protocols and real-world operation [128]. Deployment evaluation should therefore consider not only initial accuracy, latency, energy, and memory usage, but also whether these properties remain stable under changing operational conditions.

Multi-Objective and Pareto-Based Evaluation. Single-number benchmarks are often insufficient for edge AI deployment because accuracy, latency, throughput, memory footprint, and energy consumption are tightly coupled. Prior benchmarking work emphasizes that edge and TinyML systems should be evaluated with deployment-relevant metrics rather than accuracy alone, including accuracy, latency, throughput, and energy depending on the target scenario [24, 25, 201]. This is especially important because optimization techniques such as quantization, pruning, and hardware-specific acceleration can improve latency, model size, memory use, or energy efficiency, while sometimes affecting accuracy, comparability, or portability across platforms [86, 92, 115, 232]. A practical way to summarize these trade-offs is to report deployment configurations as operating points and identify the non-dominated set, or Pareto frontier, rather than ranking systems by a single scalar score. This is consistent with prior observations that efficient model deployment involves Pareto trade-offs between accuracy and real-world platform latency [201, 236]. For example, UAV and robotics deployments must balance detection accuracy, FPS, and energy or power consumption [135, 200, 270], while industrial inspection systems must trade detection accuracy against throughput, power or energy use, and hardware resource utilization [21, 57, 79]. Pareto frontiers therefore provide a more informative view of the feasible design space and help practitioners choose configurations that match application-specific constraints. In practice, Pareto-based reporting is most useful when configurations are measured under comparable datasets, hardware platforms, and evaluation protocols; otherwise, it should be treated as a qualitative framework for interpreting trade-offs rather than as a direct ranking method.

5.4 Benchmarking Frameworks and Datasets

Reproducible evaluation is critical for comparing edge AI methods under realistic constraints. To this end, several benchmarking initiatives have emerged, spanning community-driven efforts, industry consortia, and vendor-specific tools.

Standardized Benchmark Suites. Several standardized suites address benchmarking across the edge spectrum. MLPerf Tiny extends MLPerf Inference to MCU- and TinyML-class devices with tasks such as image classification, keyword spotting, visual wake words, and anomaly detection, reporting accuracy, latency, and energy in a uniform way [24]. The EEMBC consortium provides complementary industry-driven suites: ULPMark targets ultra-low-power MCUs with a focus on energy efficiency, while MLMark evaluates common ML workloads with reproducible, cross-platform procedures [245]. At the higher end of the spectrum, MLPerf Mobile standardizes benchmarking for smartphone-class SoCs (Android/iOS NPUs and GPUs), thereby complementing MLPerf Tiny [202]. BenchCouncil's Edge AIBench adopts a broader system-level perspective, modeling client-edge-cloud collaboration across four application scenarios and incorporating a federated-learning testbed [94].

In practice, platform profilers are often used to characterize kernels and end-to-end pipelines on specific boards, such as Arm CMSIS-NN for Cortex-M operators, Jetson's TensorRT-based scripts, and Coral's Edge-TPU guidance. While informative for specific devices, these are not standardized benchmarks, and results are

not directly comparable across platforms. In addition, when hardware access is limited or large sweeps are needed, simulation and emulation environments like Renode, QEMU, and gem5 support virtual prototyping and reproducible pipelines. The Kenning framework instead bridges deployment and evaluation by offering modular pipelines that integrate model optimization with hardware benchmarking, including support for Renode simulations.

Datasets. Complementing benchmark suites, a common set of widely used datasets supports most evaluations across edge AI research. These cover multiple sensing modalities while remaining practical for deployment on constrained devices.

In vision, CIFAR-10/100 remain canonical baselines for 32×32 image classification, while Visual Wake Words serves as the de facto benchmark for low-resolution detection; more challenging settings increasingly use Tiny ImageNet (64×64 , 200 classes) to stress MCU-class models. For speech and audio, the Google Speech Commands corpus underlies MLPerf Tiny’s keyword-spotting task, whereas broader environmental audio sets such as ESC-50 and UrbanSound8K are widely used in academic studies despite not being standardized in MLPerf. Anomaly and sensor workloads are covered by MLPerf Tiny’s synthetic anomaly-detection traces, complemented by industrial datasets such as MIMII and TinyAD that provide realistic benchmarks for IoT and manufacturing deployments. At the higher end of the edge spectrum, large-scale datasets such as ImageNet and COCO remain standard references for mobile-class SoCs with NPUs and GPUs, enabling full-scale classification, detection, and segmentation tasks.

Together, these datasets form the backbone of reproducible evaluation across modalities, with MLPerf Tiny offering standardized selections (KWS, VWW, image classification, anomaly detection), while broader academic and industrial use cases motivate inclusion of speech, environmental audio, and domain-specific anomaly datasets.

Overall, Section 5 shows that post-deployment support is becoming essential but remains less mature than deployment-time optimization. As summarized in Table 8, current work provides useful building blocks for rollback, telemetry, OTA updates, security, and benchmarking, but these capabilities are still weakly integrated into model-aware lifecycle pipelines. The key gap is therefore not the absence of individual mechanisms, but the lack of unified workflows that can connect monitoring, adaptation, secure updates, and evaluation across heterogeneous edge fleets.

6 Cross-Layer Interactions in Edge AI Deployment

Although the preceding sections organize edge AI deployment into pre-deployment, deployment, and post-deployment phases, practical systems rarely follow these stages as independent steps. Decisions made early in the pipeline often propagate across later stages, shaping not only inference performance but also portability, updateability, and long-term maintainability. A cross-layer perspective is therefore necessary to understand edge AI deployment as a coupled optimization problem rather than a linear engineering workflow.

Model-Hardware Feasibility. Model design choices are closely linked to hardware feasibility. Lightweight architectures, pruning, quantization, and knowledge distillation are often introduced during model design or pre-deployment optimization to reduce model size and computation [84, 92, 115, 143]. However, their benefits depend strongly on the target hardware and runtime support. For example, MCU-oriented models such as MCUNet combine architecture search with an inference engine designed around Flash and SRAM limits, showing that architectural feasibility depends on both model structure and runtime implementation [144]. Similarly, 8-bit quantization can reduce memory footprint and latency, but the realized gains depend on whether the target device provides efficient integer kernels, accelerator support, or optimized libraries such as CMSIS-NN, TensorRT, or vendor-specific NPU backends [49, 115, 134]. Structured pruning also maps more naturally to hardware accelerators than unstructured sparsity, which may require specialized sparse kernels and can yield

Table 8. Summary of key aspects and open challenges in post-deployment operation and benchmarking of edge AI systems

Aspect	Representative forts	Key Capabilities	Open Challenges
Lifecycle Management	Balena, Mender, SWUpdate, LwM2M	Automated versioning, rollback recovery, secure update rollout	Lack of model-level control, limited integration with ML workflows
Telemetry-Based Monitoring	ADWIN, PageHinkley, DBTM, TinyML telemetry	Real-time drift detection, adaptive task management, local anomaly monitoring	Accuracy-energy trade-offs, limited on-device learning support
OTA and Model Updating	RIOT-ML, OTA-FL, decentralized OTA	Delta updates, federated retraining, decentralized distribution	Partial model replacement, intermittent connectivity resilience
Security and Governance	ComSec, hash-based signing (XMSS/LMS), differential OTA	Authenticated delivery, rollback protection, lightweight cryptography	Lifecycle enforcement, downgrade attack prevention
Benchmarking and Evaluation	MLPerf Tiny, MLMark, Edge AIBench, Kenning	Standardized latency, energy, and accuracy evaluation across devices	Reproducibility under heterogeneity, dynamic-condition evaluation

limited practical speedup if the deployment backend lacks support [33, 77, 96]. Thus, compression should not be viewed only as a model-level optimization, but as a hardware- and compiler-dependent design choice.

Intermediate Representations and Portability. Intermediate representations, compiler toolchains, and portability are also strongly interconnected. Intermediate representations such as ONNX, MLIR, Relay, and Relax provide abstraction layers between model development and device execution, but they also expose limits in operator coverage, shape handling, and backend compatibility [2, 17, 116, 136, 203]. Converter failures, shape-handling constraints, and inconsistent operator support can prevent a model that appears efficient in isolation from being deployed across heterogeneous targets. This means that architectural choices made during model design should be evaluated together with toolchain compatibility. A model that performs well during training may still be unsuitable for edge deployment if its operators cannot be lowered efficiently to the target runtime or if it requires manual rewriting for each hardware backend.

Deployment Optimization and Runtime Flexibility. Deployment optimization directly affects runtime behavior and lifecycle flexibility. Kernel fusion, operator scheduling, memory planning, and accelerator mapping can reduce latency and energy consumption, but because they often specialize execution to particular tensor shapes, memory hierarchies, and accelerator targets, they may also constrain flexibility after deployment [49, 136, 223, 253, 283]. For example, aggressive graph-level optimization may produce highly efficient hardware-specific execution plans while simultaneously making later adaptation, partial model replacement, or cross-platform migration more difficult. Hardware-software co-design methods such as JAQ illustrate this interaction by jointly optimizing network architecture, quantization precision, and accelerator mapping, thereby improving efficiency while coupling the resulting deployment to specific hardware assumptions [253]. This creates a potential trade-off between deployment efficiency and lifecycle flexibility. In static applications, specialized binaries may be preferable; in long-lived IoT deployments, however, maintainability and update granularity may be equally important.

Table 9. Cross-layer interactions across the edge AI lifecycle. The table summarizes how design choices made at different lifecycle stages influence pre-deployment preparation, deployment behavior, and post-deployment monitoring, updates, and maintainability.

Design choice	Pre-deployment impact	Deployment impact	Post-deployment impact
Model architecture	Sets the operator mix, model size, and baseline compute demand [144].	Affects compilation feasibility, operator support, and accelerator mapping [49, 136].	Influences maintainability, model updates, and runtime compatibility [103, 137].
Quantization precision	Lowers weight and activation precision to reduce model footprint and enable efficient integer inference [115].	Improves memory use and latency with efficient integer kernels or hardware [115, 134].	Can reduce model/OTA payload size, but may need post-quantization accuracy recovery and drift monitoring [95, 103, 115].
Feature Extraction (FE) location	Determines if FE is implemented on-device, outside the model, or within the model/accelerator pipeline [27, 250].	On-device FE can reduce network traffic, but shifts compute, memory-movement, energy, and latency costs to the device [27, 166, 250].	Determines which inputs, features, and runtime signals are available for telemetry, constraining drift detection, and model maintenance [27, 95, 127, 250].
IR/toolchain choice	Constrains usable model formats, operators, tensor shapes, and optimization passes [2, 136, 203].	Sets portability, but may be limited by operator support, converter reliability, and runtime compatibility [17, 116].	Can support modular and partial updates, and portability across heterogeneous hardware/toolchains [103, 137].
Edge-cloud partitioning	Guides model size, compression level, privacy assumptions, and early partitioning constraints [59, 286].	Balances latency, privacy, communication, and compute across local, split, and cooperative inference [82, 275].	Guides update, telemetry, and recovery choices [231], while affecting privacy and reliability trade-offs [59].
Hardware platform	Defines memory, compute, energy, sensing, and accelerator constraints [24, 166].	Constrains memory planning, operator/kernel support, scheduling, and accelerator mapping [166].	Determines feasibility of remote telemetry, secure OTA updates, and model updates or on-device adaptation [103, 137].
Monitoring and update strategy	Specifies monitoring signals, drift checks, and update hooks before deployment [95, 103, 127].	Requires profiling integration, device-management interfaces, and telemetry paths within device resource budgets [103, 137].	Enables drift/performance monitoring, model-version management, drift-aware adaptation, and OTA-based lifecycle updates with rollback and failure-recovery considerations [137, 170, 231].

Connectivity, Partitioning, and Lifecycle Management. Connectivity constraints shape both model partitioning and lifecycle management. In intermittently connected environments, local inference improves robustness and privacy, but it increases the need for compact models, local runtime monitoring, and secure OTA update mechanisms [18, 28, 59, 66]. Conversely, edge-cloud or split-inference designs can support larger models and centralized resources, but they become sensitive to bandwidth, latency, and network availability. Split-inference systems such as EdgePipe, PArtNNer, and CO-PILOT show that latency and energy benefits depend jointly on model structure, device heterogeneity, partitioning strategy, and network conditions [82, 101, 126]. Therefore, connectivity is not only a deployment-stage concern; it should influence earlier choices such as model size, feature extraction strategy, compression level, and whether the system supports federated learning, adaptive offloading, or runtime adaptation.

Monitoring, Drift Detection, and Update Mechanisms. Post-deployment monitoring, drift detection, and update mechanisms depend on earlier architectural and deployment decisions. Monitoring can reveal performance drift, anomalous behavior, or changing resource conditions, but acting on this information depends on earlier design decisions [61, 95, 127, 170]. If the model is monolithic, updating it may require replacing a larger firmware image, increasing OTA cost and rollback complexity. If the model and runtime are modular, only selected model binaries, layers, parameters, or related components may need replacement. RIOT-ML illustrates this interaction by showing that model-level or layer-level updates can reduce transfer costs compared with full firmware replacement, but such savings require earlier design choices that support modular model artifacts and compatible deployment runtimes [103]. Similarly, monitoring itself consumes memory, energy, and communication resources, which means that telemetry requirements should be considered during hardware selection and model design rather than added only after deployment [103, 137, 231].

Lifecycle Evaluation. These cross-layer dependencies suggest that edge AI deployment should be evaluated using multi-objective criteria that span the full lifecycle. Accuracy, latency, energy, memory footprint, portability, update size, monitoring overhead, and security cost should be considered jointly rather than in isolation [24, 25, 201, 248].

Overall, this section shows that edge AI deployment decisions are interdependent rather than confined to a single lifecycle stage. Table 9 illustrates this through representative examples, showing how selected choices in model design, toolchain selection, hardware mapping, partitioning, and monitoring can affect pre-deployment, deployment, and post-deployment outcomes. This highlights the need to evaluate deployment choices by their lifecycle-wide effects, not only by their immediate impact on inference performance.

7 Applications and Use Cases

Edge and embedded AI are increasingly deployed across a range of real-world domains where latency, privacy, energy efficiency, and autonomy are critical. Below, we highlight representative use cases that illustrate the practical relevance of edge deployment tools and optimizations.

7.1 Healthcare and Wearable AI

Edge AI has become a key enabler for wearable health monitoring, supporting on-device inference for vital signs, seizure detection, and stress monitoring by reducing latency, enhancing privacy, and enabling continuous operation within tight energy budgets [199, 209]. Representative systems include VersaSens, a modular platform combining heterogeneous processors with domain-specific accelerators; in seizure detection tasks it achieved a 33% reduction in inference latency and 8% lower energy consumption compared to baseline processing [172]. DeepWear, a runtime framework that partitions workloads between wearables and smartphones, adapts decisions at runtime and delivers up to 5× faster inference with 85% energy savings relative to wearable-only execution [268]. Other efforts address cardiovascular monitoring with hybrid edge-cloud CNNs (95% accuracy, < 40 ms latency) [215], mental wellness interventions using federated LLM frameworks (30% lower inference latency) [40], and non-invasive diabetes detection on microcontrollers (89.5% accuracy within 23 kB RAM) [85]. Collectively, these works highlight that scalable healthcare edge AI requires energy-aware execution, adaptive workload management, and privacy-preserving architectures aligned with real-world constraints.

7.2 Surveillance and Smart Cities

Edge AI is reshaping smart city infrastructure by enabling real-time, privacy-preserving analysis of video and sensor streams on local devices rather than in the cloud. This reduces bandwidth demands and latency, which is crucial for traffic monitoring, anomaly detection, and environmental surveillance. Representative systems include Barbutto et al.'s hybrid fog-edge traffic monitoring platform, which combines Coral TPUs and ESP32 microcontrollers, achieving sub-20 ms latency and up to 85.8% mAP for vehicle detection while reducing upstream

bandwidth by four orders of magnitude [26], and SurveilEdge, a scalable cloud-edge query framework that integrates MobileNet-v2 at the edge with ResNet-152 in the cloud, cutting query latency by 5.4× and bandwidth by 7× while improving accuracy by 43.9% [254].

Other efforts deploy lightweight CNNs directly on Raspberry Pi and Coral TPU boards to achieve >40 FPS object detection with 60% lower latency than cloud-only baselines [15], design fully embedded noise anomaly detectors on ESP32-S3 MCUs with 99.99% accuracy at 4 ms latency and 0.33 W power draw [89], and explore hierarchical edge-cloud partitioning with reinforcement learning, reaching 99.01% accuracy and reducing latency by 84.9% [176].

These studies show that dynamic workload allocation, lightweight models, and adaptive partitioning improve scalability, responsiveness, and privacy in urban surveillance.

7.3 Drones and Autonomous Mobility

Edge AI enables autonomous drones and ground robots to perform real-time perception and control under strict latency, power, and bandwidth constraints, supporting tasks such as visual tracking, obstacle avoidance, and infrastructure inspection. Representative efforts include Lamberti et al.'s nano-drone platform integrating an STM32F405 and GAP8 SoC, which executed a quantized SSD-MobileNetV2 at only 134 mW while achieving 1.6 FPS and 90% accuracy during autonomous indoor flight [135], and Santos et al.'s inspection robot running YOLOv7 on a Jetson Xavier NX, delivering 25.5 FPS at 720×480 resolution with low CPU and RAM overhead [57]. Other work targets UAV-based infrastructure monitoring, where a quantized YOLOv6s-GRE model reached 66.7% mAP at 523 FPS with 30 J per inference, yielding an 83.5% speedup and 79.7% energy reduction over the baseline [270]. Collectively, these results illustrate the feasibility of real-time, energy-aware inference for autonomous mobility on highly resource-constrained aerial and ground platforms.

7.4 Smart Manufacturing and Predictive Maintenance

Edge AI enables predictive maintenance by supporting low-latency inference directly on embedded devices, reducing reliance on cloud connectivity and improving data privacy in factory environments. Artiushenko et al. implemented a tool condition monitoring system on a Raspberry Pi 4 with piezoelectric sensors, where a stacked ensemble achieved 97% accuracy and a 1D-CNN reached 94% with 156 ms runtime, enabling sub-second updates for milling operations [21]. Ge et al. developed YOLO-MSD, a lightweight detector for high-resolution defect inspection, achieving 96.7% mAP on PCB datasets and sustaining 20.8 FPS at 6.95 W on a Jetson Xavier NX [79]. Other work, such as LEAF-YOLO, demonstrates that lightweight models initially designed for agriculture (88.9% mAP on Raspberry Pi 4B) can be adapted for fine-grained anomaly detection in manufacturing settings [175]. These studies show that lightweight CNNs can support reliable, real-time quality monitoring within the resource and energy constraints of industrial platforms.

7.5 Agriculture and Environmental Monitoring

Edge AI enables real-time agricultural and environmental monitoring directly on-site, reducing reliance on connectivity while supporting latency- and energy-critical tasks such as weed detection, crop disease classification, and forest fire alerts. Islam et al. demonstrated UAV-based weed detection using lightweight SSD, YOLOv8n, and MobileNetV4-Seg models on Jetson platforms, achieving IoU scores of 69.9-76.8% with F1 above 82% while sustaining 44 FPS on the Orin Nano [113]. Majeed et al. evaluated tomato disease detection across four edge platforms, where GoogleNet and MobileNetV2 achieved 98.3% accuracy, and SqueezeNet provided 3.5 ms inference, with Coral TPU boards delivering 952 FPS at the best cost-performance ratio [161]. Other efforts address environmental hazards, such as an explainable forest fire detector that combines CNNs and Vision Transformers with pruning, distillation, and quantization, reaching 98.7% accuracy and >6× lower energy use on Jetson-class

devices [160]. Collectively, these works show that edge AI can support responsive, resource-efficient inference for agriculture and environmental safety in bandwidth and power-constrained field deployments.

The use cases reviewed in this section show that edge AI has no single deployment profile. Healthcare, smart cities, mobility, manufacturing, and agriculture each emphasize different combinations of privacy, latency, bandwidth, energy, and robustness. Consequently, deployment choices must be evaluated in relation to the operational setting.

8 Challenges and Future Directions

Despite significant advances in algorithms, deployment toolchains, and embedded computing platforms, scalable and reliable edge AI deployment on resource-constrained devices remains constrained by several systemic challenges. This section highlights the main recurring challenges that emerge from the literature and identifies promising directions for future work.

Fragmentation and Portability. Edge AI deployment pipelines often consist of loosely connected stages for model optimization, compilation, and runtime execution. This fragmentation reduces portability, as workflows are frequently tied to specific processor architectures, operator sets, and vendor libraries [24, 59]. The absence of universally accepted intermediate representations and modular design principles prevents seamless interoperability between toolchains. Future work must explore adaptive abstraction layers and standardized operator sets to balance developer accessibility with transparent control over optimization trade-offs [49, 136].

Consistent and Reproducible Evaluation. Performance evaluation in edge AI lacks methodological consistency. Metrics such as latency, accuracy, and energy efficiency are reported under heterogeneous preprocessing pipelines and experimental setups, limiting comparability across studies [24, 201]. While benchmark suites such as MLPerf Tiny and MLMark provide standardized tasks and metrics, broader adoption of unified evaluation frameworks is needed to cover diverse workloads and end-to-end system behaviors. An open problem is to extend these benchmarks toward multi-objective reporting (e.g., Pareto frontiers) that reflect trade-offs between latency, energy, and accuracy.

Adaptive Deployment. While much prior work targets homogeneous and static system settings, real-world edge deployments must operate under heterogeneous hardware, fluctuating network conditions, limited power budgets, and dynamically changing data distributions [101, 286]. Current methods for dynamic scheduling and adaptive inference are fragmented across research prototypes and are rarely integrated into commercial toolchains. Promising directions include runtime decision-making for model complexity and offloading strategies, as well as training methods that embed deployment constraints directly into architecture design [232].

Security and Privacy. Edge deployments face growing risks to data privacy, model integrity, and resilience against adversarial manipulation [67, 191, 231]. Security measures such as encrypted storage, secure boot, and OTA integrity verification are often implemented independently rather than through an integrated lifecycle. However, quantization and device limitations can weaken the adversarial robustness of TinyML models, underscoring the importance of addressing security and privacy consistently throughout the deployment process while operating within tight energy and memory constraints.

Emerging Workloads. The evolution of edge AI workloads is reshaping the assumptions behind constrained-device deployment. While many TinyML and embedded-AI systems remain centered on compact CNNs, keyword-spotting models, image classifiers, and sensor classifiers [24, 209], recent progress in mobile vision transformers, large language models, diffusion and other generative models, multimodal learning, and vision-language-action (VLA) models points toward workloads with different computational and memory profiles [47, 140, 164, 208, 214,

266, 282]. Attention mechanisms in transformers and LLMs introduce sequence-dependent computation, large intermediate activations, key-value cache pressure, and memory-access patterns that differ from convolutional pipelines [47, 208]. Diffusion models add further pressure through repeated denoising or generation steps, while VLA and multimodal models combine heterogeneous inputs, multiple encoders, and action-oriented outputs [214, 282]. As a result, deployment bottlenecks increasingly extend beyond model size to memory bandwidth, activation storage, operator coverage, dynamic-shape handling, and runtime coordination across heterogeneous compute units [17, 136, 223, 284]. Future research is needed to determine which parts of these workloads can be compressed, partitioned, cached, or selectively offloaded while still satisfying edge constraints on latency, energy, privacy, and reliability.

Hardware-Software Co-Design for Emerging Edge AI. The growing complexity of edge workloads strengthens the need for hardware-software co-design across model design, compiler optimization, runtime scheduling, and accelerator execution. Compiler infrastructures and intermediate representations must support dynamic execution patterns, variable input shapes, memory reuse, and heterogeneous partitioning across CPUs, GPUs, NPUs, and other accelerators [49, 136, 187, 284]. At the hardware and runtime levels, these trends increase the importance of tensor acceleration, low-precision arithmetic, memory-aware execution, energy-aware adaptation, and accelerator-aware scheduling [23, 147, 166, 187, 253]. Recent systems show that edge deployment benefits when these layers are optimized together: MCUNet co-designs neural architecture search with a memory-efficient inference engine for microcontrollers, JAQ jointly explores network architecture, ultra-low-bit quantization, compiler mapping, and accelerator architecture, while Protean combines heterogeneous battery-free hardware, adaptive runtime scheduling, and code generation for intermittent ML execution [23, 144, 253]. Future deployment pipelines should therefore move beyond treating model compression, operator lowering, memory planning, accelerator mapping, and runtime scheduling as separate stages. This is especially important if edge AI is to evolve from isolated compact inference tasks toward adaptive and interactive intelligence under constrained power, memory, and connectivity budgets.

These challenges point to a common limitation in current edge AI deployment: optimization, compilation, benchmarking, adaptation, and security are still often addressed as separate problems. A central direction for future work is therefore to develop deployment workflows that can coordinate these concerns across heterogeneous devices, evolving workloads, and long-term lifecycle requirements.

9 Conclusion

This survey reviewed edge AI deployment on constrained devices across optimization, toolchains, lifecycle management, benchmarking, and applications. The literature shows strong progress in model compression, hardware-aware execution, and deployment tooling, but weaker integration of portability, monitoring, secure updates, runtime adaptation, and reproducible evaluation. This fragmentation remains a key barrier to scalable edge AI deployment. Future pipelines should therefore move beyond isolated model or compiler optimizations toward lifecycle-aware workflows that support efficient, observable, updatable, and secure operation across heterogeneous devices.

References

- [1] 2016. Raspberry Pi 3 Model B. <https://www.raspberrypi.com/products/raspberry-pi-3-model-b/>.
- [2] 2017. Open Neural Network Exchange (ONNX). <https://onnx.ai/>.
- [3] 2017. TensorFlow Lite. <https://ai.google.dev/edge/litert>.
- [4] 2018. ARM's Project Trillium offers scalable new machine learning processors. <https://newsroom.arm.com/news/arm-project-trillium-offers-the-industrys-most-scalable-versatile-ml-compute-platform>.
- [5] 2019. Jetson Nano Developer Kit. <https://developer.nvidia.com/embedded/jetson-nano>.

- [6] 2025. AWS IoT Greengrass Documentation. <https://docs.aws.amazon.com/greengrass/v2/developer/what-is-gg.html>. Accessed: 2025-09-08.
- [7] 2025. Azure IoT Edge Documentation. <https://learn.microsoft.com/en-us/azure/iot-edge/about-iot-edge>. Accessed: 2025-09-08.
- [8] 2025. Balena IoT fleet management platform. <https://www.balena.io/>. Accessed: Sep. 2025.
- [9] 2025. Mender - OTA software updates for Linux devices. <https://mender.io/>. Accessed: Sep. 2025.
- [10] 2025. ONNX Runtime — Execution Providers. <https://onnxruntime.ai/docs/execution-providers/>. Accessed 2025-09-08.
- [11] 2025. Particle Cloud IoT platform. <https://www.particle.io/>. Accessed: Sep. 2025.
- [12] Dina Aboutahoun, Rami Zewail, and Mostafa I. Soliman. 2022. On Potentials of Few-Shot Learning for AI-Enabled Internet of Medical Things. In *2022 IEEE Globecom Workshops (GC Wkshps)*. 1062–1067. doi:10.1109/GCWkshps56602.2022.10008512
- [13] Syed Thouheed Ahmed, A. C. Kaladevi, V. Vinoth Kumar, Achyut Shankar, and Fayeze Alqahtani. 2025. Privacy Enhanced Edge-AI Healthcare Devices Authentication: A Federated Learning Approach. *IEEE Transactions on Consumer Electronics* 71, 2 (2025), 5676–5682. doi:10.1109/TCE.2025.3542955
- [14] Shamil Al-Ameen et al. 2024. Tiny-Impute: A Framework for On-device Data Quality Validation, Hybrid Anomaly Detection, and Data Imputation at the Edge. In *Proceedings of the IEEE/ACM 16th International Conference on Utility and Cloud Computing*. Association for Computing Machinery, Article 23. doi:10.1145/3603166.3632164
- [15] Kasra Aminiyeeganeh, Rodolfo W.L. Coutinho, and Azzedine Boukerche. 2024. IoT video analytics for surveillance-based systems in smart cities. *Computer Communications* 224 (2024), 95–105. doi:10.1016/j.comcom.2024.05.021
- [16] Marco Antonini, Mauricio Pincheira, Matteo Vecchio, and Fabio Antonelli. 2023. An Adaptable and Unsupervised TinyML Anomaly Detection System for Extreme Industrial Environments. *Sensors* 23, 4 (Feb. 2023), 2344. doi:10.3390/s23042344
- [17] Apache TVM Community. 2021. Apache TVM Unity: a vision for the ML software & hardware ecosystem in 2022. <https://tvm.apache.org/2021/12/15/tvm-unity>
- [18] Konstantinos Arakadakis, Pavlos Charalampidis, Antonis Makrogiannakis, and Alexandros Fragkiadakis. 2021. Firmware Over-the-air Programming Techniques for IoT Networks - A Survey. *ACM Comput. Surv.* 54, 9, Article 178 (Oct. 2021), 36 pages. doi:10.1145/3472292
- [19] ARM. 2024. ARM Trusted Firmware: Trusted Execution and Secure Boot. <https://www.trustedfirmware.org/>
- [20] Arm. 2025. CMSIS-DSP: Overview. <https://arm-software.github.io/CMSIS-DSP/main/>. Accessed: 2025-08-21.
- [21] Viktor Artiushenko et al. 2024. Resource-efficient Edge AI solution for predictive maintenance. *Procedia Computer Science* 232 (2024), 348–357. doi:10.1016/j.procs.2024.01.034 5th International Conference on Industry 4.0 and Smart Manufacturing (ISM 2023).
- [22] D.K. Arvind, R.D. Mullins, and V.E.F. Rebello. 1995. Micronets: a model for decentralising control in asynchronous processor architectures. In *Proceedings Second Working Conference on Asynchronous Design Methodologies*. 190–199. doi:10.1109/WCADM.1995.514656
- [23] Abu Bakar et al. 2023. Protean: An Energy-Efficient and Heterogeneous Platform for Adaptive and Hardware-Accelerated Battery-Free Computing. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems (SenSys '22)*. Association for Computing Machinery, 207–221. doi:10.1145/3560905.3568561
- [24] Colby Banbury et al. 2021. MLPerf Tiny Benchmark. *arXiv preprint arXiv:2106.07597* (2021). <https://arxiv.org/abs/2106.07597>
- [25] Colby R Banbury et al. 2020. Benchmarking TinyML Systems: Challenges and Direction. *arXiv preprint arXiv:2003.04821* (2020). doi:10.48550/arXiv.2003.04821
- [26] Vincenzo Barbuto, Claudio Savaglio, Roberto Minerva, Noel Crespi, and Giancarlo Fortino. 2023. Towards an Edge Intelligence-Based Traffic Monitoring System. In *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*. 3434–3439. doi:10.1109/SMC53992.2023.10393907
- [27] Pietro Bartoli et al. 2025. Benchmarking Energy and Latency in TinyML: A Novel Method for Resource-Constrained AI. In *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8. doi:10.1109/ijcnn64981.2025.11228997
- [28] Jan Bauwens, Peter Ruckebusch, Spilios Giannoulis, Ingrid Moerman, and Eli De Poorter. 2020. Over-the-Air Software Updates in the Internet of Things: An Overview of Key Principles. *IEEE Communications Magazine* 58, 2 (2020), 35–41. doi:10.1109/MCOM.001.1900125
- [29] Mohammed Djameleddine Belgoumri, Mohamed Reda Bouadjene, Sunil Aryal, and Hakim Hacid. 2024. Data Quality in Edge Machine Learning: A State-of-the-Art Survey. *arXiv preprint arXiv:2406.02600* (2024). doi:10.48550/arXiv.2406.02600
- [30] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Hei Li Kwing, Titouan Parcollet, Pedro PB de Gusmão, and Nicholas D Lane. 2020. Flower: A Friendly Federated Learning Research Framework. *arXiv preprint arXiv:2007.14390* (2020).
- [31] Romil Bhardwaj et al. 2022. Ekya: Continuous Learning of Video Analytics Models on Edge Compute Servers. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 119–135. <https://www.usenix.org/conference/nsdi22/presentation/bhardwaj>
- [32] Arpan Bhattacharjee et al. 2024. Poster: Edge-Assisted Over-the-Air Software Updates. In *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing* (Wilmington, DE, USA) (SEC '23). Association for Computing Machinery, New York, NY, USA, 285–286. doi:10.1145/3583740.3628425
- [33] Aart Bik, Penporn Koanantakool, Tatiana Shpeisman, Nicolas Vasilache, Bixia Zheng, and Fredrik Kjolstad. 2022. Compiler Support for Sparse Tensor Computations in MLIR. *ACM Trans. Archit. Code Optim.* 19, 4, Article 50 (Sept. 2022), 25 pages. doi:10.1145/3544559

- [34] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. 2020. YOLOv4: Optimal Speed and Accuracy of Object Detection. arXiv:2004.10934 [cs.CV] doi:10.48550/arXiv.2004.10934
- [35] Keith Bonawitz et al. 2019. Towards Federated Learning at Scale: System Design. In *Proceedings of the 2nd SysML Conference*. <https://arxiv.org/abs/1902.01046>
- [36] Flavio Bonomi et al. 2012. Fog computing and its role in the internet of things. In *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing (Helsinki, Finland) (MCC '12)*. Association for Computing Machinery, New York, NY, USA, 13–16. doi:10.1145/2342509.2342513
- [37] Siddhartha S. Borkotoky. 2025. Balancing the Energy Consumption and Latency of Over-the-Air Firmware Updates in LoRaWAN. arXiv:2508.10588 [cs.NI] <https://arxiv.org/abs/2508.10588>
- [38] Conner Bradley and David Barrera. 2022. Towards Characterizing IoT Software Update Practices. In *Foundations and Practice of Security: 15th International Symposium, FPS 2022, Ottawa, ON, Canada, December 12–14, 2022, Revised Selected Papers*. Springer-Verlag, Berlin, Heidelberg, 406–422. doi:10.1007/978-3-031-30122-3_25
- [39] Marcelo Brandalero et al. 2020. AITIA: Embedded AI Techniques for Embedded Industrial Applications. In *2020 International Conference on Omni-layer Intelligent Systems (COINS)*. 1–7. doi:10.1109/COINS49042.2020.9191672
- [40] Salman Bukhari, Ravi Sharma, Amril Nazir, and Edmund Evangelista. 2025. Wearable IoMT, Edge AI and LLMs for Monitoring Health and Wellness: A Harmonization Framework for Responsible AI Interventions. Preprint on Cambridge Open Engage. doi:10.33774/coe-2025-1cnc9
- [41] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. 2020. Albumentations: Fast and Flexible Image Augmentations. *Information* 11, 2 (2020). doi:10.3390/info11020125
- [42] Han Cai, Chuang Gan, Tianzhe Wang, Zhekai Zhang, and Song Han. 2019. Once-for-All: Train One Network and Specialize it for Efficient Deployment. *arXiv preprint arXiv:1908.09791* (2019). doi:10.48550/arXiv.1908.09791
- [43] Han Cai, Ligeng Zhu, and Song Han. 2018. ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware. *arXiv preprint arXiv:1812.00332* (2018). doi:10.48550/arXiv.1812.00332
- [44] Fabio Campos, Tim Kohlstadt, Steffen Reith, and Marc Stoettinger. 2020. LMS vs XMSS: Comparison of Stateful Hash-Based Signature Schemes on ARM Cortex-M4. *Cryptology ePrint Archive*, Paper 2020/470. <https://eprint.iacr.org/2020/470>
- [45] Riccardo Cantini, Alessio Orsino, and Domenico Talia. 2024. XAI-driven knowledge distillation of large language models for efficient deployment on low-resource devices. *Journal of Big Data* 11, 1 (2024), 63. doi:10.1186/s40537-024-00928-3
- [46] CEVA, Inc. 2025. 2025 Edge AI Technology Report. <https://www.ceva-ip.com/wp-content/uploads/2025-Edge-AI-Technology-Report.pdf>
- [47] Jiyu Chen, Shuang Peng, Daxiong Luo, Fan Yang, Renshou Wu, Fangyuan Li, and Xiaoxin Chen. 2025. EdgeInfinite: A Memory-Efficient Infinite-Context Transformer for Edge Devices. arXiv:2503.22196 [cs.CL] <https://arxiv.org/abs/2503.22196>
- [48] Jiasi Chen and Xukan Ran. 2019. Deep Learning With Edge Computing: A Review. *Proc. IEEE* 107, 8 (2019), 1655–1674. doi:10.1109/JPROC.2019.2921977
- [49] Tianqi Chen et al. 2018. TVM: an automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (Carlsbad, CA, USA) (OSDI'18)*. USENIX Association, USA, 579–594.
- [50] Dwith Chenna. 2023. Edge AI: Quantization as the Key to On-Device Smartness. *International Journal of Artificial Intelligence & Applications (IJAIAP)* 2, 1 (2023), 58–69. doi:10.17605/OSF.IO/H8ERT
- [51] Sergei Chuprov, Raman Zatsarenko, Dmitrii Korobeinikov, and Leon Reznik. 2024. Robust Training on the Edge: Federated vs. Transfer Learning for Computer Vision in Intelligent Transportation Systems. In *2024 IEEE World AI IoT Congress (AllIoT)*. 172–178. doi:10.1109/AllIoT61789.2024.10578970
- [52] Cristian Cioflan et al. 2024. On-Device Domain Learning for Keyword Spotting on Low-Power Extreme Edge Embedded Systems. 6–10. doi:10.1109/AICAS59952.2024.10595987
- [53] Google Cloud. 2018. Bringing machine learning to the edge with Google Cloud IoT Edge and Edge TPU. <https://cloud.google.com/blog/products/gcp/bringing-intelligence-edge-cloud-iot>.
- [54] Ivan Culjak, David Abram, Tomislav Pribanic, Hrvoje Dzapov, and Mario Cifrek. 2012. A brief introduction to OpenCV. (2012), 1725–1730.
- [55] Bruno da Silva, Axel W. Happi, An Braeken, and Abdellah Touhafi. 2019. Evaluation of Classical Machine Learning Techniques towards Urban Sound Recognition on Embedded Systems. *Applied Sciences* 9, 18 (2019). doi:10.3390/app9183885
- [56] Mike Davies, Andreas Wild, Garrick Orchard, Yulia Sandamirskaya, Gabriel A. Fonseca Guerra, Prasad Joshi, Philipp Plank, and Sumedh R. Risbud. 2021. Advancing Neuromorphic Computing With Loihi: A Survey of Results and Outlook. *Proc. IEEE* 109, 5 (2021), 911–934. doi:10.1109/JPROC.2021.3067593
- [57] Ricardo C. Câmara de M. Santos et al. 2023. Towards Autonomous Mobile Inspection Robots Using Edge AI. In *Proceedings of the 25th International Conference on Enterprise Information Systems (ICEIS 2023) - Volume 1*. SCITEPRESS – Science and Technology Publications, Lda, 555–562. doi:10.5220/0011972200003467
- [58] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80. doi:10.1145/2408776.2408794

- [59] Shuiguang Deng, Hailiang Zhao, Weijia Fang, Jianwei Yin, Schahram Dustdar, and Albert Y. Zomaya. 2020. Edge Intelligence: The Confluence of Edge Computing and Artificial Intelligence. *IEEE Internet of Things Journal* 7, 8 (2020), 7457–7469. doi:10.1109/JIOT.2020.2984887
- [60] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. 2024. D-STACK: High Throughput DNN Inference by Effective Multiplexing and Spatio-Temporal Scheduling of GPUs. *IEEE Transactions on Cloud Computing* (2024).
- [61] Simone Disabato and Manuel Roveri. 2021. Tiny Machine Learning for Concept Drift. arXiv:2107.14759 [cs.LG] <https://arxiv.org/abs/2107.14759>
- [62] Shi Dong, Junxiao Tang, Khushnood Abbas, Ruizhe Hou, Joarder Kamruzzaman, Leszek Rutkowski, and Rajkumar Buyya. 2024. Task offloading strategies for mobile edge computing: A survey. *Computer Networks* 254 (2024), 110791. doi:10.1016/j.comnet.2024.110791
- [63] Dr. Lachit Dutta and Swapna Bharali. 2021. TinyML Meets IoT: A Comprehensive Survey. *Internet of Things* 16 (2021), 100461. doi:10.1016/j.iot.2021.100461
- [64] Edge Impulse. 2024. Edge AI Lifecycle. <https://docs.edgeimpulse.com/docs/concepts/edge-ai-fundamentals/edge-ai-lifecycle>
- [65] Edge Impulse. 2024. Edge Impulse Documentation. <https://docs.edgeimpulse.com/docs>
- [66] Saad El Jaouhari and Eric Bouvet. 2022. Secure firmware Over-The-Air updates for IoT: Survey, challenges, and discussions. *Internet of Things* 18 (2022), 100508. doi:10.1016/j.iot.2022.100508
- [67] Khalid Elgazzar, Haytham Khalil, Taghreed Alghamdi, Ahmed Badr, Ghadeer Abdelkader, Abdelrahman Elewah, and Rajkumar Buyya. 2022. Revisiting the internet of things: New trends, opportunities and grand challenges. *Frontiers in the Internet of Things* 1 (Nov. 2022), 1073780. doi:10.3389/friot.2022.1073780
- [68] Embedded Vision Alliance. 2011. Welcome to the Embedded Vision Alliance! Blog post on Edge AI and Vision Alliance. <https://www.edge-ai-vision.com/2011/05/welcome-to-the-embedded-vision-alliance/> Published May 2011.
- [69] Linus Ericsson, Henry Gouk, Chen Change Loy, and Timothy M. Hospedales. 2022. Self-Supervised Representation Learning: Introduction, advances, and challenges. *IEEE Signal Processing Magazine* 39, 3 (2022), 42–62. doi:10.1109/MSP.2021.3134634
- [70] ETSI. 2014. Mobile-Edge Computing Initiative – Executive Briefing, Issue 1. https://portal.etsi.org/Portals/0/TBpages/MEC/Docs/Mobile-edge_Computing_-_Introductory_Technical_White_Paper_V1%2018-09-14.pdf
- [71] ETSI. 2017. ETSI Multi-access Edge Computing starts second phase and renews leadership team. <https://www.etsi.org/newsroom/news/1180-2017-03-news-etsi-multi-access-edge-computing-starts-second-phase-and-renews-leadership-team>.
- [72] Dave Evans. 2011. The Internet of Things: How the Next Evolution of the Internet Is Changing Everything. https://www.cisco.com/web/about/ac79/docs/innov/IoT_IBSG_041FINAL.pdf
- [73] Wai-Chi Fang et al. 2019. Development and Validation of an EEG-Based Real-Time Emotion Recognition System Using Edge AI Computing Platform With Convolutional Neural Network System-on-Chip Design. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9, 4 (2019), 645–657. doi:10.1109/JETCAS.2019.2951232
- [74] Samer Franczy and Raghuraj Singh. 2024. Edge AI: Evaluation of Model Compression Techniques for Convolutional Neural Networks. *arXiv preprint arXiv:2409.02134* (2024). doi:10.48550/arXiv.2409.02134
- [75] Dustin Franklin. 2015. NVIDIA Jetson TX1 Supercomputer-on-Module Drives Next Wave of Autonomous Machines. <https://developer.nvidia.com/blog/nvidia-jetson-tx1-supercomputer-on-module-drives-next-wave-of-autonomous-machines/> Accessed: 2025-10-06.
- [76] Cinthya M. França, Rodrigo S. Couto, and Pedro B. Velloso. 2021. Missing Data Imputation in Internet of Things Gateways. *Information* 12, 10 (2021). doi:10.3390/info12100425
- [77] Chinthaka Gamanayake, Lahiru Jayasinghe, Benny Kai Kiat Ng, and Chau Yuen. 2020. Cluster Pruning: An Efficient Filter Pruning Method for Edge AI Vision Applications. *IEEE Journal of Selected Topics in Signal Processing* 14, 4 (2020), 802–816. doi:10.1109/JSTSP.2020.2971418
- [78] Răzvan Gavriliuț and Ionuț Stoica. 2023. Edge AI Lifecycle Management. In *Edge AI*. CRC Press/Taylor & Francis, Chapter 2. doi:10.1201/9788770041027-2
- [79] Yifei Ge, Zhuo Li, and Lin Meng. 2025. YOLO-MSD: a robust industrial surface defect detection model via multi-scale feature fusion. *Applied Intelligence* 55, 12 (2025), 840. doi:10.1007/s10489-025-06739-0
- [80] Neil Gershenfeld, Raffi Krikorian, and Danny Cohen. 2004. The Internet of Things. *Scientific American* 291, 4 (2004), 76–81. https://fab.cba.mit.edu/classes/S62.12/docs/Cohen_Internet.pdf
- [81] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. 2021. A Survey of Quantization Methods for Efficient Neural Network Inference. *arXiv preprint arXiv:2103.13630* (2021). doi:10.48550/arXiv.2103.13630 Book Chapter: Low-Power Computer Vision: Improving the Efficiency of Artificial Intelligence.
- [82] Soumendu Kumar Ghosh, Arnab Raha, Vijay Raghunathan, and Anand Raghunathan. 2024. PArtNNer: Platform-Agnostic Adaptive Edge-Cloud DNN Partitioning for Minimizing End-to-End Latency. *ACM Trans. Embed. Comput. Syst.* 23, 1, Article 6 (Jan. 2024), 38 pages. doi:10.1145/3630266
- [83] Google. 2021. Introducing Google Tensor. <https://blog.google/products/pixel/introducing-google-tensor/>. Official Google announcement of the Tensor SoC.

- [84] Jianping Gou, Baosheng Yu, Stephen J. Maybank, and Dacheng Tao. 2021. Knowledge Distillation: A Survey. *International Journal of Computer Vision* 129 (2021), 1789–1819. doi:10.1007/s11263-021-01453-z
- [85] Maria Gragnaniello, Vincenzo Romano Marrazzo, Alessandro Borghese, Luca Maresca, Giovanni Breglio, and Michele Riccio. 2025. Edge-AI Enabled Wearable Device for Non-Invasive Type 1 Diabetes Detection Using ECG Signals. *Bioengineering* 12, 1 (2025). <https://www.mdpi.com/2306-5354/12/1/4>
- [86] Piotr Grzesik and Dariusz Mrozek. 2024. Combining Machine Learning and Edge Computing: Opportunities, Challenges, Platforms, Frameworks, and Use Cases. *Electronics* 13, 3 (2024), 640. doi:10.3390/electronics13030640 Open Access under CC BY 4.0.
- [87] Bin Guo, Daqing Zhang, Zhiwen Yu, et al. 2013. From the Internet of Things to Embedded Intelligence. *World Wide Web* 16, 4 (July 2013), 399–420. doi:10.1007/s11280-012-0188-y
- [88] Priyanka Gupta, Sathvik Bhaskarpandit, and Manik Gupta. 2021. Similarity Learning based Few Shot Learning for ECG Time Series Classification. In *2021 Digital Image Computing: Techniques and Applications (DICTA)*. 1–8. doi:10.1109/DICTA52665.2021.9647357
- [89] Sahibzada Saadoon Hammad, Ditsuhi Iskandaryan, and Sergio Trilles. 2023. An unsupervised TinyML approach applied to the detection of urban noise anomalies under the smart cities environment. *Internet of Things* 23 (2023), 100848. doi:10.1016/j.iot.2023.100848
- [90] Kai Han, Yunhe Wang, Qi Tian, Jianyuan Guo, Chunjing Xu, and Chang Xu. 2020. GhostNet: More Features from Cheap Operations. *arXiv preprint arXiv:1911.11907* (2020). doi:10.48550/arXiv.1911.11907 Presented at CVPR 2020.
- [91] Lixiang Han, Zimu Zhou, and Zhenjiang Li. 2024. Pantheon: Preemptible Multi-DNN Inference on Mobile Edge GPUs. In *Proceedings of the 22nd Annual International Conference on Mobile Systems, Applications and Services (Minato-ku, Tokyo, Japan) (MOBISYS '24)*. Association for Computing Machinery, New York, NY, USA, 465–478. doi:10.1145/3643832.3661878
- [92] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations (ICLR)*. doi:10.48550/arXiv.1510.00149
- [93] Osama A. Hanna, Yahya H. Ezzeldin, Christina Fragouli, and Suhas Diggavi. 2021. Quantization of Distributed Data for Learning. *IEEE Journal on Selected Areas in Information Theory* 2, 3 (2021), 987–1001. doi:10.1109/JSAIT.2021.3105359
- [94] Tianshu Hao, Yunyou Huang, Xu Wen, Wanling Gao, Fan Zhang, Chen Zheng, Lei Wang, Hainan Ye, Kai Hwang, Zujie Ren, and Jianfeng Zhan. 2018. Edge AIBench: Towards Comprehensive End-to-end Edge Computing Benchmarking. In *BenchCouncil International Symposium on Benchmarking, Measuring and Optimizing (Bench'18)*. BenchCouncil; Edge AIBench-Bench18.
- [95] Fabian Hinder, Valerie Vaquet, and Barbara Hammer. 2024. One or two things we know about concept drift—a survey on monitoring in evolving environments. Part A: detecting concept drift. *Frontiers in Artificial Intelligence* 7 (2024), 1330257. doi:10.3389/frai.2024.1330257
- [96] Torsten Hoefler, Dan Alistarh, Tal Ben-Nun, Nikoli Dryden, and Alexandra Peste. 2021. Sparsity in Deep Learning: Pruning and growth for efficient inference and training in neural networks. *Journal of Machine Learning Research* 22, 241 (2021), 1–124. <http://jmlr.org/papers/v22/21-0366.html>
- [97] Chenyu Hou, Kai Jiang, Tiantian Li, Meng Zhou, and Jun Jiang. 2025. Co-active: an efficient selective relabeling model for resource constrained edge AI. *Wirel. Netw.* 31, 3 (Feb. 2025), 2653–2666. doi:10.1007/s11276-025-03903-9
- [98] Andrew Howard, Mark Sandler, Bo Chen, Weijun Wang, Liang-Chieh Chen, Mingxing Tan, Grace Chu, Vijay Vasudevan, Yukun Zhu, Ruoming Pang, Hartwig Adam, and Quoc Le. 2019. Searching for MobileNetV3. In *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*. 1314–1324. doi:10.1109/ICCV.2019.00140
- [99] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv preprint arXiv:1704.04861* (2017). <https://doi.org/10.48550/arXiv.1704.04861>
- [100] Weiqi Hu, Huicheng Zhou, Jianzhong Yang, Enming Hui, and Chaoren Dai. 2023. Adaptive Quantization Range Division Technique for Electronic Control Data Compression in CNC Machine Tools. *Electronics* 12, 16 (2023). doi:10.3390/electronics12163387
- [101] Yang Hu, Connor Imes, Xuanang Zhao, Souvik Kundu, Peter A. Beerel, Stephen P. Crago, and John Paul N. Walters. 2021. Pipeline Parallelism for Inference on Heterogeneous Edge Computing. *arXiv preprint arXiv:2110.14895* (2021). doi:10.48550/arXiv.2110.14895 Submitted on 28 October 2021.
- [102] Zhaojing Huang, Luis Fernando Herbozo Contreras, Wing Hang Leung, Leping Yu, Nhan Duy Truong, Armin Nikpour, and Omid Kavehei. 2024. Efficient Edge-AI Models for Robust ECG Abnormality Detection on Resource-Constrained Hardware. *Journal of Cardiovascular Translational Research* 17, 4 (2024), 879–892. doi:10.1007/s12265-024-10504-y
- [103] Zhaolan Huang, Koen Zandberg, Kaspar Schleiser, and Emmanuel Baccelli. 2025. RIOT-ML: toolkit for over-the-air secure updates and performance evaluation of TinyML models. *Annals of Telecommunications* 80 (2025), 283–297. doi:10.1007/s12243-024-01041-5
- [104] Jacob Huckelberry, Yuke Zhang, Allison Sansone, James Mickens, Peter Bereel, and Vijay Janapa Reddi. 2026. TinyML Security: Attacks, Defenses, and Open Challenges in Resource-Constrained Machine Learning Systems. *ACM Comput. Surv.* (Feb. 2026). doi:10.1145/3793197
- [105] Chung-Wen Hung, Cheng-Yu Tsai, Chun-Chieh Wang, and Ching-Hung Lee. 2025. TinyFL_HKD: Enhancing Edge AI Federated Learning With Hierarchical Knowledge Distillation Framework. *IEEE Sensors Journal* 25, 7 (2025), 12038–12047. doi:10.1109/JSEN.2025.3544861
- [106] Muhammad Hussain. 2024. YOLOv5, YOLOv8 and YOLOv10: The Go-To Detectors for Real-time Vision. *arXiv:2407.02988 [cs.CV]* doi:10.48550/arXiv.2407.02988

- [107] Shawn Hymel, Colby Banbury, Daniel Situnayake, Alex Ellum, Carl Ward, Mat Kelcey, Mathijs Baaijens, Mateusz Majchrzycki, Jenny Plunkett, David Tischler, Alessandro Grande, Louis Moreau, Dmitry Maslov, Artie Beavis, Jan Jongboom, and Vijay Janapa Reddi. 2023. Edge Impulse: An MLOps Platform for Tiny Machine Learning. arXiv:2212.03332 [cs.DC] <https://arxiv.org/abs/2212.03332>
- [108] Forrest N. Iandola, Song Han, Matthew W. Moskewicz, Khalid Ashraf, William J. Dally, and Kurt Keutzer. 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size. *arXiv preprint arXiv:1602.07360* (2016). doi:10.48550/arXiv.1602.07360
- [109] IDC. 2025. IDC Estimates Global Spending on Edge Computing to Approach \$380 Billion by 2028. <https://www.idc.com/getdoc.jsp?containerId=prUS53261225> Press release reporting 2025 baseline and 2028 projection.
- [110] BrainChip Inc. 2022. Akida Product Brief. https://brainchip.com/wp-content/uploads/2022/06/Akida-1.0-IP-Product-Brief_final.pdf.
- [111] Xilinx Inc. 2023. Vitis AI Development Environment. <https://www.xilinx.com/products/design-tools/vitis/vitis-ai.html>.
- [112] OpenVINO Toolkit-Deep Learning Deployment Intel. 2018. Toolkit repository. *GitHub Repository* (2018).
- [113] Md Didarul Islam, Wenxin Liu, Pascal Izere, Puranjit Singh, Chong Yu, Benjamin Riggan, Kuan Zhang, Amit J. Jhala, Stevan Knezevic, Yufeng Ge, Santosh Pitla, Joe Luck, and Yeyin Shi. 2025. Towards real-time weed detection and segmentation with lightweight CNN models on edge devices. *Computers and Electronics in Agriculture* 237 (2025), 110600. doi:10.1016/j.compag.2025.110600
- [114] Victormills Iyieke, Hesamaldin Jadidbonab, Abdur Rakib, Jeremy Bryans, Don Dhaliwal, and Odysseas Kosmas. 2025. An adaptable security-by-design approach for ensuring a secure Over the Air (OTA) update in modern vehicles. *Computers & Security* 150 (2025), 104268. doi:10.1016/j.cose.2024.104268
- [115] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2704–2713.
- [116] Purvish Jajal, Wenxin Jiang, Arav Tewari, Erik Kocinare, Joseph Woo, Anusha Sarraf, Yung-Hsiang Lu, George K. Thiruvathukal, and James C. Davis. 2024. Interoperability in Deep Learning: A User Survey and Failure Analysis of ONNX Model Converters (*ISSTA 2024*). Association for Computing Machinery, New York, NY, USA, 1466–1478. doi:10.1145/3650212.3680374
- [117] Petar Jakuš and Hrvoje Džapo. 2025. Implementing Keyword Spotting on the MCUX947 Microcontroller with Integrated NPU. *arXiv preprint arXiv:2506.08911* (2025). <https://arxiv.org/abs/2506.08911>
- [118] Jan Moritz Joseph, Murat Sezgin Baloglu, Yue Pan, Rainer Leupers, and Lennart Bamberg. 2021. NEWROMAP: mapping CNNs to NoC-interconnected self-contained data-flow accelerators for edge-AI. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3479876.3481591
- [119] Oumayma Jouini, Kaouthar Sethom, Abdallah Namoun, Nasser Aljohani, Meshari Huwaytim Alanazi, and Mohammad N. Alanazi. 2024. A Survey of Machine Learning in Edge Computing: Techniques, Frameworks, Applications, Issues, and Research Directions. *Technologies* 12, 6 (2024). doi:10.3390/technologies12060081
- [120] Hyung Kook Jun, Taeho Kim, Sang Cheol Kim, and Young Ik Eom. 2025. A Hierarchical Dispatcher for Scheduling Multiple Deep Neural Networks (DNNs) on Edge Devices. *Sensors* 25, 7 (2025). doi:10.3390/s25072243
- [121] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Keith Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al. 2021. Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning* 14, 1–2 (2021), 1–210. doi:10.1561/22000000083
- [122] R. Kanda, H. L. Blevec, N. Onizawa, M. Leonardon, V. Gripon, and T. Hanyu. 2025. Bit-Width-Aware Design Environment for Few-Shot Learning on Edge AI Hardware. In *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. doi:10.1109/ISCAS56072.2025.11044227
- [123] R. Kanda, N. Onizawa, M. Leonardon, V. Gripon, and T. Hanyu. 2024. Design Environment of Quantization-Aware Edge AI Hardware for Few-Shot Learning. In *2024 IEEE 67th International Midwest Symposium on Circuits and Systems (MWSCAS)*. 928–931. doi:10.1109/MWSCAS60917.2024.10658789
- [124] Rahima Khanam and Muhammad Hussain. 2024. YOLOv11: An Overview of the Key Architectural Enhancements. arXiv:2410.17725 [cs.CV] doi:10.48550/arXiv.2410.17725
- [125] Grace Y.E. Kim, Conor K. Corbin, François Grolleau, Michael Baiocchi, and Jonathan H. Chen. 2025. Monitoring strategies for continuous evaluation of deployed clinical prediction models. *Journal of Biomedical Informatics* 168 (2025), 104854. doi:10.1016/j.jbi.2025.104854
- [126] Hyochan Kim, Ji Sub Choi, Jungrae Kim, and Jong Hwan Ko. 2024. A DNN partitioning framework with controlled lossy mechanisms for edge-cloud collaborative intelligence. *Future Generation Computer Systems* 154 (2024), 426–439. doi:10.1016/j.future.2024.01.006
- [127] Janis Klaise, Arnaud Van Looveren, Clive Cox, Giovanni Vacanti, and Alexandru Coca. 2020. Monitoring and explainability of models in production. arXiv:2007.06299 [stat.ML] <https://arxiv.org/abs/2007.06299>
- [128] Pang Wei Koh, Shiori Sagawa, Henrik Marklund, Sang Michael Xie, Marvin Zhang, Akshay Balsubramani, Weihua Hu, Michihiro Yasunaga, Richard Lanas Phillips, Irena Gao, Tony Lee, Etienne David, Ian Stavness, Wei Guo, Berton Earnshaw, Imran Haque, Sara M Beery, Jure Leskovec, Anshul Kundaje, Emma Pierson, Sergey Levine, Chelsea Finn, and Percy Liang. 2021. WILDS: A Benchmark of in-the-Wild Distribution Shifts. In *Proceedings of the 38th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 139)*, Marina Meila and Tong Zhang (Eds.). PMLR, 5637–5664. <https://proceedings.mlr.press/v139/koh21a.html>

- [129] Anil Kore, Elham Abbasi Babil, V. Subasri, et al. 2024. Empirical data drift detection experiments on real-world medical imaging data. *Nature Communications* 15 (2024), 1887. doi:10.1038/s41467-024-46142-w
- [130] Scott Kovach, Praneeth Kolichala, Tiancheng Gu, and Fredrik Kjolstad. 2023. Indexed Streams: A Formal Intermediate Representation for Fused Contraction Programs. *Proc. ACM Program. Lang.* 7, PLDI, Article 154 (June 2023), 25 pages. doi:10.1145/3591268
- [131] Yunus Koç, Ahmet Ayberk Tarçın, and Doğukan Köse. 2024. Evaluation of Voice Recognition Platforms and Methods for Edge AI Devices. In *2024 8th International Artificial Intelligence and Data Processing Symposium (IDAP)*. 1–6. doi:10.1109/IDAP64064.2024.10711025
- [132] Endah Kristiani, Yu-Tse Tsan, Po-Yu Liu, Neil Yuwen Yen, and Chao-Tung Yang. 2022. Binary and Multi-Class Assessment of Face Mask Classification on Edge AI Using CNN and Transfer Learning. *Human-centric Computing and Information Sciences* 12, 53 (2022). doi:10.22967/HGIS.2022.12.053
- [133] Seyyidahmed Lahmer, Aria Khoshshirat, Michele Rossi, and Andrea Zanella. 2022. Energy Consumption of Neural Networks on NVIDIA Edge Boards: an Empirical Model. In *2022 20th International Symposium on Modeling and Optimization in Mobile, Ad hoc, and Wireless Networks (WiOpt)*. 365–371. doi:10.23919/WiOpt56218.2022.9930584
- [134] Liangzhen Lai, Naveen Suda, and Vikas Chandra. 2018. CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs. *arXiv preprint arXiv:1801.06601* (2018). <https://arxiv.org/abs/1801.06601>
- [135] Lorenzo Lamberti, Luca Bompani, Victor Javier Kartsch, Manuele Rusci, Daniele Palossi, and Luca Benini. 2023. Bio-inspired Autonomous Exploration Policies with CNN-based Object Detection on Nano-drones. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE.
- [136] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Nicolas Vasilache, Tobias Grosser, T. Matthew Sullivan, River Riddle, Yuriy O'Donnell, Simon Moll, et al. 2021. MLIR: A Compiler Infrastructure for the End of Moore's Law. *arXiv preprint arXiv:2002.11054* (2021).
- [137] Sam Leroux, Pieter Simoens, Meelis Lootus, Kartik Thakore, and Akshay Sharma. 2022. TinyMLOps: Operational Challenges for Widespread Edge AI Adoption. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 1003–1010. doi:10.1109/IPDPSW55747.2022.00160
- [138] Haiao Li, Lina Ge, and Lei Tian. 2024. Federated Learning, data security and privacy-preserving in edge-Internet of Things. *Artificial Intelligence Review* (2024). doi:10.1007/s10462-024-10774-7
- [139] Peisong Li, Xinheng Wang, Kaizhu Huang, Yi Huang, Shancang Li, and Muddesar Iqbal. 2022. Multi-Model Running Latency Optimization in an Edge Computing Paradigm. *Sensors* 22, 16 (2022). doi:10.3390/s22166097
- [140] Yanyu Li et al. 2022. EfficientFormer: Vision Transformers at MobileNet Speed. *arXiv preprint arXiv:2206.01191* (2022). doi:10.48550/arXiv.2206.01191
- [141] Yuhang Li, Ruihao Gong, Xu Tan, Yang Yang, Peng Hu, Qi Zhang, Fengwei Yu, Wei Wang, and Shi Gu. 2021. BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction. *arXiv preprint arXiv:2102.05426* (2021). doi:10.48550/arXiv.2102.05426
- [142] Yue Li, Hongxia Wang, and Mauro Barni. 2021. A survey of Deep Neural Network watermarking techniques. *Neurocomputing* 461 (2021), 171–193. doi:10.1016/j.neucom.2021.07.051
- [143] Tailin Liang, Huimin Cao, Sheng Sun, and Jian Wu. 2021. Pruning and Quantization for Deep Neural Network Acceleration: A Survey. *arXiv preprint arXiv:2101.09671* (2021).
- [144] Ji Lin, Wei-Ming Chen, Yujun Lin, John Cohn, Chuang Gan, and Song Han. 2020. MCUNet: tiny deep learning on IoT devices. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 982, 12 pages.
- [145] Xiancheng Lin, Rongkai Liu, Jiajie Xie, Qian Wei, Zhi Zhou, Xu Chen, Zhilan Huang, and Gang Lu. 2023. Online Scheduling of CPU-NPU Co-inference for Edge AI Tasks. In *2023 IEEE Wireless Communications and Networking Conference (WCNC)*. 1–6. doi:10.1109/WCNC55385.2023.10118755
- [146] Yun-Wei Lin, Yi-Bing Lin, Ted C.-Y. Chang, and Bo-Xun Lu. 2023. An Edge Transfer Learning Approach for Calibrating Soil Electrical Conductivity Sensors. *Sensors* 23, 21 (2023). doi:10.3390/s23218710
- [147] Neiwien Ling, Xuan Huang, Zhihe Zhao, Nan Guan, Zhenyu Yan, and Guoliang Xing. 2023. BlastNet: Exploiting Duo-Blocks for Cross-Processor Real-Time DNN Inference. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems* (Boston, Massachusetts) (*SenSys '22*). Association for Computing Machinery, New York, NY, USA, 91–105. doi:10.1145/3560905.3568520
- [148] Hao Liu and Zhiping Ni. 2025. Machine Learning for IoT Sensor Data: Enhancing Environmental Sensing with AI Algorithms. Available at SSRN: <https://ssrn.com/abstract=5175807>. <https://ssrn.com/abstract=5175807> SSRN Preprint.
- [149] Shiwei Liu, Tianlong Chen, Xiaohan Chen, Zahra Atashgahi, Lu Yin, Huanyu Kou, Li Shen, Mykola Pechenizkiy, Zhangyang Wang, and Decebal Constantin Mocanu. 2021. Sparse Training via Boosting Pruning Plasticity with Neuroregeneration. In *Advances in Neural Information Processing Systems*, Vol. 34. 9908–9922.
- [150] Shiwei Liu, Tianlong Chen, Xiaohan Chen, Li Shen, Decebal Constantin Mocanu, Zhangyang Wang, and Mykola Pechenizkiy. 2022. The Unreasonable Effectiveness of Random Pruning: Return of the Most Naive Baseline for Sparse Training. *arXiv preprint arXiv:2202.02643* (2022). doi:10.48550/arXiv.2202.02643

- [151] Yizhi Liu, Yao Wang, Ruofei Yu, Mu Li, Vin Sharma, and Yida Wang. 2019. Optimizing CNN Model Inference on CPUs. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 1025–1040. <https://www.usenix.org/conference/atc19/presentation/liu-yizhi>
- [152] Vincenzo Lomonaco et al. 2021. Avalanche: an End-to-End Library for Continual Learning. In *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*. 3595–3605. doi:10.1109/CVPRW53098.2021.00399
- [153] BrainChip Holdings Ltd. 2019. BrainChip Sponsors tinyML Summit and Will Demonstrate Industry-Leading Akida Neuromorphic Processor. <https://www.globenewswire.com/news-release/2019/03/05/1748046/0/en/BrainChip-Sponsors-tinyML-Summit-and-Will-Demonstrate-Industry-Leading-Akida-Neuromorphic-Processor.html> Accessed: 2025-06-12.
- [154] Martin Paul Lücke, Oleksandr Zinenko, William S. Moses, Michel Steuwer, and Albert Cohen. 2025. The MLIR Transform Dialect: Your Compiler Is More Powerful Than You Think. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization (Las Vegas, NV, USA) (CGO '25)*. Association for Computing Machinery, New York, NY, USA, 241–254. doi:10.1145/3696443.3708922
- [155] Daniel Lukats, Oliver Zielinski, Axel Hahn, and Frederic Stahl. 2025. A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *International Journal of Data Science and Analytics* 19, 1 (2025), 1–31. doi:10.1007/s41060-024-00620-y
- [156] Chengtao Lv, Hong Chen, Jinyang Guo, Yifu Ding, and Xianglong Liu. 2024. PTQ4SAM: Post-Training Quantization for Segment Anything. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 15941–15951.
- [157] Ningning Ma, Xiangyu Zhang, Hai-Tao Zheng, and Jian Sun. 2018. ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design. (2018), 116–131. <https://arxiv.org/abs/1807.11164>
- [158] Martina Maggio, Arne Hamann, Eckart Mayer-John, and Dirk Ziegenbein. 2020. Control-System Stability Under Consecutive Deadline Misses Constraints. In *32nd Euromicro Conference on Real-Time Systems (ECRTS 2020) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 166)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 21:1–21:23. doi:10.4230/LIPIcs.ECRTS.2020.21
- [159] Mahmoud Mahgoub, Hassan Moharram, Passent Elkafrawy, and Ahmed Awad. 2023. Benchmarking Concept Drift Detectors for Online Machine Learning. In *Model and Data Engineering*, Philippe Fournier-Viger, Ahmed Hassan, and Ladjel Bellatreche (Eds.). Springer Nature Switzerland, Cham, 43–57.
- [160] Sidi Ahmed Mahmoudi, Maxime Gloesener, Mohamed Benkedadra, and Jean-Sébastien Lerat. 2025. Edge AI System for Real-Time and Explainable Forest Fire Detection Using Compressed Deep Learning Models. In *Proceedings of the 20th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2025) - Volume 3: VISAPP*. SCITEPRESS – Science and Technology Publications, 847–854. doi:10.5220/0013382500003912 CC BY-NC-ND 4.0.
- [161] Yaqoob Majeed, Mike O. Ojo, and Azlan Zahid. 2024. Standalone Edge AI-based solution for Tomato Diseases Detection. *SSRN Electronic Journal* (2024). Preprint. Available at: <https://ssrn.com/abstract=4824801>.
- [162] Asad Waqar Malik, Anis U. Rahman, Arsalan Ahmad, and Max Mauro Dias Santos. 2022. Over-the-Air Software-Defined Vehicle Updates Using Federated Fog Environment. *IEEE Transactions on Network and Service Management* 19, 4 (2022), 5078–5089. doi:10.1109/TNSM.2022.3181027
- [163] Brian McFee, Colin Raffel, Dawen Liang, Daniel P.W. Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. 2015. librosa: Audio and Music Signal Analysis in Python. In *Proceedings of the 14th Python in Science Conference*. 18–25. <https://librosa.org/> Accessed: 9 August 2025.
- [164] Sachin Mehta and Mohammad Rastegari. 2022. MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer. In *Proceedings of the International Conference on Learning Representations (ICLR)*. doi:10.48550/arXiv.2110.02178 Accepted at ICLR 2022.
- [165] Umberto Michieli, Pablo Peso Parada, and Mete Ozay. 2023. Online Continual Learning in Keyword Spotting for Low-Resource Devices via Pooling High-Order Temporal Statistics. In *Proceedings of Interspeech 2023*. 1628–1632. doi:10.21437/Interspeech.2023-90
- [166] Josh Millar, Yushan Huang, Sarab Sethi, Hamed Haddadi, and Anil Madhavapeddy. 2025. Benchmarking Ultra-Low-Power μ NPU. arXiv:2503.22567 [cs.LG] <https://arxiv.org/abs/2503.22567>
- [167] Simon Mittermaier, Ludwig Kürzinger, Bernd Waschneck, and Gerhard Rigoll. 2020. Small-footprint keyword spotting on raw audio data with sinc-convolutions. In *2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 7454–7458.
- [168] Yuki Mori, Yusuke Asai, Kesuke Higashiura, Shin Katayama, Kenta Urano, Takuro Yonezawa, and Nobuo Kawaguchi. 2025. Efficient Edge AI Based Annotation and Detection Framework for Logistics Warehouses. In *2025 IEEE 22nd Consumer Communications & Networking Conference (CCNC)*. 1–4. doi:10.1109/CCNC54725.2025.10976142
- [169] Radouan Ait Mouha. 2021. Internet of Things (IoT). *Journal of Data Analysis and Information Processing* 9, 2 (April 2021), 77–101. doi:10.4236/jdaip.2021.92006
- [170] Thanasis Moustakas and Kostas Kolomvatsos. 2024. Drift-based task management in support of pervasive edge applications. *Internet of Things* 27 (2024), 101277. doi:10.1016/j.iot.2024.101277
- [171] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. 2021. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295* (2021). doi:10.48550/arXiv.2106.08295

- [172] Taraneh Aminosharieh Najafi, José Angel Miranda Calero, Jérôme Thevenot, Benjamin Duc, Stefano Albini, Alireza Amirshahi, Hossein Taji, Maria José Belda Beneyto, Antonio Affanni, and David Atienza. 2024. VersaSens: An Extendable Multimodal Platform for Next-Generation Edge-AI Wearables. *IEEE Transactions on Circuits and Systems for Artificial Intelligence* 1, 1 (2024), 83–96. doi:10.1109/TCASAI.2024.3453809
- [173] Sabuzima Nayak, Ripon Patgiri, Lilapati Waikhom, and Arif Ahmed. 2024. A review on edge analytics: Issues, challenges, opportunities, promises, future directions, and applications. *Digital Communications and Networks* 10, 3 (2024), 783–804. doi:10.1016/j.dcan.2022.10.016
- [174] Bernardino Pinto Neves, Victor D. N. Santos, and António Valente. 2024. Innovative Firmware Update Method to Microcontrollers during Runtime. *Electronics* 13, 7 (2024). doi:10.3390/electronics13071328
- [175] Van Quang Nghiem, Huy Hoang Nguyen, and Minh Son Hoang. 2025. LEAF-YOLO: Lightweight Edge-Real-Time Small Object Detection on Aerial Imagery. *Intelligent Systems with Applications* 25 (2025), 200484. doi:10.1016/j.iswa.2025.200484
- [176] Mao V. Ngo, Tie Luo, and Tony Q. S. Quek. 2021. Adaptive Anomaly Detection for Internet of Things in Hierarchical Edge Computing: A Contextual-Bandit Approach. *ACM Trans. Internet Things* 3, 1, Article 4 (Oct. 2021), 23 pages. doi:10.1145/3480172
- [177] Dinh C. Nguyen, Ming Ding, Pubudu N. Pathirana, and Aruna Seneviratne. 2021. Federated Learning for Internet of Things: A Comprehensive Survey. *IEEE Communications Surveys & Tutorials* 23, 3 (2021), 1622–1658. doi:10.1109/COMST.2021.3075439
- [178] David Nigenda, Zohar Karnin, Muhammad Bilal Zafar, Raghu Ramesha, Alan Tan, Michele Donini, and Krishnaram Kenthapadi. 2022. Amazon SageMaker Model Monitor: A System for Real-Time Insights into Deployed Machine Learning Models. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Washington DC, USA) (KDD '22). Association for Computing Machinery, New York, NY, USA, 3671–3681. doi:10.1145/3534678.3539145
- [179] Noprianto, Nobuo Funabiki, Htoo Htoo Sandi Kyaw, Komang Candra Brata, and I Nyoman Darma Kotama. 2026. A Proposal of Secure and Automated Over-the-Air Firmware Update Mechanism for IoT Devices Using Continuous Integration and Continuous Delivery. *Sensors* 26, 5 (2026). doi:10.3390/s26051535
- [180] Aleksandra Nowak, Bram Grooten, Decebal Constantin Mocanu, and Jacek Tabor. 2023. Fantastic Weights and How to Find Them: Where to Prune in Dynamic Sparse Training. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., 55160–55192. https://proceedings.neurips.cc/paper_files/paper/2023/file/ad02c6f3824f871395112ae71a28eff7-Paper-Conference.pdf
- [181] NVIDIA Developer Blog. 2014. Jetson TK1: A Mobile Supercomputer for Embedded Systems. <https://developer.nvidia.com/blog/jetson-tk1-mobile-embedded-supercomputer-cuda-everywhere/>.
- [182] Nick O'Leary and Dave Conway-Jones. 2013–. Node-RED: Low-code programming for event-driven applications. <https://nodered.org/>. Accessed: Aug. 21, 2025.
- [183] José Oliveira, Pedro Almeida, Pedro Rito, Duarte Raposo, and Susana Sargento. 2024. Over-The-Air updates for Software Defined Vehicle services with IPFS. In *NOMS 2024-2024 IEEE Network Operations and Management Symposium*. 1–9. doi:10.1109/NOMS59830.2024.10575388
- [184] OpenAI. 2022. Whisper: Speech recognition model. <https://github.com/openai/whisper>. Accessed: 2025-07-28.
- [185] Fotios Paparounas, Vasileios Christofas, Petros Amanatidis, Dimitris Karampatzakis, and Thomas Lagkas. 2025. Evaluating Knowledge Distillation and Compression Techniques for Edge Class Devices. In *Proceedings of the 28th Pan-Hellenic Conference on Progress in Computing and Informatics (PCI '24)*. Association for Computing Machinery, New York, NY, USA, 444–449. doi:10.1145/3716554.3716621
- [186] German I. Parisi, Ronald Kemker, Jose L. Part, Christopher Kanan, and Stefan Wermter. 2019. Continual lifelong learning with neural networks: A review. *Neural Networks* 113 (2019), 54–71. doi:10.1016/j.neunet.2019.01.012
- [187] Jeman Park, Misun Yu, Jinse Kwon, Junmo Park, Jemin Lee, and Yongin Kwon. 2024. NEST-C: A deep learning compiler framework for heterogeneous computing systems with artificial intelligence accelerators. *ETRI Journal* 46, 5 (2024), 851–864. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.4218/etrij.2024-0139> doi:10.4218/etrij.2024-0139
- [188] Leandro Antonio Pazmiño Ortiz, Ivonne Fernanda Maldonado Soliz, and Vanessa Katherine Guevara Balarezo. 2025. Advancing TinyML in IoT: A Holistic System-Level Perspective for Resource-Constrained AI. *Future Internet* 17, 6 (2025). doi:10.3390/fi17060257
- [189] Alfredo Petrella, Marco Miozzo, and Paolo Dini. 2024. Mobile Traffic Prediction at the Edge Through Distributed and Deep Transfer Learning. *IEEE Access* 12 (2024), 191288–191303. doi:10.1109/ACCESS.2024.3518483
- [190] Michael Petry, Patrick Gest, Andreas Koch, Max Ghiglione, and Martin Werner. 2023. Accelerated Deep-Learning inference on FPGAs in the Space Domain. In *Proceedings of the 20th ACM International Conference on Computing Frontiers* (Bologna, Italy) (CF '23). Association for Computing Machinery, New York, NY, USA, 222–228. doi:10.1145/3587135.3592763
- [191] Davy Preuveneers, Willem Verheyen, Sander Joos, and Wouter Joosen. 2023. On the adversarial robustness of full integer quantized TinyML models at the edge. In *Proceedings of the 2nd International Workshop on Middleware for the Edge* (Bologna, Italy) (MiddleWEdge '23). Association for Computing Machinery, New York, NY, USA, 7–12. doi:10.1145/3630180.3631201
- [192] Jiby J. Puthiyadam, Shelbi Joseph, and Bharat Bhushan. 2024. Temporal ECDSA: A timestamp and signature mask enabled ECDSA algorithm for IoT client node authentication. *Computer Communications* 216 (2024), 307–323. doi:10.1016/j.comcom.2024.01.016
- [193] PyTorch Developers. 2024. PyTorch Mobile: Deploying PyTorch Models on Mobile Devices. https://pytorch.org/mobile/?__hstc=114018008.2f3f33a24b44870ec4a577029c49e44b.1726790400146.1726790400147.1726790400148.1&__hssc=114018008.1.

- 1726790400149&__hsfp=451136374
- [194] PyTorch Team. 2023. ExecuTorch: PyTorch Edge Runtime. <https://github.com/pytorch/executorch>.
 - [195] Qeexo. 2024. Qeexo AutoML Documentation. <https://docs.qeexo.com/guides/userguides/getting-started>
 - [196] Mahfuzur Rahman and Bashir I. Morshed. 2025. Resource-Constrained On-Chip AI Classifier for Beat-by-Beat Real-Time Arrhythmia Detection with an ECG Wearable System. *Electronics* 14, 13 (2025). doi:10.3390/electronics14132654
 - [197] Anirudh Raju, Sankaran Panchapagesan, Xing Liu, Arindam Mandal, and Nikko Strom. 2018. Data Augmentation for Robust Keyword Spotting under Playback Interference. *arXiv preprint arXiv:1808.00563* (2018). doi:10.48550/arXiv.1808.00563
 - [198] Reshawn Ramjattan, Rajeev Ratan, Shiva Ramoudith, Patrick Hosein, and Daniele Mazzei. 2023. Using Continual Learning on Edge Devices for Cost-Effective, Efficient License Plate Detection. In *Proceedings of the 18th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP 2023) - Volume 5: VISAPP*. INSTICC, SciTePress, 544–550. doi:10.5220/0011656300003417
 - [199] Partha Pratim Ray. 2022. A review on TinyML: State-of-the-art and prospects. *Journal of King Saud University - Computer and Information Sciences* 34, 4 (2022), 1595–1623. doi:10.1016/j.jksuci.2021.11.019
 - [200] Wamiq Raza, Anas Osman, Francesco Ferrini, and Francesco De Natale. 2021. Energy-Efficient Inference on the Edge Exploiting TinyML Capabilities for UAVs. *Drones* 5, 4 (2021). doi:10.3390/drones5040127
 - [201] Vijay Janapa Reddi et al. 2020. MLPerf Inference Benchmark. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. 446–459. doi:10.1109/ISCA45697.2020.00045
 - [202] V. J. Reddi, D. Kanter, P. Mattson, J. Duke, T. Nguyen, R. Chukka, K. Shiring, K.-S. Tan, M. Charlebois, W. Chou, M. El-Khamy, J. Hong, T. St. John, C. Trinh, M. Buch, M. Mazumder, R. Markovic, T. Atta, F. Cakir, M. Charkhabi, X. Chen, C.-M. Chiang, D. Dexter, T. Heo, G. Schmuelling, M. Shabani, and D. Zika. 2022. MLPerf Mobile Inference Benchmark: An Industry-Standard Open-Source Machine Learning Benchmark for On-Device AI. In *Proc. 5th MLSys Conf.* Santa Clara, CA, USA.
 - [203] Jared Roesch, Steven Lyubomirsky, Logan Weber, Josh Pollock, Marisa Kirsame, Tianqi Chen, and Zachary Tatlock. 2018. Relay: a new IR for machine learning frameworks. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages* (Philadelphia, PA, USA) (MAPL 2018). Association for Computing Machinery, New York, NY, USA, 58–68. doi:10.1145/3211346.3211348
 - [204] Diletta Romano, Luca Mottola, and Thiemo Voigt. 2026. Neuro-C: Neural Inference Shaped by Hardware Limits. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys '26)*. Edinburgh, Scotland, UK. Accepted for publication; to appear.
 - [205] Daniel Rosendo, Alexandru Costan, Patrick Valduriez, and Gabriel Antoniu. 2022. Distributed intelligence on the Edge-to-Cloud Continuum: A systematic literature review. *J. Parallel and Distrib. Comput.* 166 (2022), 71–94. doi:10.1016/j.jpdc.2022.04.004
 - [206] Ning Ruan, Kun Li, Qiyang Zhang, Lauri Loven, Praveen Kumar Donta, Yi Jia, and Schahram Dustdar. 2025. Edge AI for Earth Observation. *IEEE Internet Computing* (2025), 1–10. doi:10.1109/MIC.2025.3587325
 - [207] David Sachs, Tiezhen Wang, and Pete Warden. 2020. TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems. In *Proceedings of Machine Learning on the Phone and other Consumer Devices (MLPCD) Workshop at ICLR*. <https://arxiv.org/abs/2010.08678>
 - [208] Shaibal Saha and Lanyu Xu. 2025. Vision transformers on the edge: A comprehensive survey of model compression and acceleration strategies. *Neurocomputing* 643 (Aug. 2025), 130417. doi:10.1016/j.neucom.2025.130417
 - [209] Swapnil Sayan Saha, Sandeep Singh Sandha, and Mani Srivastava. 2022. Machine Learning for Microcontroller-Class Hardware: A Review. *IEEE Sensors Journal* 22, 22 (2022), 21362–21390. doi:10.1109/JSEN.2022.3210773
 - [210] Guilherme Yukio Sakurai, Jessica Fernandes Lopes, Bruno Bogaz Zarpelão, and Sylvio Barbon Junior. 2023. Benchmarking Change Detector Algorithms from Different Concept Drift Perspectives. *Future Internet* 15, 5 (2023). doi:10.3390/fi15050169
 - [211] Ayodeji Olalekan Salau and Shruti Jain. 2019. Feature Extraction: A Survey of the Types, Techniques, Applications. In *2019 International Conference on Signal Processing and Communication (ICSC)*. 158–164. doi:10.1109/ICSC45622.2019.8938371
 - [212] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. MobileNetV2: Inverted Residuals and Linear Bottlenecks. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4510–4520. doi:10.1109/CVPR.2018.00474
 - [213] Pongpanod Sankosik and Chotirat Ratanamahatana. 2024. Data Augmentation Strategies for Improving Time Series Classification Accuracy. In *Proceedings of the 2024 7th International Conference on Robot Systems and Applications (ICRSA '24)*. Association for Computing Machinery, New York, NY, USA, 51–58. doi:10.1145/3702468.3702479
 - [214] Ranjan Sapkota, Yang Cao, Konstantinos I. Roumeliotis, and Manoj Karkee. 2025. Vision-Language-Action Models: Concepts, Progress, Applications and Challenges. *arXiv:2505.04769 [cs.CV]* <https://arxiv.org/abs/2505.04769>
 - [215] A. Sathya, D. Angel, M. Iswarya, R. Poonkodi, Kandavalli Michael Angelo, and Nirmala Priyadharshini P. 2025. IoT Enabled Healthcare Framework Using Edge AI and Advanced Wearable Sensors for Real Time Health Monitoring. In *2025 International Conference on Multi-Agent Systems for Collaborative Intelligence (ICMSCI)*. 384–392. doi:10.1109/ICMSCI62561.2025.10894492
 - [216] Mahadev Satyanarayanan. 2017. The Emergence of Edge Computing. *Computer* 50, 1 (2017), 30–39. doi:10.1109/MC.2017.9
 - [217] C. Savard, N. Manganelli, B. Holzman, et al. 2024. Optimizing High-Throughput Inference on Graph Neural Networks at Shared Computing Facilities with the NVIDIA Triton Inference Server. *Computing and Software for Big Science* 8, 14 (2024). doi:10.1007/s41781-024-00123-2

- [218] B. Schilit, N. Adams, and R. Want. 1994. Context-Aware Computing Applications. In *1994 First Workshop on Mobile Computing Systems and Applications*. 85–90. doi:10.1109/WMCSA.1994.16
- [219] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in Machine learning systems. In *Proceedings of the 29th International Conference on Neural Information Processing Systems - Volume 2 (Montreal, Canada) (NIPS'15)*. MIT Press, Cambridge, MA, USA, 2503–2511.
- [220] Boris Sekachev, Nikita Manovich, Maxim Zhiltsov, Andrey Zhavoronkov, Dmitry Kalinin, Ben Hoff, TOSmanov, Dmitry Kruchinin, Artyom Zankevich, DmitriySidnev, Maksim Markelov, Johannes222, Mathis Chenuet, a andre, telenachos, Aleksandr Melnikov, Jijoong Kim, Liron Ilouz, Nikita Glazov, Priya4607, Rush Tehrani, Seungwon Jeong, Vladimir Skubriev, Sebastian Yonekura, vugia truong, zliang7, lizhming, and Tritin Truong. 2020. *opencv/cvat: v1.1.0*. doi:10.5281/zenodo.4009388
- [221] Kah Phooi Seng and Li-Minn Ang. 2022. Embedded Intelligence: State-of-the-Art and Research Challenges. *IEEE Access* 10 (2022), 59236–59258. doi:10.1109/ACCESS.2022.3175574
- [222] Weisong Shi, Jie Cao, Quan Zhang, Youhui Li, and Lanyu Xu. 2016. Edge Computing: Vision and Challenges. *IEEE Internet of Things Journal* 3, 5 (2016), 637–646. doi:10.1109/JIOT.2016.2579198
- [223] Yining Shi et al. 2023. Welder: Scheduling Deep Learning Memory Access via Tile-graph. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 701–718. <https://www.usenix.org/conference/osdi23/presentation/shi>
- [224] Ali Shoker, Fernando Alves, and Paulo Esteves-Verissimo. 2023. ScaIOTA: Scalable Secure Over-the-Air Software Updates for Vehicles. In *2023 42nd International Symposium on Reliable Distributed Systems (SRDS)*. 151–161. doi:10.1109/SRDS60354.2023.00024
- [225] Shakhrol Iman Siam, Hyunho Ahn, Li Liu, Samiul Alam, Hui Shen, Zhichao Cao, Ness Shroff, Bhaskar Krishnamachari, Mani Srivastava, and Mi Zhang. 2025. Artificial Intelligence of Things: A Survey. *ACM Trans. Sen. Netw.* 21, 1, Article 9 (Jan. 2025), 75 pages. doi:10.1145/3690639
- [226] Raghubir Singh et al. 2023. Edge AI: A survey. *Internet of Things and Cyber-Physical Systems* 3 (2023), 71–92. doi:10.1016/j.iotcps.2023.02.004
- [227] Jayden Renee Sorensen, Syed Aftab Rashid, Hossam ElHussini, and Andrei-Marian Dan. 2025. Evaluating Differential Firmware Updates for Embedded IoT Device Fleets. In *2025 IEEE 21st International Conference on Factory Communication Systems (WFCS)*. 1–8. doi:10.1109/WFCS63373.2025.11077647
- [228] Bozhidar Stevanoski, Ana Kostovska, Panče Panov, and Sašo Džeroski. 2024. Change detection and adaptation in multi-target regression on data streams. *Machine Learning* 113 (Oct 2024), 8585–8622. doi:10.1007/s10994-024-06621-z
- [229] STMicroelectronics. 2025. *STM32 AI Model Zoo*. <https://github.com/STMicroelectronics/stm32ai-modelzoo> Accessed: Oct. 5, 2025.
- [230] STMicroelectronics. 2025. *STM32Cube.AI – Machine Learning Toolkit for STM32 Microcontrollers*. <https://stm32ai.st.com/stm32-cube-ai/> Accessed: Oct. 5, 2025.
- [231] Bharath Sudharsan, John G. Breslin, Mehreen Tahir, Muhammad Intizar Ali, Omer Rana, Schahram Dustdar, and Rajiv Ranjan. 2022. OTA-TinyML: Over the Air Deployment of TinyML Models and Execution on IoT Devices. *IEEE Internet Computing* 26, 3 (2022), 69–78. doi:10.1109/MIC.2021.3133552
- [232] Chellammal Surianarayanan, John Jeyasekaran Lawrence, Pethuru Raj Chelliah, Edmond Prakash, and Chaminda Hewage. 2023. A Survey on Optimization Techniques for Edge Artificial Intelligence (AI). *Sensors* 23, 3 (2023). doi:10.3390/s23031279
- [233] Swati Swati et al. 2025. Exploring Quantization Approaches for Optimized Training and Inference for Edge AI Applications. In *2025 11th International Conference on Communication and Signal Processing (ICCSP)*. 1362–1367. doi:10.1109/ICCSP64183.2025.11088699
- [234] Hossein Taji, José Miranda, Miguel Peón-Quirós, and David Atienza. 2025. MEDEA: A Design-Time Multi-Objective Manager for Energy-Efficient DNN Inference on Heterogeneous Ultra-Low Power Platforms. arXiv:2506.19067 [cs.AR] <https://arxiv.org/abs/2506.19067>
- [235] Tarik Taleb et al. 2017. On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration. *IEEE Communications Surveys & Tutorials* 19, 3 (2017), 1657–1681. doi:10.1109/COMST.2017.2705720
- [236] Mingxing Tan, Bo Chen, Ruoming Pang, Vijay Vasudevan, Mark Sandler, Andrew Howard, and Quoc V. Le. 2019. MnasNet: Platform-Aware Neural Architecture Search for Mobile. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2815–2823. doi:10.1109/CVPR.2019.00293
- [237] Mingxing Tan and Quoc V. Le. 2019. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. arXiv:1905.11946 [cs.LG] <https://doi.org/10.48550/arXiv.1905.11946>
- [238] Mingxing Tan, Ruoming Pang, and Quoc V. Le. 2020. EfficientDet: Scalable and Efficient Object Detection. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 10778–10787. doi:10.1109/CVPR42600.2020.01079
- [239] Arvin Tashakori, Wenwen Zhang, Z. Jane Wang, and Peyman Servati. 2023. SemiPFL: Personalized Semi-Supervised Federated Learning Framework for Edge Intelligence. *IEEE Internet of Things Journal* 10, 10 (2023), 9161–9176. doi:10.1109/JIOT.2022.3233599
- [240] Sooraj George Thomas and Praveen Kumar Myakala. 2025. Beyond the Cloud: Federated Learning and Edge AI for the Next Decade. *Journal of Computer and Communications* 13, 2 (2025), 37–50. doi:10.4236/jcc.2025.132004
- [241] Tinygrad Contributors. 2024. Tinygrad: Open-Source Deep Learning Framework. <https://github.com/tinygrad/tinygrad>. Accessed: 2025-10-07.

- [242] TinyML Deployment Working Group. 2023. Deploying TinyML Models at Scale: Insights on Monitoring and Lifecycle Management. https://cms.tinyml.org/wp-content/uploads/ew2023/tinyML_Deployment_WG_White_Paper_1.pdf
- [243] Maxim Tkachenko, Mikhail Malyuk, Andrey Holmanyuk, and Nikolai Liubimov. 2020-2025. Label Studio: Data labeling software. <https://github.com/HumanSignal/label-studio> Open source software available from <https://github.com/HumanSignal/label-studio>.
- [244] Qiang Tong, Jinrui Wang, Wenshuang Yang, Songtao Wu, Wenqi Zhang, Chen Sun, and Kuanhong Xu. 2024. Edge AI-enabled chicken health detection based on enhanced FCOS-Lite and knowledge distillation. *Computers and Electronics in Agriculture* 226 (2024), 109432. doi:10.1016/j.compag.2024.109432
- [245] Peter Torelli and Mohit Bangale. [n. d.]. *Measuring Inference Performance of Machine-Learning Frameworks on Edge-class Devices with the MLMark™ Benchmark*. Technical Report. EEMBC. <https://www.eembc.org/techlit/articles/MLMARK-WHITEPAPER-FINAL-1.pdf>
- [246] Antonio Torralba and Alexei A. Efros. 2011. Unbiased look at dataset bias. In *CVPR 2011*. 1521–1528. doi:10.1109/CVPR.2011.5995347
- [247] Jingke Tu, Lei Yang, and Jiannong Cao. 2025. Distributed Machine Learning in Edge Computing: Challenges, Solutions and Future Directions. *ACM Comput. Surv.* 57, 5, Article 132 (Jan. 2025), 37 pages. doi:10.1145/3708495
- [248] Xiaolong Tu et al. 2024. Unveiling Energy Efficiency in Deep Learning: Measurement, Prediction, and Scoring across Edge Devices. In *Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing (SEC '23)*. Association for Computing Machinery, New York, NY, USA, 80–93. doi:10.1145/3583740.3628442
- [249] Ihsan UI Haq, Rafia Mumtaz, Muhammad Talha, Zunaira Shafaq, and Muhammad Owais. 2022. Wheat Rust Disease Classification using Edge-AI. In *2022 2nd International Conference on Artificial Intelligence (ICAI)*. 58–63. doi:10.1109/ICAI55435.2022.9773489
- [250] Mehmet Gorkem Ulkar and Osman Erman Okman. 2021. Ultra-Low Power Keyword Spotting at the Edge. *arXiv preprint arXiv:2111.04988* (2021). <https://arxiv.org/abs/2111.04988>
- [251] J. P. van der Vorst, J. M. Smit, D. van de Sande, B. van der Ster, F. Daams, R. Schasfoort, et al. 2025. Importance of model governance in clinical AI models: case study on the relevance of data drift detection. *BMJ Digital Health & AI* 1 (2025), e000046. doi:10.1136/bmjdhai-2025-000046
- [252] Ao Wang, Hui Chen, Lihao Liu, Kai Chen, Zijia Lin, Jungong Han, and Guiguang Ding. 2024. YOLOv10: Real-Time End-to-End Object Detection. *arXiv:2405.14458 [cs.CV]* doi:10.48550/arXiv.2405.14458
- [253] Mingzi Wang et al. 2025. JAQ: joint efficient architecture design and low-bit quantization with hardware-software co-exploration. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence and Thirty-Seventh Conference on Innovative Applications of Artificial Intelligence and Fifteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'25/IAAI'25/EAAI'25)*. AAAI Press. doi:10.1609/aaai.v39i20.35415
- [254] Shibo Wang, Shusen Yang, and Cong Zhao. 2020. SurveilEdge: Real-time Video Query based on Collaborative Cloud-Edge Deep Learning. In *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications*. 2519–2528. doi:10.1109/INFOCOM41043.2020.9155284
- [255] Xiaofei Wang, Yiwen Han, Chenyang Wang, Qiyang Zhao, Xu Chen, and Min Chen. 2019. In-Edge AI: Intelligentizing Mobile Edge Computing, Caching and Communication by Federated Learning. *IEEE Network* 33, 5 (2019), 156–165. doi:10.1109/MNET.2019.1800286
- [256] Xiaobo Wang and Wei Jia. 2025. Optimizing Edge AI: A Comprehensive Survey on Data, Model, and System Strategies. *arXiv preprint arXiv:2501.03265* (2025). <https://arxiv.org/abs/2501.03265>
- [257] Xubin Wang, Zhiqing Tang, Jianxiong Guo, Tianhui Meng, Chenhao Wang, Tian Wang, and Weijia Jia. 2025. Empowering Edge Intelligence: A Comprehensive Survey on On-Device AI Models. *ACM Comput. Surv.* 57, 9, Article 228 (April 2025), 39 pages. doi:10.1145/3724420
- [258] Xiaojie Wang, Beibei Wang, Yu Wu, Zhaolong Ning, Song Guo, and Fei Richard Yu. 2025. A Survey on Trustworthy Edge Intelligence: From Security and Reliability to Transparency and Sustainability. *IEEE Communications Surveys & Tutorials* 27, 3 (2025), 1729–1757. doi:10.1109/COMST.2024.3446585
- [259] Zi Wang, Fei Wu, Feng Yu, Yurui Zhou, Jia Hu, and Geyong Min. 2024. Federated Continual Learning for Edge-AI: A Comprehensive Survey. *arXiv:2411.13740 [cs.LG]* <https://arxiv.org/abs/2411.13740>
- [260] Wei Wei, Jishnu Banerjee, Sahidul Islam, Chen Pan, and Mimi Xie. 2024. Energy-aware Incremental OTA Update for Flash-based Batteryless IoT Devices. In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*. 51–56. doi:10.1109/ISVLSI61997.2024.00021
- [261] Mark Weiser. 1999. The computer for the 21st century. *SIGMOBILE Mob. Comput. Commun. Rev.* 3, 3 (July 1999), 3–11. doi:10.1145/329124.329126
- [262] Bichen Wu et al. 2018. FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search. *arXiv preprint arXiv:1812.03443* (2018). doi:10.48550/arXiv.1812.03443
- [263] Yuhao Wu et al. 2024. Your Firmware Has Arrived: A Study of Firmware Update Vulnerabilities. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 5627–5644. <https://www.usenix.org/conference/usenixsecurity24/presentation/wu-yuhao>
- [264] Yirui Wu, Lilai Zhang, Zonghua Gu, Hu Lu, and Shaohua Wan. 2023. Edge-AI-Driven Framework with Efficient Mobile Network Design for Facial Expression Recognition. 22, 3, Article 57 (April 2023), 17 pages. doi:10.1145/3587038
- [265] Bingnan Xiao, Xichen Yu, Wei Ni, Xin Wang, and H. Vincent Poor. 2025. Over-the-air federated learning: Status quo, open challenges, and future directions. *Fundamental Research* 5, 4 (2025), 1710–1724. doi:10.1016/j.fmre.2024.01.011

- [266] Guangxuan Xiao et al. 2023. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research)*. PMLR. <https://proceedings.mlr.press/v202/xiao23a.html>
- [267] Dianlei Xu, Tong Li, Yong Li, Xiang Su, Sasu Tarkoma, Tao Jiang, Jon Crowcroft, and Pan Hui. 2020. Edge Intelligence: Architectures, Challenges, and Applications. *arXiv preprint arXiv:2003.12172* (2020). doi:10.48550/arXiv.2003.12172
- [268] Mengwei Xu, Feng Qian, Mengze Zhu, Feifan Huang, Saumay Pushp, and Xuanzhe Liu. 2020. DeepWear: Adaptive Local Offloading for On-Wearable Deep Learning. *IEEE Transactions on Mobile Computing* 19, 2 (2020), 314–330. doi:10.1109/TMC.2019.2893250
- [269] Takeya Yamada and Hiroki Matsutani. 2023. A Lightweight Concept Drift Detection Method for On-Device Learning on Resource-Limited Edge Devices. *2023 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (2023).
- [270] Xiaofei Yang, Enrique del Rey Castillo, Yang Zou, and Liam Wotherspoon. 2024. UAV-deployed deep learning network for real-time multi-class damage detection using model quantization techniques. *Automation in Construction* 159 (2024), 105254. doi:10.1016/j.autcon.2023.105254
- [271] Zhewei Yao, Cheng Li, Xiaoxia Wu, Stephen Youn, and Yuxiong He. 2023. A Comprehensive Study on Post-Training Quantization for Large Language Models. *arXiv preprint arXiv:2303.08302* (2023). doi:10.48550/arXiv.2303.08302
- [272] Achyut Yaragal and Kirankumar Bendigeri. 2024. Data Aggregation in Internet of Things (DAIT) with Edge Computing. In *2024 5th International Conference on Electronics and Sustainable Communication Systems (ICESC)*. 472–479. doi:10.1109/ICESC60852.2024.10690020
- [273] Naoto Yoshida, Hina Miura, Takashi Matsutani, and Hideto Motomura. 2022. Data Augmentation for Edge-AI on-chip Learning. In *2022 IEEE 8th World Forum on Internet of Things (WF-IoT)*. 1–6. doi:10.1109/WF-IoT54382.2022.10152266
- [274] Koen Zandberg, Kaspar Schleiser, Francisco Acosta, Hannes Tschofenig, and Emmanuel Baccelli. 2019. Secure Firmware Updates for Constrained IoT Devices Using Open Standards: A Reality Check. *IEEE Access* 7 (2019), 71907–71920. doi:10.1109/ACCESS.2019.2919760
- [275] Liekang Zeng, Xu Chen, Zhi Zhou, Lei Yang, and Junshan Zhang. 2021. CoEdge: Cooperative DNN Inference With Adaptive Workload Partitioning Over Heterogeneous Edge Devices. *IEEE/ACM Transactions on Networking* 29, 2 (2021), 595–608. doi:10.1109/TNET.2020.3042320
- [276] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2017. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. *arXiv preprint arXiv:1707.01083* (2017). doi:10.48550/arXiv.1707.01083
- [277] Yundong Zhang et al. 2017. Hello Edge: Keyword Spotting on Microcontrollers. *arXiv preprint arXiv:1711.07128* (2017).
- [278] Ziyang Zhang, Yang Zhao, Huan Li, Changyao Lin, and Jie Liu. 2024. DVFO: Learning-Based DVFS for Energy-Efficient Edge-Cloud Collaborative Inference. *IEEE Transactions on Mobile Computing* 23, 10 (2024), 9042–9059. doi:10.1109/TMC.2024.3357218
- [279] Bowen Zhao, Huanlai Xing, Lexi Xu, Yang Li, Li Feng, Jincheng Peng, and Zhiwen Xiao. 2024. On Forecasting-Oriented Time Series Transmission: A Federated Semantic Communication System. *IEEE Transactions on Mobile Computing* 23, 12 (2024), 13728–13744. doi:10.1109/TMC.2024.3438285
- [280] Xiaotian Zhao, Ruge Xu, and Xinfei Guo. 2023. Post-training Quantization or Quantization-aware Training? That is the Question. In *2023 China Semiconductor Technology International Conference (CSTIC)*. 1–3. doi:10.1109/CSTIC58779.2023.10219214
- [281] Yifan Zhao et al. 2024. Felix: Optimizing Tensor Programs with Gradient Descent. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*. Association for Computing Machinery. doi:10.1145/3620666.3651348
- [282] Dongqi Zheng. 2025. Diffusion Models on the Edge: Challenges, Optimizations, and Applications. *arXiv:2504.15298 [cs.DC]* <https://arxiv.org/abs/2504.15298>
- [283] Lianmin Zheng et al. 2020. Anso: Generating High-Performance Tensor Programs for Deep Learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association, 863–879. <https://www.usenix.org/conference/osdi20/presentation/zheng>
- [284] Zhen Zheng et al. 2023. BladeDISC: Optimizing Dynamic Shape Machine Learning Workloads via Compiler Approach. *Proc. ACM Manag. Data* 1, 3, Article 206 (Nov. 2023), 29 pages. doi:10.1145/3617327
- [285] Wen Zhou et al. 2024. Bubble feature extraction in subcooled flow boiling using AI-based object detection and tracking techniques. *International Journal of Heat and Mass Transfer* 222 (2024), 125188. doi:10.1016/j.ijheatmasstransfer.2024.125188
- [286] Zhi Zhou, Xu Chen, En Li, Liekang Zeng, Ke Luo, and Junshan Zhang. 2019. Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing. *Proc. IEEE* 107, 8 (2019), 1738–1762. doi:10.1109/JPROC.2019.2918951
- [287] Shuai Zhu, Thimo Voigt, Fatemeh Rahimian, and Jeonggil Ko. 2024. On-device training: A first overview on existing systems. *ACM transactions on sensor networks* 20, 6 (2024), 1–39.

Received 21 October 2025; revised 30 May 2026; accepted 16 June 2026